

Vue 3 + Nuxt — Zero to Hero (o'zbek tilida)

Bu qo'llanma **Vue 3 (Composition API)** va **Nuxt 3/4** ni noldan professional darajagacha o'rgatish uchun yozilgan. Har bir modul nazariya + `why` (nega shunday) + **kamida 20 ta masala** dan iborat.

Sen backend (Laravel/DDD) tajribasi bor odamsan, shuning uchun ba'zi joylarda **Laravel/PHP analogiyalari** beriladi — yangi tushunchani eski bilim ustiga "ulash" eng tez o'rganish usuli.

Qanday ishlatish kerak

- Modullarni **tartib bilan** o'q (01 → 02 → ...). Har biri oldingisiga tayanadi.
- Har modul oxiridagi masalalarni **o'zing yoz**, keyin yechimga qara. Faqat o'qib ketsang — 2 haftada unutasan.
- Har modul uchun alohida `playground` papka ochib, kod yozib ko'r:

```
npm create vue@latest playground
```



- Masalalarning ☒ belgisi bor yechimi shu faylning oxirida yoki keyingi qatorda beriladi. Belgisiz masalalar — mustaqil mashq (yechimi yo'q, ataylab).

Talablar (prerequisites)

Kerak	Daraja
JavaScript (ES6+): <code>let/const</code> , arrow fn, destructuring, modules, Promise/async-await, array methods (<code>map/filter/reduce</code>)	Yaxshi bilish shart
HTML/CSS	O'rta
Terminal, npm	Asoslar
TypeScript	Shart emas, lekin foydali — yo'l-yo'lakay o'rganamiz

Agar JS'ning yuqoridagilarini yaxshi bilmasang — avval o'shani mustahkamla, aks holda Vue'da qiynalasan. Vue qiyin emas, JS zaif bo'lsa qiyin tuyuladi.

To'liq dastur (roadmap)

I qism — Vue 3 yadrosi

Nº	Modul	Nima o'rganasan
01	Vue asoslari va template syntax	Vue nima, SFC, Vite, direktivalar, <code>v-bind/v-if/v-for/v-on</code>
02	Reactivity	<code>ref</code> , <code>reactive</code> , <code>computed</code> , <code>watch</code> , <code>watchEffect</code> , reaktivlik ichki mexanizmi
03	Komponentlar	<code>props</code> , <code>emits</code> , <code>slots</code> , <code>v-model</code> , <code>provide/inject</code>
04	Composition API & Composables	Lifecycle, composable yozish, advanced reactivity, <code><script setup></code> chuqur
05	Vue Router	Routing, nested routes, guards, lazy loading
06	Pinia (state management)	Store, actions, getters, plugins, persist

II qism — Nuxt

Nº	Modul	Nima o'rganasan
07	Nuxt asoslari	Setup, file-based routing, layouts, auto-imports, <code>app.vue</code>
08	Data fetching & Server (Nitro)	<code>useFetch</code> , <code>useAsyncData</code> , <code>\$fetch</code> , server API, middleware, <code>useState</code>

III qism — Nuxt UI (komponentlar kutubxonasi)

Nº	Modul	Nima o'rganasan
09	Nuxt UI: asoslar va theming	O'rnatish, <code>UApp</code> , design system (color/variant/size), 3 qatlamli theming, <code>:ui</code> prop
10	Nuxt UI komponentlari (to'liq)	125+ komponent kategoriya bo'yicha: form (Zod), table (TanStack), overlay (<code>useOverlay</code>), dashboard

Nuxt UI **v4** — Nuxt UI va Pro birlashgan, butunlay bepul (`@nuxt/ui`), Reka UI + Tailwind v4 asosida. 09 — tizimni tushunish, 10 — har bir komponent.

Yadro to'plam (Vue 100% + Nuxt'ning eng muhim ~70% + Nuxt UI). Hali yozilmagan, so'rasang qo'shiladigan mavzular: rendering modes (SSR/SSG/ISR/hybrid), SEO/meta chuqur, Nitro chuqur (storage/caching), modules ecosystem, testing (Vitest), deployment (VPS/Nginx/Docker), TypeScript chuqur, performance — barchasi shu modullar uslubida.

Vue vs React — bir og'iz (kontekst uchun)

Vue va React bir muammoni hal qiladi: **UI = f(state)**. Farqi:

- **React** — "hammasi JavaScript". JSX, manual re-render optimizatsiya (`useMemo`, `useCallback`).
- **Vue** — template + **avtomatik reaktivlik** (Proxy asosida). Ko'p narsa "o'zi" optimallasadi. Kamroq boilerplate.

Laravel termini bilan: React — "hamma narsani qo'lda yig'asan", Vue — "framework ko'p ishni convention bilan o'zi qiladi" (Laravel'ning "convention over configuration" falsafasiga yaqin).

Vue ekotizimi xaritasi

```

Vue 3 (core)
├─ Vue Router      → sahifalar orasida navigatsiya (SPA)
├─ Pinia           → global state (Vuex'ning vorisi)
├─ Vite            → build tool / dev server (Webpack vorisi, juda tez)
└─ Nuxt            → "Vue uchun Laravel" - fullstack meta-framework
  
```

(SSR, routing, server API, auto-import hammasi
tayyor)

Tayyor bo'lsang — [01-vue-asoslari.md](#) dan boshla.

01 — Vue asoslari va Template Syntax

[🏠 README / Mundarija](#) · [Keyingi: 02 — Reactivity](#) ➔

Vue nima va nega kerak?

Klassik yondashuvda (jQuery/vanilla JS) sen DOM'ni **qo'lda** o'zgartirasan:

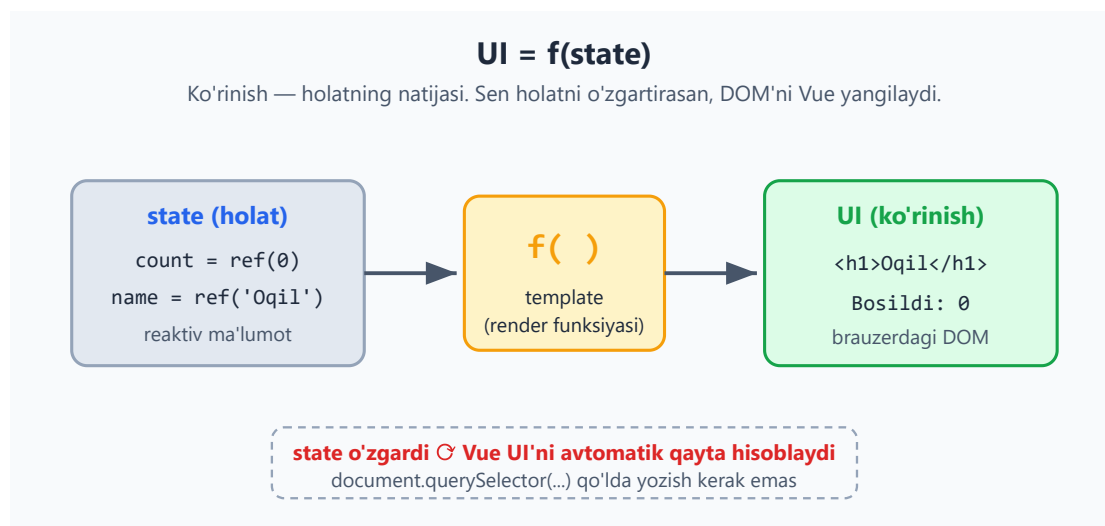
`document.querySelector(...).textContent = ...`. Ma'lumot ko'payganda bu kod chalkashadi, bug ko'payadi.

Vue **deklarativ**: sen "natija qanday ko'rinishi kerak"ligini aytasan, DOM'ni yangilashni Vue o'zi qiladi.

```
UI = f(state)
```

State (ma'lumot) o'zgardi → Vue kerakli DOM qismini avtomatik yangilaydi. Sen DOM'ga tegmaysan.

Quyidagi diagramma bu g'oyani ko'rsatadi: ko'rinish — holatning natijasi.



Laravel analogiyasi: Blade'da `{{ $user->name }}` yozasan, controller'dan kelgan data render bo'ladi — lekin Blade *bir marta* server'da render bo'lib tugaydi. Vue esa brauzerda *uzluksiz* ishlaydi: `name` o'zgarsa, sahifa qayta yuklanmasdan ekrandagi qiymat yangilanadi. Bu — reaktivlik.

1.1 Birinchi Vue ilovasi (CDN — tushunish uchun)

Build tool'siz, faqat tushunish uchun:

```
<!DOCTYPE html>
<html>
<head>
  <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
</head>
<body>
  <div id="app">
    <h1>{{ message }}</h1>
    <button @click="count++">Bosildi: {{ count }}</button>
  </div>

  <script>
    const { createApp, ref } = Vue

    createApp({
      setup() {
        const message = ref('Salom Vue!')
        const count = ref(0)
        return { message, count } // template'ga ochiladi
      }
    }).mount('#app')
  </script>
</body>
</html>
```

Tushunchalar: - `createApp(...)` — ilova instansiyasi (Laravel'da `$app` kabi root). - `.mount('#app')` — ilovani DOM'dagi `#app` elementiga "ulaydi". - `ref(0)` — **reaktiv** qiymat. `count.value` o'zgarsa, template yangilanadi. - `{{ }}` — interpolation (Blade'dagi `{{ }}` bilan deyarli bir xil ko'rinish). - `@click` — event listener (pastda batafsil).

CDN faqat o'rganish uchun. **Real loyihada** Vite + SFC ishlatamiz.

1.2 Real setup: Vite + SFC

```
npm create vue@latest
# Project name: my-app
# TypeScript? - Yes (tavsiya, lekin xohlasang No)
# Router/Pinia/Vitest - keyin qo'shamiz, hozir No
cd my-app
npm install
npm run dev
```

SFC (Single File Component) — .vue fayl

Bitta faylda 3 ta blok: **template**, **logika**, **stil**:

```
<script setup>
import { ref } from 'vue'

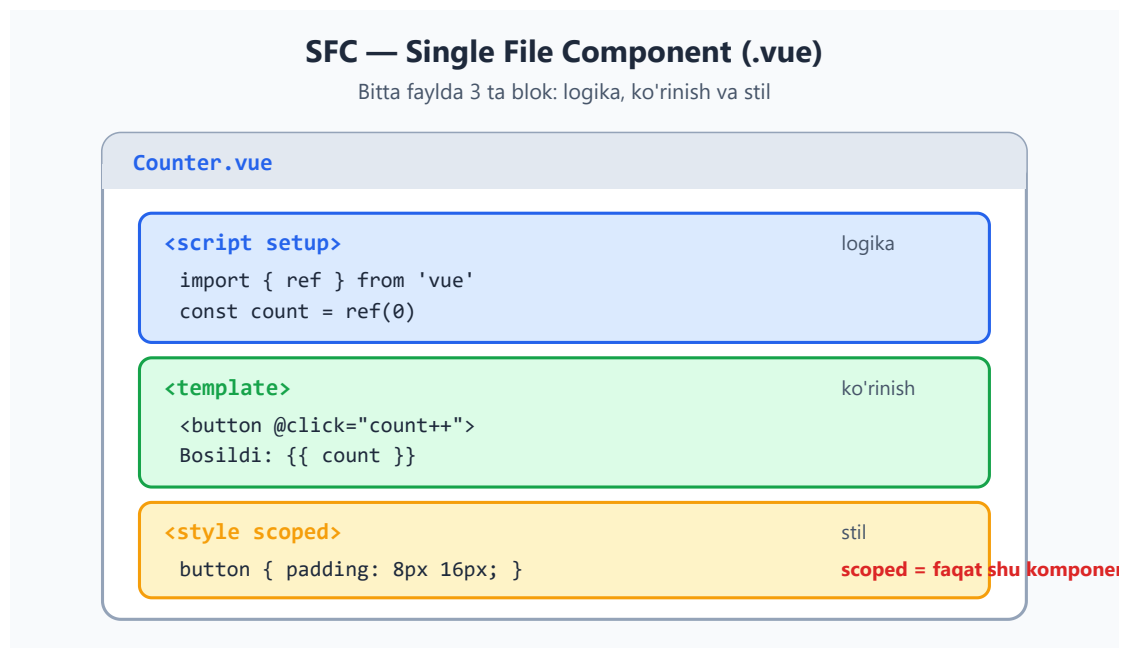
const message = ref('Salom Vue!')
const count = ref(0)
</script>

<template>
  <h1>{{ message }}</h1>
  <button @click="count++">Bosildi: {{ count }}</button>
</template>

<style scoped>
button { padding: 8px 16px; }
</style>
```

`<script setup>` — Composition API'ning eng qisqa ko'rinishi. `setup()` funksiya yozish, `return` qilish shart emas — top-level e'lon qilingan hamma narsa avtomatik template'ga ochiladi.

Bitta `.vue` faylda uchala blok qanday joylashishini quyida ko'rishingiz mumkin.



scoped — stil faqat shu komponentga ta'sir qiladi (CSS leak bo'lmaydi). Laravel'da component-level encapsulation kabi.

Bundan keyin hamma misol `<script setup>` uslubida bo'ladi — bu zamonaviy standart.

1.3 Template syntax — Interpolation

```
<script setup>
import { ref } from 'vue'
const name = ref('Oqil')
const rawHtml = ref('<b>qalin</b>')
</script>

<template>
  <!-- Text -->
  <p>Ism: {{ name }}</p>

  <!-- JS ifoda ishlaydi (statement EMAS) -->
  <p>{{ name.toUpperCase() }}</p>
  <p>{{ name.length > 3 ? 'uzun' : 'qisqa' }}</p>

  <!-- Raw HTML - XSS xavfi! Faqat ishonchli data uchun -->
  <p v-html="rawHtml"></p>
</template>
```

Qoidalar: - `{{ }}` ichida **ifoda (expression)** bo'ladi, **statement** emas. `{{ if (x) {...} }}` — XATO. `{{ x ? a : b }}` — to'g'ri. - `v-html` — HTML'ni render qiladi, lekin **XSS xavfli**. Foydalanuvchi kiritgan datada hech qachon ishlatma.

1.4 Direktivalar (Directives)

Direktiva — `v-` bilan boshlanadigan maxsus atribut. DOM'ga reaktiv xulq beradi.

`v-bind` — atributni bog'lash

```
<script setup>
import { ref } from 'vue'
const url = ref('https://ioqil.uz')
const isDisabled = ref(true)
const imgSrc = ref('/logo.png')
</script>

<template>
  <a v-bind:href="url">Link</a>

  <!-- Qisqartma: : -->
  <a :href="url">Link</a>
  
  <button :disabled="isDisabled">Tugma</button>
```

```
<!-- Bir nechta atributni obyekt bilan -->
<div v-bind="{ id: 'box', class: 'card' }"></div>
</template>
```

`:href` — bu `v-bind:href` ning qisqartmasi. Real kodda doim qisqartma ishlatiladi.

`v-if` / `v-else-if` / `v-else` — shartli render

```
<script setup>
import { ref } from 'vue'
const role = ref('admin')
</script>

<template>
  <div v-if="role === 'admin'">Admin panel</div>
  <div v-else-if="role === 'editor'">Editor panel</div>
  <div v-else>Mehmon</div>

  <!-- <template> - ko'rinmas o'rovchi (DOM'da ortiqcha element
  qoldirmaydi) -->
  <template v-if="role === 'admin'">
    <h2>Sarlavha</h2>
    <p>Bir nechta element, ortiqcha div'siz</p>
  </template>
</template>
```

`v-show` — ko'rsatish/yashirish

```
<p v-show="isVisible">CSS display bilan yashiriladi</p>
```

`v-if` vs `v-show` — muhim farq:

	<code>v-if</code>	<code>v-show</code>
Mexanizm	Elementni DOM'dan o'chiradi/qo'shadi	<code>display: none</code> qiladi (DOM'da qoladi)
Boshlang'ich xarajat	Past (false bo'lsa render qilinmaydi)	Yuqori (har doim render bo'ladi)
Toggle xarajati	Yuqori	Past
Qachon	Kamdan-kam o'zgaradigan shart	Tez-tez toggle bo'ladigan (masalan, dropdown)

v-for — ro'yxat render qilish

```
<script setup>
import { ref } from 'vue'
const users = ref([
  { id: 1, name: 'Ali' },
  { id: 2, name: 'Vali' },
])
const obj = ref({ a: 1, b: 2 })
</script>

<template>
  <!-- key DOIM kerak (unique, barqaror) -->
  <ul>
    <li v-for="user in users" :key="user.id">
      {{ user.name }}
    </li>
  </ul>

  <!-- index bilan -->
  <li v-for="(user, index) in users" :key="user.id">
    {{ index + 1 }}. {{ user.name }}
  </li>

  <!-- obyekt bo'yicha -->
  <li v-for="(value, key) in obj" :key="key">
    {{ key }}: {{ value }}
  </li>

  <!-- range -->
  <span v-for="n in 5" :key="n">{{ n }} </span>
</template>
```

:key nega shart? Vue ro'yxat o'zgarganda elementlarni qayta tartiblamasdan, faqat o'zgartirganini yangilash uchun `key` orqali har bir elementni "taniydi". `key` siz yoki `key="index"` bilan — input fokus yo'qolishi, animatsiya xatosi kabi buglar chiqadi. **Doim barqaror, unique key** (odatda DB `id`) ber.

Eslatma: `v-if` va `v-for` ni **bir elementda** ishlatma — bu anti-pattern (priority chalkashadi). Buning o'rniga:

```
<!-- YOMON -->
<li v-for="u in users" v-if="u.active" :key="u.id">...</li>

<!-- YAXSHI: computed bilan filtrlash (02-modulda) -->
<li v-for="u in activeUsers" :key="u.id">...</li>
```

v-on — event tinglash

```

<script setup>
import { ref } from 'vue'
const count = ref(0)

function handleClick(event) {
  console.log(event.target)
  count.value++
}
</script>

<template>
  <!-- To'liq shakl -->
  <button v-on:click="count++">+</button>

  <!-- Qisqartma: @ -->
  <button @click="count++">+</button>
  <button @click="handleClick">Method bilan</button>
  <button @click="handleClick($event)">Argument + event</button>

  <!-- Event modifiers -->
  <form @submit.prevent="onSubmit">...</form>      <!--
preventDefault() -->
  <div @click.stop="...">...</div>              <!--
stopPropagation() -->
  <input @keyup.enter="onEnter">                  <!-- faqat Enter --
>
  <div @click.self="...">...</div>                <!-- faqat shu
element -->
  <button @click.once="...">Bir marta</button>
</template>

```

Modifierlar — Vue'ning kuchli tomoni. `@submit.prevent` yozasan, `event.preventDefault()` ni qo'lda chaqirish shart emas.

1.5 Class va Style bog'lash

Class

```

<script setup>
import { ref } from 'vue'
const isActive = ref(true)
const hasError = ref(false)
const activeClass = ref('text-green')
</script>

<template>
  <!-- Obyekt: kalit = class, qiymat = bool -->
  <div :class="{ active: isActive, 'text-red': hasError }"></div>

```

```

<!-- Massiv -->
<div :class="[activeClass, hasError ? 'border-red' : '']"></div>

<!-- Obyekt + massiv aralash -->
<div :class="[ { active: isActive }, 'always-here' ]"></div>

<!-- Statik + dinamik birga (Vue ikkisini birlashtiradi) -->
<div class="card" :class="{ active: isActive }"></div>
</template>

```

Style

```

<template>
  <!-- camelCase yoki 'kebab-case' -->
  <div :style="{ color: 'red', fontSize: '16px' }"></div>
  <div :style="{ 'font-size': size + 'px' }"></div>

  <!-- Obyektlar massivi -->
  <div :style="[baseStyles, overrideStyles]"></div>
</template>

```

Amalda TailwindCSS bilan ko'pincha `:class` ga shartli klass berasan:

```
:class="active ? 'bg-blue-500' : 'bg-gray-200'".
```

1.6 Reactivity'ga kichik kirish (to'lig'i — 02-modulda)

Template'da ishlash uchun shu yetarli:

```

<script setup>
import { ref } from 'vue'

const count = ref(0) // <script>'da .value bilan ishlaysan

function increment() {
  count.value++      // MAJBURIY: .value
}
</script>

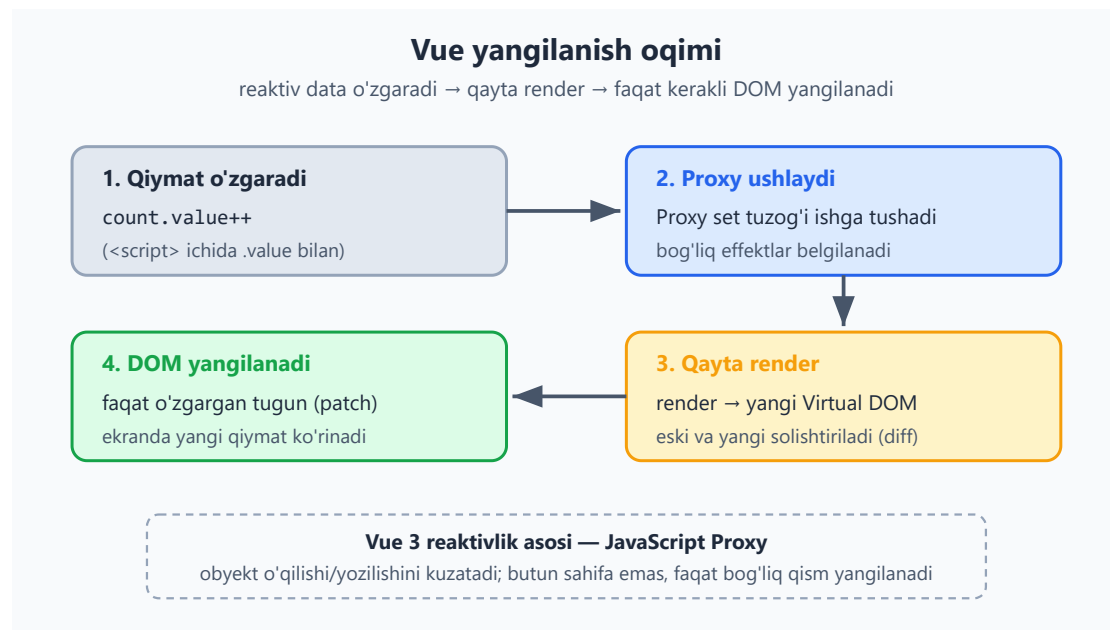
<template>
  {{ count }}        <!-- template'da .value KERAK EMAS, Vue ochadi ->
  <button @click="increment">+</button>
</template>

```

Eng ko'p qilinadigan xato: `<script>` ichida `.value` ni unutish.


```
count++          // ❌ ishlamaydi (count – obyekt, son emas)
count.value++    // ✅
```

`count.value` o'zgargandan keyin nima sodir bo'lishini quyidagi oqim ko'rsatadi: Proxy o'zgarishni ushlaydi, komponent qayta render bo'ladi va faqat o'zgargan DOM qismi yangilanadi.



Xulosa (tez takrorlash)

- `{{ }}` – interpolation (ifoda, statement emas)
- `v-bind ( : )` – atribut bog'lash
- `v-if / v-show` – shartli ko'rinish (farqini bil)
- `v-for` – ro'yxat (`:key` doim shart)
- `v-on ( @ )` – event + modifierlar
- `:class / :style` – dinamik stil
- `ref()` – reaktiv qiymat, `<script>` 'da `.value`

Masalalar (kamida 20 ta)

Har birini **alohida komponent** sifatida yoz. Yulduzcha (★) – qiyinroq.

Asosiy (1–10)

1. **Salom dunyo:** `name` ref'i bo'lsin, `<input>` ga yozilgan ismni `<h1>Salom, {{ name }}!</h1>` da ko'rsat. (Maslahat: keyingi modulda `v-model`, hozir `:value` + `@input` bilan qil.)
2. **Counter:** `+`, `-`, `Reset` tugmalari. Son manfiy bo'lib ketmasin (min 0).
3. **Toggle:** Bitta tugma matnni `Yoqilgan / O'chirilgan` ga almashtirsin va `:class` bilan rangi o'zgarsin.
4. **Light/Dark:** Tugma bosilganda `<div>` foni qora/oq bo'lsin (`:style` yoki `:class`).
5. **Ro'yxat render:** `['Olma', 'Anor', 'Uzum']` massivini `<ul>` da chiqar, har biriga tartib raqami qo'shib (`1. Olma`).
6. **Obyektlar ro'yxati:** `[{id, name, price}]` mahsulotlarni jadval (`<table>`) qilib chiqar.
7. **v-if mashqi:** `age` ref'i bo'lsin; `<18` → "Voyaga yetmagan", `18–60` → "Kattalar", `>60` → "Keksa".
8. **v-show vs v-if:** Bir xil kontentni ikkalasi bilan ko'rsat, DevTools Elements'da farqni kuzat va kodga izoh yoz: qaysi biri DOM'dan o'chadi.
9. **Conditional class:** Inputga matn yoz; uzunligi 8 dan kam bo'lsa border qizil, aks holda yashil.
10. **Event modifier:** Formada `@submit.prevent` ishlat; submit'da inputdagi matnni `console.log` qil va sahifa **qayta yuklanmasin**.

O'rta (11–18)

1. **Filtrlanadigan ro'yxat (★):** Mahsulotlardan faqat `price > 100` bo'lganini ko'rsat. (`v-for` + `v-if` ni bir elementda EMAS – vaqtincha massivni `.filter()` bilan template'da yoki alohida o'zgaruvchida.)
2. **Tab tizimi:** 3 ta tab tugmasi. Tanlangan tabga `active` klass berilsin, pastida mos kontent (`v-if / v-show`) ko'rinsin.
3. **Accordion (★):** Bir nechta savol-javob. Bosilganda javob ochilsin/yopilsin. Bir vaqtda faqat bittasi ochiq bo'lsin.
4. **Rangli ro'yxat:** `v-for` da juft indeksli qatorlar kulrang, toq indeksli oq bo'lsin (`:class + index % 2`).
5. **Keyboard:** Inputda `@keyup.enter` bo'lsa matnni ro'yxatga qo'shsin (to-do boshlanishi).
6. **Disabled tugma:** Input bo'sh bo'lsa "Qo'shish" tugmasi `:disabled` bo'lsin.

7. **Inline style slider:** Input range (0–100) qiymatiga qarab `<div>` kengligi `%` da o'zgarsin (progress bar).
8. **Dynamic attribute (★):** `:[attrName]="value"` (dynamic argument) ishlatib, tugma orqali `id` yoki `title` atributini almashtir.

Loyiha darajasi (19–24)

1. **To-Do v0 (★):** Vazifa qo'shish, ro'yxatda ko'rsatish, har birini "bajarildi" (line-through klass) qilish, o'chirish. (`computed` / `v-model` siz, faqat shu modul vositalari bilan — keyin yaxshilaysan.)
2. **Mahsulot kartochkalari (★):** Mahsulotlar massivini grid kartochka qilib chiqar. Har kartochkada nom, narx, "Sotuvda yo'q" badge (agar `stock === 0`).
3. **Soddagina kalkulyator (★★):** Ikki input + amal tugmalari (+ – × ÷). Natijani ko'rsat. Nolga bo'lishni ushlab qol.
4. **FAQ qidiruv (★★):** FAQ ro'yxati + qidiruv input. Faqat matni qidiruvga mos keladiganlari ko'rinsin.
5. **Yulduzli reyting (★★):** 5 ta yulduz. Hover/click bilan reyting tanlash, tanlangani to'ldirilgan ko'rinsin.
6. **Mini galereya (★★):** Rasmlar massivi; biriga bossang katta ko'rinishda (preview) chiqsin, `active` rasm chegarasi ajralib tursin.

✅ Ba'zi yechimlar (o'zing urinib ko'rgach qara!)

 **2-masala — Counter** 

```
<script setup>
import { ref } from 'vue'
const count = ref(0)
const inc = () => count.value++
const dec = () => { if (count.value > 0) count.value-- }
const reset = () => count.value = 0
</script>

<template>
<p>Hisob: {{ count }}</p>
<button @click="dec">-</button>
<button @click="reset">Reset</button>
<button @click="inc">+</button>
</template>
```



13-masala — Accordion (bir vaqtda bitta ochiq)



```
<script setup>
import { ref } from 'vue'
const faqs = ref([
  { id: 1, q: 'Vue nima?', a: 'Progressive JS framework.' },
  { id: 2, q: 'Nuxt nima?', a: 'Vue uchun meta-framework.' },
])
const openId = ref(null)
function toggle(id) {
  openId.value = openId.value === id ? null : id
}
</script>

<template>
  <div v-for="f in faqs" :key="f.id">
    <button @click="toggle(f.id)">{{ f.q }}</button>
    <p v-show="openId === f.id">{{ f.a }}</p>
  </div>
</template>
```



```
<script setup>
import { ref } from 'vue'
const text = ref('')
const todos = ref([])
let nextId = 1

function add() {
  const t = text.value.trim()
  if (!t) return
  todos.value.push({ id: nextId++, title: t, done: false })
  text.value = ''
}

function toggle(todo) { todo.done = !todo.done }
function remove(id) {
  todos.value = todos.value.filter(t => t.id !== id)
}
</script>

<template>
  <input :value="text" @input="text = $event.target.value"
  @keyup.enter="add" placeholder="Vazifa...">
  <button @click="add" :disabled="!text.trim()">Qo'shish</button>

  <ul>
    <li v-for="todo in todos" :key="todo.id">
      <span :class="{ done: todo.done }" @click="toggle(todo)">{{ todo.title }}</span>
      <button @click="remove(todo.id)">x</button>
    </li>
  </ul>
</template>

<style scoped>
.done { text-decoration: line-through; opacity: 0.5; cursor: pointer; }
</style>
```

02 — Reactivity (Reaktivlik)

← Oldingi: 01 — Vue asoslari · 🏠 README · Keyingi: 03 — Komponentlar →

Bu Vue'ning **yuragi**. Buni chuqur tushunsang, qolgan hammasi oson. Yuzaki tushunsang — "nega yangilanmayapti?" degan buglar bilan kurashasan.

Reaktivlik nima?

State o'zgarsa, unga **bog'liq** hamma narsa (UI, computed, watcher) avtomatik yangilanadi.

Excel analogiyasi: $C1 = A1 + B1$. $A1$ ni o'zgartirsang, $C1$ o'zi qayta hisoblanadi. Vue ham shunday "bog'liqliklar grafini" (dependency graph) yuritadi.

Laravel analogiyasi: Eloquent'da `$user->name` — oddiy property. O'zgartirsang hech narsa bo'lmaydi. Vue'da reaktiv qiymat — model observer'iga o'xshaydi: o'zgarish "kuzatiladi" va kerakli reaksiya (re-render) ishga tushadi.

2.1 `ref()` — birlamchi (primitiv) qiymatlar uchun

```
import { ref } from 'vue'

const count = ref(0)
const name = ref('Oqil')
const user = ref({ id: 1, name: 'Ali' }) // obyekt ham bo'ladi

console.log(count.value) // 0 - .value ORQALI o'qiladi
count.value++           // o'zgartirish ham .value bilan
count.value = 10
```

Nega `.value`? JavaScript'da primitivlar (`number`, `string`, `boolean`) **qiymat bo'yicha** uzatiladi — ularni "kuzatib" bo'lmaydi. Shu sabab Vue ularni `{ value: ... }` obyektga o'raydi. Obyektni esa Proxy bilan kuzatish mumkin.

- **Template'da** `.value` shart emas (Vue avtomatik ochadi/"unwrap" qiladi):

```
<template>{{ count }}</template> <!-- count.value emas -->
```

- **<script>** ichida `.value` doim kerak.

```
// ❌ Eng ko'p uchraydigan xato
const total = ref(0)
total = total + 5 // ref obyektini son bilan almashtiryapsan - buziladi

// ✅
total.value = total.value + 5
```

2.2 `reactive()` – faqat obyekt/massiv uchun

```
import { reactive } from 'vue'

const state = reactive({
  count: 0,
  user: { name: 'Ali' },
  todos: [],
})

state.count++ // .value YO'Q – to'g'ridan-to'g'ri
state.user.name = 'Vali' // chuqur (deep) reaktiv
state.todos.push('x')
```

`reactive` – obyektни ES6 **Proxy** ga o'raydi. Property o'qilishi/yozilishi "tutib olinadi" (intercept) va kuzatiladi.

`ref` vs `reactive` – qaysi birini ishlatish?

	<code>ref</code>	<code>reactive</code>
Tur	Har qanday (primitiv ham, obyekt ham)	Faqat obyekt/massiv/Map/Set
Kirish	<code>.value</code>	To'g'ridan-to'g'ri
Almashtirish (reassign)	<code>x.value = newObj</code> ✅	<code>state = newObj</code> ❌ reaktivlik yo'qoladi
Destructure	Reaktivlik yo'qoladi (<code>toRefs</code> kerak)	Reaktivlik yo'qoladi

Amaliy tavsiya: **Hamma joyda `ref` ishlat.** Bitta qoidani (`.value`) yodda tutish, ikkita modelni aralashtirishdan oson. `reactive` ning cheklovlari ko'p:

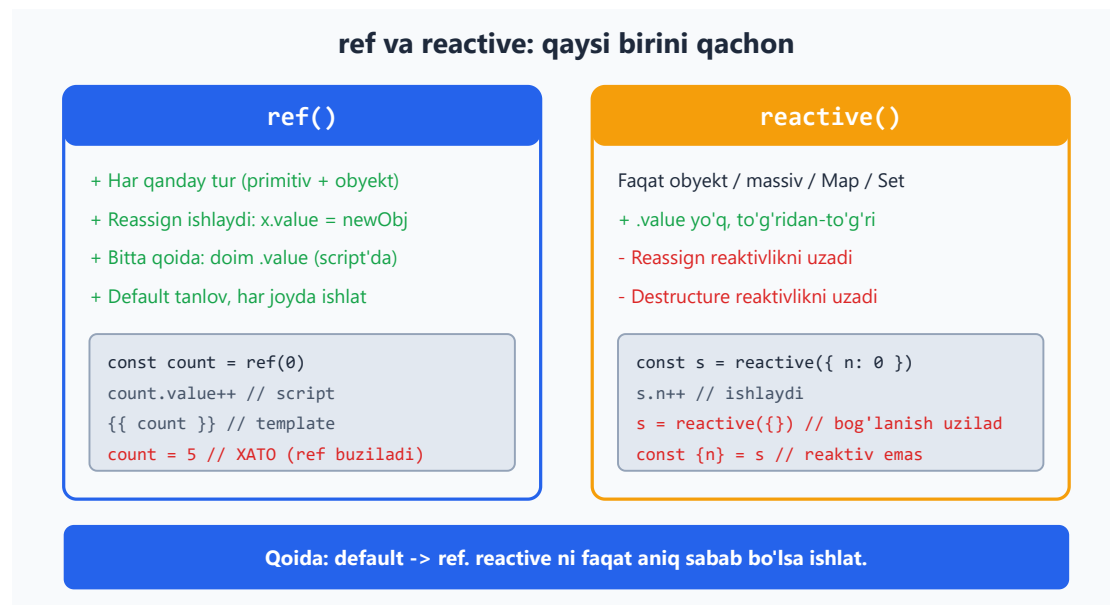
```
// reactive muammosi 1: reassign buziladi
let state = reactive({ count: 0 })
state = reactive({ count: 5 }) // eski kuzatuvchilar ulanib qoladi - bug

// reactive muammosi 2: destructure reaktivlikni uzadi
const { count } = reactive({ count: 0 }) // count endi oddiy son, reaktiv emas

// ref bilan bunday muammo yo'q:
const count = ref(0)
count.value = 5 // har doim ishlaydi
```

Vue jamoasining ko'p a'zolari ham "default — `ref`" deydi. `reactive` ni faqat aniq sabab bo'lsa ishlat.

Quyidagi diagramma ikkalasini va qaysi birini qachon ishlatishni solishtiradi:



2.3 `computed()` — hosilaviy (derived) qiymat

State'dan **kelib chiqadigan** qiymatni hisoblaydi va **keshlaydi**.

```
import { ref, computed } from 'vue'

const firstName = ref('Oqil')
const lastName = ref('Dev')

const fullName = computed(() => `${firstName.value} ${lastName.value}`)

console.log(fullName.value) // "Oqil Dev"
```



```
firstName.value = 'Ali'  
console.log(fullName.value)    // "Ali Dev" – avtomatik yangilandi
```

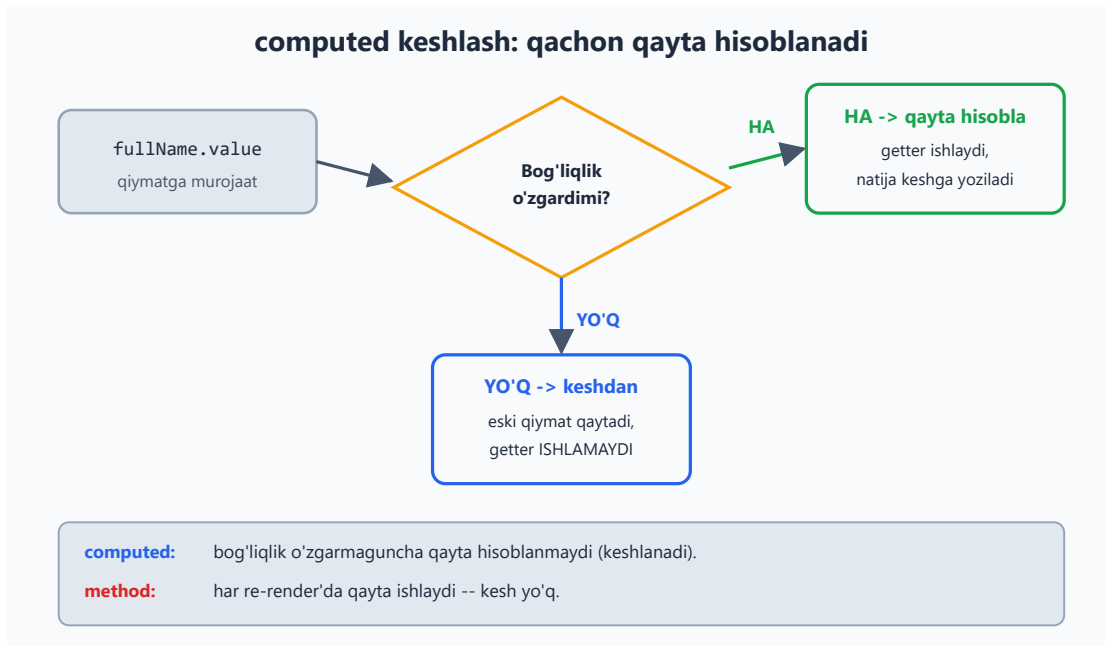
Laravel analogiyasi: `computed = Eloquent accessor (getFullNameAttribute)`. Bog'liq atributlardan hosilaviy qiymat. Farqi — Vue'da u **keshlanadi**.

Computed vs Method — MUHIM

```
<script setup>  
import { ref, computed } from 'vue'  
const list = ref([1, 2, 3, 4, 5])  
  
// computed – keshlanadi  
const evenComputed = computed(() => list.value.filter(n => n % 2 === 0))  
  
// method – har render'da QAYTA ishlaydi  
function evenMethod() { return list.value.filter(n => n % 2 === 0) }  
</script>  
  
<template>  
  {{ evenComputed }}    <!-- bog'liqlik (list) o'zgarmaguncha qayta  
hisoblanMAYDI -->  
  {{ evenMethod() }}    <!-- har re-render'da qayta ishlaydi -->  
</template>
```

`computed` **bog'liqliklarini** (`list`) kuzatadi. Ular o'zgarmasa — keshdagi qiymatni qaytaradi, qayta hisoblamaydi. Og'ir hisob-kitoblar uchun bu katta tejamkorlik.

Quyidagi diagramma keshlash mantig'ini ko'rsatadi — qachon qayta hisoblanadi, qachon keshdan oladi:



Writable computed (get/set)

```
const fullName = computed({
  get: () => `${firstName.value} ${lastName.value}`,
  set: (val) => {
    [firstName.value, lastName.value] = val.split(' ')
  }
})
fullName.value = 'Yangi Ism' // set ishga tushadi
```

Computed qoidalari

- **Sof (pure) bo'lsin:** faqat hisoblab qaytarsin, side-effect (API chaqirish, state o'zgartirish) qilmasin.
- Argument qabul qilmaydi. Parametr kerak bo'lsa — `computed` funksiya qaytarsin yoki method ishlat.

2.4 `watch()` — o'zgarishga reaksiya (side-effect)

`computed` qiymat **qaytaradi**. `watch` esa o'zgarishga **harakat** qiladi (API, localStorage, log...).

```
import { ref, watch } from 'vue'

const query = ref('')
```

```

watch(query, (newVal, oldVal) => {
  console.log(`${oldVal} → ${newVal}`)
  // masalan: API qidiruv
})

// Bir nechta manbani kuzatish
watch([firstName, lastName], ([newF, newL], [oldF, oldL]) => { ... })

// reactive obyekt property'sini kuzatish – getter funksiya bilan
watch(() => state.count, (newCount) => { ... })

```

watch opsiyalari

```

watch(source, callback, {
  immediate: true, // komponent yuklanganda darrov bir marta ishga
  tushsin
  deep: true,      // obyekt ichidagi har qanday o'zgarishni kuzat
  (chuqur)
  once: true,      // faqat bir marta (Vue 3.4+)
})

```

deep haqida: `ref` ichidagi obyektning *ichki* property'si o'zgarsa, oddiy `watch` ko'rmasligi mumkin. Chuqur kuzatish kerak bo'lsa `deep: true`. Lekin u qimmat — katta obyektlarda ehtiyot bo'l. Ko'pincha aniq property'ni getter bilan kuzatish `() => state.user.name`) yaxshiroq.

2.5 `watchEffect()` — bog'liqliklarni avtomatik aniqlaydi

`watch` da manbani aniq ko'rsatasan. `watchEffect` da esa — funksiya ichida ishlatilgan **hamma reaktiv qiymat** avtomatik bog'liqlikka aylanadi.

```

import { ref, watchEffect } from 'vue'

const userId = ref(1)

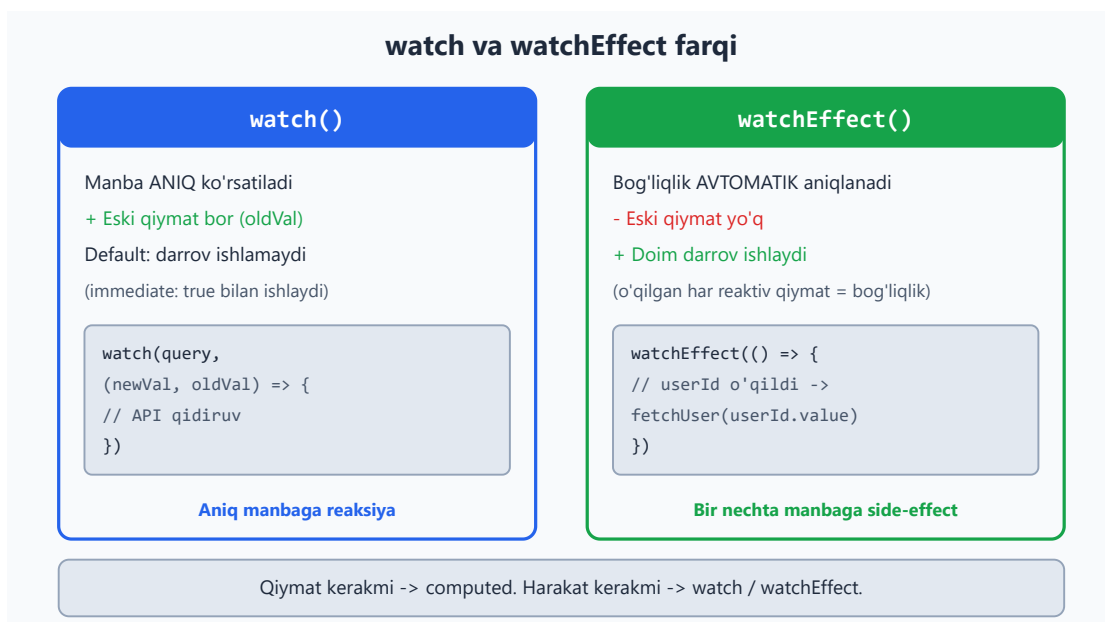
watchEffect(() => {
  // userId ishlatilgani uchun u avtomatik kuzatiladi
  console.log(`User ${userId.value} ni yuklash...`)
  fetchUser(userId.value)
})
// Darrov bir marta ishlaydi (immediate kabi), keyin userId har
o'zgarganda qayta

```

watch vs watchEffect

	watch	watchEffect
Bog'liqlik	Aniq ko'rsatiladi	Avtomatik aniqlanadi
Eski qiymat	Bor (<code>oldVal</code>)	Yo'q
Boshlang'ich ishga tushish	Default yo'q (<code>immediate</code> bilan)	Doim darrov ishlaydi
Qachon	Aniq manbaga reaksiya, eski qiymat kerak	Bir nechta manbaga bog'liq side-effect

Quyidagi diagramma ikkalasining farqini yonma-yon ko'rsatadi:



Cleanup (tozalash)

```
watch(id, async (newId, oldId, onCleanup) => {
  const controller = new AbortController()
  onCleanup(() => controller.abort()) // oldingi so'rovni bekor qil
  (race condition)
  await fetch(`/api/x/${newId}`, { signal: controller.signal })
})
```

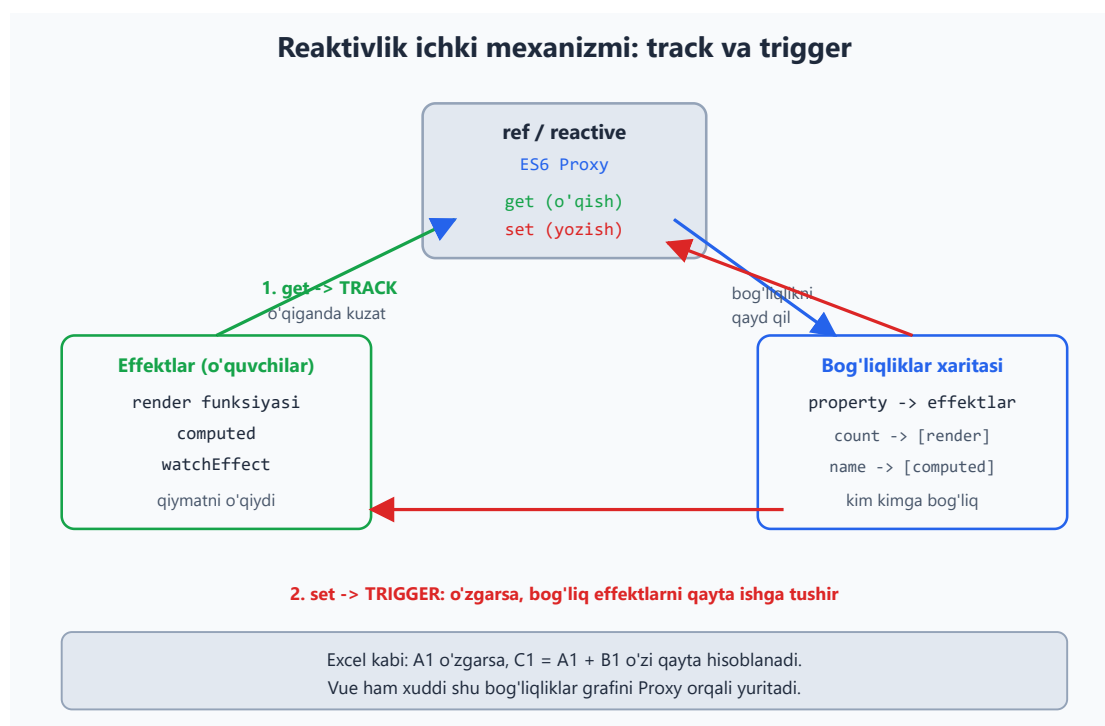
2.6 Reaktivlik ICHKI mexanizmi (nega ishlaydi)

Buni bilish — debugging'ni osonlashtiradi:

1. **Track (kuzatish):** Komponent render bo'lganda yoki `computed` / `watchEffect` ishlaganda, ular o'qigan har bir reaktiv property "bu effekt menga bog'liq" deb belgilanadi (Proxy `get` trap orqali).
2. **Trigger (qo'zg'atish):** Property o'zgarganda (Proxy `set` trap), Vue o'sha propertyga bog'liq hamma effektzni qayta ishga tushiradi.

```
[reactive obyekt] --get--> kim o'qiyapti? --> bog'liqlikni qayd qil
                        --set--> o'zgardi! --> bog'liq effektlarni qayta
                        ishga tushir
```

Quyidagi diagramma bu ichki mexanizmni (track va trigger) ko'rsatadi:



Bundan kelib chiqadigan **muhim cheklolar**:

```
// ❌ Yangi property qo'shish (reactive obyektida eski Vue'larda muammo edi, Vue 3 Proxy bunda yaxshi, lekin ref afzal)
// ❌ Destructure reaktivlikni uzadi:
const { count } = reactive({ count: 0 }) // count - oddiy son
// ✅ toRefs bilan saqlash:
import { toRefs } from 'vue'
const state = reactive({ count: 0, name: 'x' })
const { count, name } = toRefs(state) // count.value, name.value - reaktiv qoladi
```

2.7 Foydali reactivity yordamchilari

```
import { toRef, toRefs, unref, isRef, toRaw } from 'vue'

isRef(x)          // x ref'mi?
unref(x)          // ref bo'lsa .value, bo'lmasa o'zini qaytaradi
(x?.value ?? x)
toRaw(proxy)      // Proxy'dan asl obyektini oladi (kuzatuvsiz)
toRef(obj, 'k')   // obyekt property'sidan ref yasaydi (bog'lanish
saqlanadi)
toRefs(obj)       // butun obyektini ref'lar obyektiga aylantiradi
```

Advanced (kerak bo'lganda): - `shallowRef` — faqat `.value` almashishini kuzatadi, ichini emas (katta obyektlar uchun performance). - `readonly` — o'zgartirib bo'lmaydigan reaktiv nusxa. - `markRaw` — obyektini reaktiv qilmaslikni belgilaydi (masalan, 3rd-party class instansiyasi).

Xulosa

- `ref` — default tanlov, hammasi uchun, `.value` (script'da)
- `reactive` — faqat obyekt, cheklovlari ko'p, ehtiyotkorlik bilan
- `computed` — hosilaviy + keshlangan qiymat (Eloquent accessor kabi)
- `watch` — aniq manbaga reaksiya, eski qiymat bor
- `watchEffect` — avtomatik bog'liqlik, doim darrov
- Reaktivlik = Proxy `get` (track) + `set` (trigger)
- Destructure reaktivlikni **uzadi** → `toRefs`

Oltin qoida: Qiymat kerakmi → `computed`. Harakat kerakmi → `watch`.

Masalalar (kamida 22 ta)

ref / reactive (1–6)

1. `price` (ref) va `qty` (ref) bo'lsin. `total` ni **computed** qilib chiqar (`price*qty`).
2. `reactive({ x:0, y:0 })` ob'yektini yarat, ikki tugma `x` va `y` ni oshirsin, ekranda ko'rsat.
3. Yuqoridagi reactive obyektini `const { x } = state` qilib destructure qil — nega yangilanmayapti? Tushuntir va `toRefs` bilan tuzat.

4. **ref** ichida obyekt: `const user = ref({ name:'', age:0 })`. Inputlardan name/age yangilansin (`.value.name`).
5. **?** `count = ref(0); count = count + 1` nega xato? Kodga izoh yozib to'g'rila.
6. **reactive** massiv: `state.items` ga element qo'shish/o'chirish tugmalari (`push`, `filter`).

computed (7–13)

1. **Full name:** `firstName`, `lastName` → `fullName` computed.
2. **Writable computed (★):** `fullName` get/set bilan; bitta inputga to'liq ism yozilsa, `firstName` / `lastName` ajralib saqlansin.
3. **Filtr:** Mahsulotlar massivi + `minPrice` (ref). `filtered` computed faqat `price >= minPrice` larni qaytarsin.
4. **Saralash (★):** Massivni `asc` / `desc` toggle qiluvchi tugma; `sorted` computed mos saralasin (asl massivni **mutatsiya qilmasdan** — `[...arr].sort()`).
5. **Statistika:** Sonlar massividan `sum`, `avg`, `max`, `min` ni 4 ta computed qilib chiqar.
6. **Computed vs method (★):** Bir xil og'ir hisob-kitobni computed va method qilib yoz, `console.log` qo'yib, qaysi biri kamroq chaqirilishini kuzat. Xulosani yoz.
7. **Savat (★):** `[{name,price,qty}]` savat. `subtotal` (har element), `grandTotal`, `itemCount` computed bo'lsin.

watch / watchEffect (14–20)

1. **Logger:** `name` (ref) o'zgarsa, eski→yangi qiymatni `console.log` qil.
2. **localStorage sync (★):** `theme` ref'ini `watch` qilib `localStorage` ga saqla; sahifa ochilganda undan o'qib boshlang'ich qiymat qil.
3. **immediate:** `watch` ga `immediate:true` ber, komponent yuklanganda darrov ishlashini ko'rsat.
4. **deep (★):** `ref({user:{name}})`. Oddiy `watch` ichki o'zgarishni ko'rmasligini, `deep:true` ko'rishini namoyish qil.
5. **watchEffect:** `a`, `b` ref'lariga bog'liq `watchEffect`; ikkalasidan biri o'zgarsa qayta ishlasin. Keyin shuni `watch([a,b])` ga aylantir — farqni yoz.
6. **Debounced search (★★):** Qidiruv inputi. `watch` ichida `setTimeout` + `onCleanup` bilan oldingi timer'ni tozalab, 400ms kechikish bilan "qidiruv yuborildi" deb log qil.
7. **Race condition (★★):** `userId` (ref) o'zgarsa "API" (`setTimeout` bilan soxta) chaqir; `onCleanup` orqali eski so'rovni bekor qilib, faqat oxirgisi natija berishini ta'minla.

Aralash loyiha (21–25)

1. **Konvertor (★)**: Som ↔ Dollar. Bir inputga som yozsang dollar chiqsin va aksincha (2 ta writable computed yoki watch).
2. **Parol kuchi (★)**: Parol inputi. `computed` bilan kuch darajasini hisobla (uzunlik, raqam, katta harf, belgi) va rangli indikator ko'rsat.
3. **To-Do v1 (★★)**: 01-moduldagi To-Do'ni yaxshila: `activeCount` / `completedCount` `computed`, `filter` (all/active/done) `computed`, ro'yxatni `localStorage` ga `watch(deep)` bilan saqla.
4. **Forma validatsiya (★★)**: Email + parol. Har biri uchun `computed` xato xabari (`isValidEmail`, `isValidPassword`); `canSubmit` `computed` ikkalasi to'g'ri bo'lsagina `true`.
5. **Real-time filter+sort+search (★★★)**: Mahsulotlar jadvali: qidiruv (matn), kategoriya filtri, narx bo'yicha saralash — uchalasi **bitta** `displayProducts` `computed` orqali zanjir bilan birlashsin.

✓ Tanlangan yechimlar

8 — Writable computed

```
<script setup>
import { ref, computed } from 'vue'
const firstName = ref('Oqil')
const lastName = ref('Dev')

const fullName = computed({
  get: () => `${firstName.value} ${lastName.value}`,
  set: (val) => {
    const parts = val.trim().split(/\s+/)
    firstName.value = parts[0] ?? ''
    lastName.value = parts.slice(1).join(' ')
  },
})
</script>

<template>

<p>Ism: {{ firstName }} | Familiya: {{ lastName }}</p>
</template>
```




19 — Debounced search



```
<script setup>
import { ref, watch } from 'vue'
const query = ref('')

watch(query, (val, _old, onCleanup) => {
  const timer = setTimeout(() => {
    console.log('Qidiruv yuborildi:', val) // bu yerda API
  }, 400)
  onCleanup(() => clearTimeout(timer)) // tez yozilsa oldingisini
  bekor qil
})
</script>

<template>
  <input :value="query" @input="query = $event.target.value"
  placeholder="Qidiruv...">
</template>
```



25 — Filter + sort + search zanjiri



```
<script setup>
import { ref, computed } from 'vue'
const products = ref([
  { id:1, name:'Laptop', cat:'tech', price:1200 },
  { id:2, name:'Olma', cat:'food', price:5 },
  { id:3, name:'Telefon', cat:'tech', price:800 },
])
const search = ref('')
const category = ref('all')
const sortDir = ref('asc')

const displayProducts = computed(() => {
  let list = products.value
  // 1) search
  if (search.value) {
    const q = search.value.toLowerCase()
    list = list.filter(p => p.name.toLowerCase().includes(q))
  }
  // 2) category
  if (category.value !== 'all') {
    list = list.filter(p => p.cat === category.value)
  }
  // 3) sort (nusxa olib - asl massivni buzmaymiz)
  return [...list].sort((a, b) =>
    sortDir.value === 'asc' ? a.price - b.price : b.price - a.price
  )
})
</script>

<template>
  <input :value="search" @input="search = $event.target.value"
  placeholder="Qidiruv...">
  <button @click="sortDir = sortDir === 'asc' ? 'desc' : 'asc'">
    Narx: {{ sortDir }}
  </button>
  <ul>
    <li v-for="p in displayProducts" :key="p.id">{{ p.name }} - ${{ p.price
  }}</li>
  </ul>
</template>
```



Keyingi: 03 — Komponentlar

03 — Komponentlar

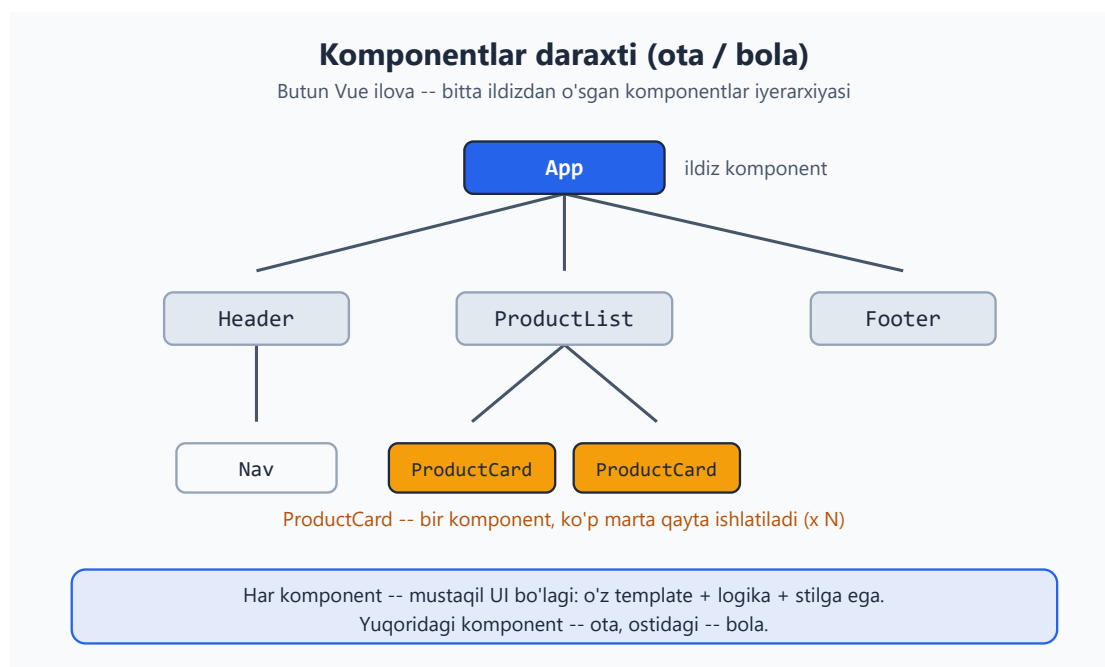
← Oldingi: 02 — Reactivity · 🏠 README · Keyingi: 04 — Composition API →

Komponent — qayta ishlatiladigan, mustaqil UI bo'lagi (o'z template + logika + stilga ega). Butun Vue ilova — komponentlar **daraxti**.

Laravel analogiyasi: Blade component (`<x-card>`) yoki Livewire component. Lekin Vue komponenti brauzerda interaktiv yashaydi va o'z holatini (state) saqlaydi.

```
App
├── Header
│   └── Nav
├── ProductList
│   └── ProductCard (x N)
└── Footer
```

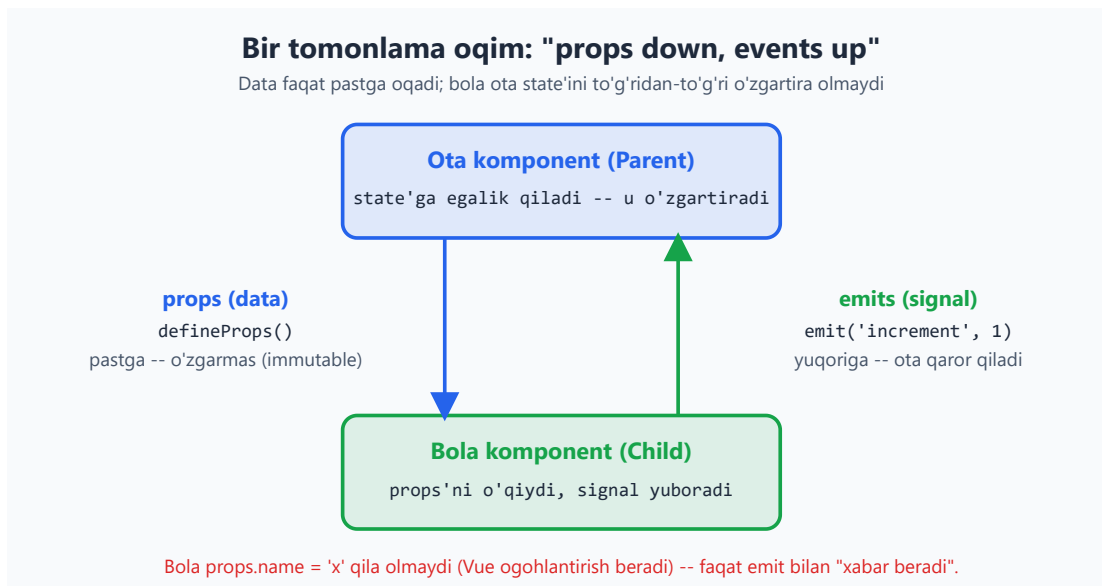
Quyidagi diagramma shu daraxtni vizual ko'rsatadi (ota yuqorida, bola ostida):



Ma'lumot oqimi (data flow): Vue'da **bir tomonlama** (one-way) — yuqoridan pastga: - **Pastga:** ota → bola, **props** orqali. - **Tepaga:** bola → ota, **events (emit)** orqali.

Bu "props down, events up" qoidasi. Bola props'ni **o'zgartira olmaydi** (faqat ota o'zgartiradi). Bu — taxmin qilinadigan (predictable) data flow uchun.

Quyidagi diagramma bu bir tomonlama oqimni ko'rsatadi: props pastga (data), emit yuqoriga (signal):



3.1 Komponentni ro'yxatdan o'tkazish va ishlatish

```
<!-- ChildCard.vue -->
<script setup>
</script>
<template>
  <div class="card">Men bolaman</div>
</template>
```

```
<!-- Parent.vue -->
<script setup>
import ChildCard from './ChildCard.vue' // import - bas,
registratsiya avtomatik
</script>

<template>
  <ChildCard />
  <ChildCard /> <!-- qayta ishlatish -->
</template>
```

`<script setup>` da import qilingan komponent darrov template'da ishlatishga tayyor. Alohida `components: {}` e'lon qilish shart emas.

3.2 Props — ota'dan bola'ga data

```
<!-- UserCard.vue -->
<script setup>
// defineProps — kompilyator makrosi (import qilish shart emas)
const props = defineProps({
  name: { type: String, required: true },
  age: { type: Number, default: 0 },
  isActive: { type: Boolean, default: false },
  roles: { type: Array, default: () => [] }, // obyekt/massiv
  default — funksiya orqali!
  user: { type: Object, default: () => ({}) },
})

// props.name, props.age — script'da shunday ishlatasan
console.log(props.name)
</script>

<template>
  <div :class="{ active: isActive }">
    {{ name }}, {{ age }} yosh
  </div>
</template>
```

```
<!-- Parent -->
<UserCard name="Oqil" :age="25" :is-active="true" :roles="['admin']" />
```

Diqqat: - Statik string: `name="Oqil"` (tirnoqsiz `v-bind`). - Son/bool/massiv/obyekt: `:` (**v-bind**) **shart** — `:age="25"`, aks holda `"25"` string bo'lib ketadi. - Template'da `camelCase` prop → HTML'da `kebab-case`: `isActive` → `:is-active`. - **Obyekt/massiv default qiymati funksiya orqali** beriladi (`default: () => []`), aks holda barcha instansiyalar bitta obyektни baham ko'radi.

TypeScript bilan props (zamonaviy, tavsiya)

```
<script setup lang="ts">
interface Props {
  name: string
  age?: number
  roles?: string[]
}
const props = withDefaults(defineProps<Props>(), {
  age: 0,
  roles: () => [],
})
</script>
```

Props o'zgarmas (immutable)!

```
// ❌ props'ni mutatsiya qilish – Vue ogohlantirish beradi
props.name = 'Boshqa'

// ✅ Agar lokal o'zgartirish kerak bo'lsa – computed yoki lokal ref
const localName = ref(props.name)
const upper = computed(() => props.name.toUpperCase())
```

3.3 Emits — bola'dan ota'ga signal

```
<!-- Counter.vue -->
<script setup>
const props = defineProps({ modelValue: Number })
const emit = defineEmits(['increment', 'reset']) // qaysi eventlarni
chiqarishini e'lon qil

function add() {
  emit('increment', 1) // event nomi + payload
}
</script>

<template>
  <button @click="add">+</button>
  <button @click="emit('reset')">Reset</button>
</template>
```

```
<!-- Parent -->
<script setup>
import { ref } from 'vue'
import Counter from './Counter.vue'
const total = ref(0)
</script>

<template>
  <Counter @increment="total += $event" @reset="total = 0" />
  <p>{{ total }}</p>
</template>
```

TS bilan emits (type-safe)

```
const emit = defineEmits<{
  increment: [amount: number]
  reset: []
}>()
```

Nega emit? Bola ota'ning state'ini to'g'ridan-to'g'ri o'zgartira olmaydi. U faqat "menda biror narsa bo'ldi" deb **xabar beradi**, ota qaror qabul qiladi. Bu — DDD'dagi domain event'ga o'xshaydi: agregat o'zgarishni e'lon qiladi, listener reaksiya qiladi.

3.4 `v-model` komponentlarda (two-way binding)

`v-model` — props + emit ning "shirin" qisqartmasi. Form komponentlari uchun ideal.

Vue 3.4+ (`defineModel` — zamonaviy, eng oson)

```
<!-- MyInput.vue -->
<script setup>
const model = defineModel() // ref kabi ishlaydi, o'qish/yozish two-
way
</script>

<template>
  <input :value="model" @input="model = $event.target.value">
</template>
```

```
<!-- Parent -->
<script setup>
import { ref } from 'vue'
import MyInput from './MyInput.vue'
const name = ref('')
</script>

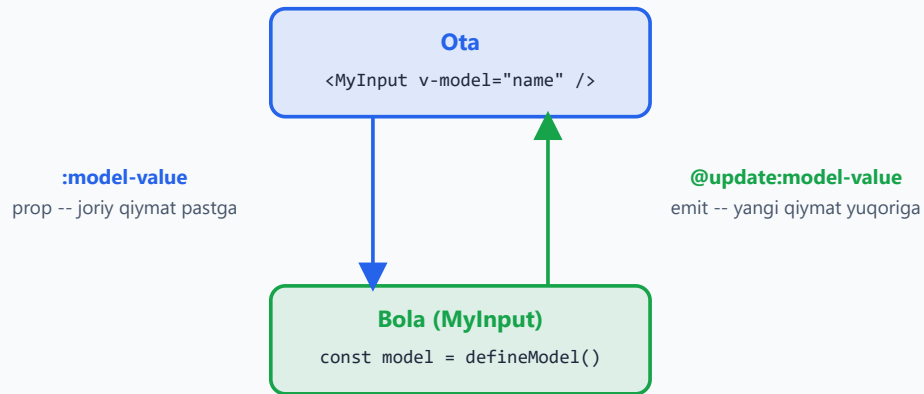
<template>
  <MyInput v-model="name" />
  <p>{{ name }}</p>
</template>
```

`v-model="name"` ichki tomondan `:model-value="name"` `@update:model-value="name = $event"` ga teng. `defineModel()` shu mexanizmni avtomatik o'raydi.

Quyidagi diagramma `v-model` ning ikki tomonlama mexanizmini (prop pastga + emit yuqoriga) ochib beradi:

v-model -- ikki tomonlama (two-way) bog'lanish

Bu sehr emas: props (pastga) + emit (yuqoriga) ning "shirin" qisqartmasi



`v-model="name" === :model-value="name" @update:model-value="name = $event"`

Bir nechta v-model (named)

```
<!-- Parent -->
<UserForm v-model:first-name="first" v-model:last-name="last" />
```

```
<!-- UserForm.vue -->
<script setup>
const firstName = defineModel('firstName')
const lastName = defineModel('lastName')
</script>
```

Native input'da v-model (asoslar)

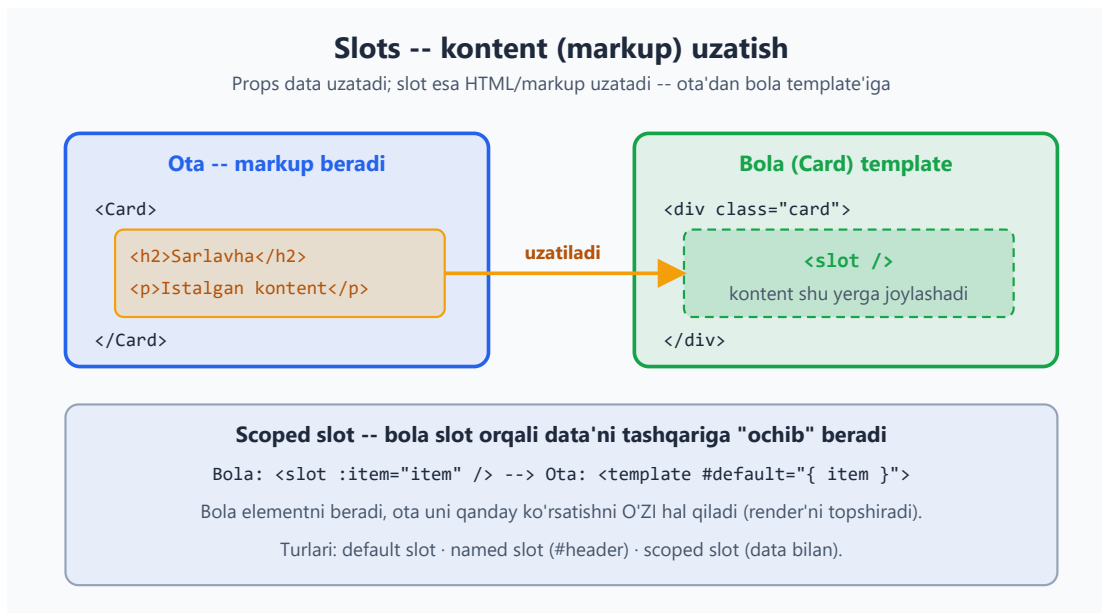
```
<input v-model="text">           <!-- text input -->
<input type="checkbox" v-model="agree"> <!-- boolean -->
<input type="checkbox" v-model="picks" value="a"> <!-- massivga
to'planadi -->
<select v-model="country">...</select>
<textarea v-model="bio"></textarea>

<!-- modifierlar -->
<input v-model.trim="name">      <!-- bo'shliqni kesadi -->
<input v-model.number="age">     <!-- songa aylantiradi -->
<input v-model.lazy="x">         <!-- change'da (input emas)
yangilanadi -->
```

3.5 Slots — kontent uzatish

Props — *data* uzatadi. Slot — *markup/HTML* uzatadi. Komponentni "o'rovchi" (wrapper) qilib moslashuvchan qiladi.

Quyidagi diagramma ota bergan markup bola template'idagi `<slot>` teshigiga qanday joylashishini ko'rsatadi:



Default slot

```
<!-- Card.vue -->
<template>
  <div class="card">
    <slot>Default kontent (hech narsa berilmasa)</slot>
  </div>
</template>
```

```
<!-- Parent -->
<Card>
  <h2>Sarlavha</h2>
  <p>Ichidagi istalgan markup</p>
</Card>
```

Named slots (nomli)

```
<!-- Layout.vue -->
<template>
  <header><slot name="header" /></header>
  <main><slot /></main>           <!-- default -->
  <footer><slot name="footer" /></footer>
</template>
```

```

<!-- Parent -->
<Layout>
  <template #header><h1>Logo</h1></template>
  <p>Asosiy kontent (default slot)</p>
  <template #footer>© 2026</template>
</Layout>

```

#header — v-slot:header ning qisqartmasi.

Scoped slots (bola → ota slot'ga data uzatadi)

Eng kuchli pattern. Ro'yxat/jadval kabi qayta ishlatiladigan komponentlarda render'ni iste'molchiga topshirish uchun:

```

<!-- List.vue -->
<script setup>
defineProps({ items: Array })
</script>
<template>
  <ul>
    <li v-for="item in items" :key="item.id">
      <slot :item="item" :index="item.id">{{ item.name }}</slot>
    </li>
  </ul>
</template>

```

```

<!-- Parent — render qanday bo'lishini O'ZI hal qiladi -->
<List :items="users">
  <template #default="{ item }">
    <strong>{{ item.name }}</strong> — {{ item.email }}
  </template>
</List>

```

Laravel analogiyasi: Blade slot ({{ \$slot }} , {{ \$header }}) bilan deyarli bir xil g'oya. Scoped slot esa — bola ichki ma'lumotni tashqariga "ochib" beradi, bu Blade'da yo'q kuchli imkoniyat.

3.6 provide / inject — chuqur uzatish (prop drilling'siz)

Ota → uzoq nabira'gacha props'ni har bosqichda uzatish charchatadi (**prop drilling**).

provide/inject — daraxtning istalgan chuqurligiga to'g'ridan-to'g'ri yetkazadi.

```

<!-- Ota (yuqorida) -->
<script setup>
import { provide, ref } from 'vue'
const theme = ref('dark')
provide('theme', theme) // kalit + qiymat (reaktiv ham bo'ladi)
provide('toggleTheme', () => {
  theme.value = theme.value === 'dark' ? 'light' : 'dark'
})
</script>

```

```

<!-- Uzoq nabira -->
<script setup>
import { inject } from 'vue'
const theme = inject('theme') // reaktiv qoladi
const toggle = inject('toggleTheme')
const x = inject('maybe', 'default-qiymat') // bo'lmasa default
</script>

```

provide/inject ni **lokal/feature-darajadagi** umumiy holat uchun ishlat (masalan, theme, form konteksti). **Global ilova holati** uchun esa **Pinia** (06-modul) yaxshiroq — u DevTools, type-safety, struktura beradi.

Type-safe provide/inject (TS):

```

import { InjectionKey } from 'vue'
const ThemeKey: InjectionKey<Ref<string>> = Symbol('theme')
provide(ThemeKey, theme)
const t = inject(ThemeKey) // tipi to'g'ri aniqlanadi

```

3.7 Boshqa muhim narsalar (qisqacha)

```

<!-- Dynamic component -->
<component :is="currentTab" /> <!-- currentTab – komponent yoki nomi -->

<!-- KeepAlive – komponent holatini eslab qoladi (qayta yaratmaydi) -->
<KeepAlive><component :is="currentTab" /></KeepAlive>

<!-- Async component (lazy loading) -->
<script setup>
import { defineAsyncComponent } from 'vue'
const Heavy = defineAsyncComponent(() => import('./Heavy.vue'))
</script>

<!-- Fallthrough attributes -->

```

```
<!-- Ota <MyBtn class="x" @click="..."> bersa, MyBtn ichidagi root elementga avtomatik o'tadi -->
```

Component naming: Fayl/komponent nomi — `PascalCase` (`UserCard.vue`).
Template'da ham `<UserCard>`. Bu native HTML teglardan ajratib turadi.

Xulosa

- **Props** — ota→bola data (immutable, bola o'zgartirmaydi)
 - **Emits** — bola→ota signal (`defineEmits`, payload bilan)
 - **v-model** — props+emit qisqartmasi (`defineModel()`, 3.4+)
 - **Slots** — markup uzatish (default / named / scoped)
 - **provide/inject** — chuqur uzatish, prop drilling'siz (lokal scope uchun)
 - Data flow: **props down, events up**
-

Masalalar (kamida 22 ta)

Props (1–6)

1. `BaseButton` komponenti: `label` (string), `variant` ('primary'/'danger') props. `variant` ga qarab `:class` bilan rang berilsin.
2. `Avatar`: `src`, `alt`, `size` (number) props. `size` ga qarab inline `width/height`.
3. `Badge`: `count` (number) prop. `count > 99` bo'lsa `99+` ko'rsat.
4. `ProductCard` (★): `product` (obyekt) props — nom, narx, rasm. `defineProps` da obyekt default'ini to'g'ri ber (`() => ({})`).
5. ? Bolada `props.title = 'x'` qilsang nima bo'ladi? Sinab ko'r, console'dagi warning'ni o'qib, to'g'ri yondashuvni yoz.
6. TS bilan: `interface Props` orqali `UserCard` props'ini type qil, `withDefaults` ishlat.

Emits (7–11)

1. `Counter` komponenti `@change` event chiqarsin (yangi qiymat bilan); ota qabul qilib ko'rsatsin.
2. `DeleteButton` (★): bosilganda `@confirm-delete` event'ini `id` bilan chiqarsin; ota ro'yxatdan o'chirsin.

3. `SearchBox`: input o'zgarganda `@search` event'ini matn bilan chiqarsin (debounce bo'lsa ★★).
4. `Rating` (★): yulduz bosilsa `@rate` event'ini `1..5` bilan chiqarsin.
5. TS bilan: yuqoridagi `Rating` emits'ini `defineEmits<{...}>()` orqali type-safe qil.

v-model (12–16)

1. `TextField` (★): `defineModel()` bilan reusable input yarat; otada `v-model` bilan ulang.
2. `Toggle` (★): switch ko'rinishidagi boolean `v-model` komponenti (`defineModel()`).
3. `QuantityInput` (★): `- / +` tugmali son inputi; `v-model` bilan, min/max props bilan chegaralangan.
4. **Bir nechta v-model (★★)**: `NameForm` komponenti `v-model:first` va `v-model:last` qabul qilsin.
5. `StarRating` ni `v-model` ga aylantir (12-emit o'rniga two-way).

Slots (17–21)

1. `Card` (★): default slot + `#header / #footer` named slotlar bilan.
2. `Modal` (★★): `#title`, default (body), `#actions` slotlari; `isOpen` prop + `@close` emit bilan.
3. `Alert`: `type` prop (success/error/warning) + default slot (xabar matni).
4. **Scoped slot (★★)**: `DataList` komponenti `items` props oladi, har element renderini scoped slot orqali otaga topshiradi.
5. **Scoped slot jadval (★★)**: `DataTable` — `columns` va `rows` props; har katak renderini `#cell="{ row, column }"` scoped slot bilan moslashtir.

provide/inject va kompozitsiya (22–26)

1. `provide/inject` (★): Ota `theme` (ref) bersin, uzoqdagi `ThemedButton` `inject` qilib rangini moslashtirsin; ota'dagi tugma theme'ni toggle qilsin.
2. **Prop drilling vs inject (★★)**: Bir xil datani 3 qavat props bilan, keyin `provide/inject` bilan uzat — kod farqini taqqosla, izoh yoz.
3. **Dynamic component (★★)**: 3 ta tab komponenti; `<component :is>` bilan almashtir; `<KeepAlive>` qo'shib, tab almashganda input qiymati saqlanishini ko'rsat.

4. **Composable + komponent (★★★):** `useToggle()` composable yoz (04-modul oldindan), `Modal` da ishlatib `isOpen` ni boshqar.
5. **Mini dizayn tizimi (★★★):** `BaseButton`, `BaseInput`, `BaseCard`, `BaseModal` — barchasi props/emits/slots bilan izchil API'ga ega bo'lsin. Bittagina demo sahifada hammasidan foydalan (EduCore UI kit boshlanishi 😊).

✓ Tanlangan yechimlar

12 — TextField (defineModel)

```
<!-- TextField.vue -->
<script setup>
defineProps({ label: String, placeholder: String })
const model = defineModel()
</script>

<template>
<label>
  {{ label }}
  <input :value="model" :placeholder="placeholder"
    @input="model = $event.target.value">
</label>
</template>

<!-- Parent -->
<script setup>
import { ref } from 'vue'
import TextField from './TextField.vue'
const email = ref('')
</script>
<template>
  <TextField v-model="email" label="Email" placeholder="you@mail.uz" />
  <p>{{ email }}</p>
</template>
```



20 — Scoped slot (DataList)



```
<!-- DataList.vue -->
<script setup>
defineProps({ items: { type: Array, default: () => [] } })
</script>
<template>
  <ul>
    <li v-for="(item, i) in items" :key="item.id ?? i">
      <slot :item="item" :index="i">{{ item }}</slot>
    </li>
  </ul>
</template>

<!-- Parent - render formatini O'ZI belgilaydi -->
<DataList :items="users">
  <template #default="{ item, index }">
    {{ index + 1 }}. <b>{{ item.name }}</b> ({{ item.email }})
  </template>
</DataList>
```



22 — provide/inject theme



```
<!-- App.vue (ota) -->
<script setup>
import { provide, ref } from 'vue'
import ThemedButton from './ThemedButton.vue'
const theme = ref('dark')
provide('theme', theme)
</script>
<template>
  <button @click="theme = theme === 'dark' ? 'light' : 'dark'">Theme: {{
theme }}</button>
  <ThemedButton /> <!-- istalgancha chuqur bo'lsa ham ishlaydi -->
</template>
```

```
<!-- ThemedButton.vue (nabira) -->
<script setup>
import { inject, computed } from 'vue'
const theme = inject('theme', ref('light'))
const cls = computed(() => theme.value === 'dark' ? 'bg-black text-white' :
'bg-white text-black')
</script>
<template>
  <button :class="cls">Men theme'ga moslashaman</button>
</template>
```

➡ Keyingi: 04 — Composition API & Composables

04 — Composition API & Composables

← Oldingi: 03 — Komponentlar · 🏠 README · Keyingi: 05 — Vue Router →

Bu modul Vue'ni **toza arxitektura** bilan yozishning kalitidir. Backend tajribang shu yerda juda asqotadi: composable — bu **frontend'dagi "service"**, logikani UI'dan ajratib, qayta ishlatiladigan qilib o'raydigan birlik.

Options API vs Composition API

Eski Vue (2) — **Options API**: logika `data`, `methods`, `computed`, `watch` "qutilariga" bo'linadi. Bitta feature (masalan, "qidiruv") kodi 4 ta joyga tarqaladi.

Yangi — **Composition API** (`<script setup>`): logikani **feature bo'yicha** birga ushlaysan va composable'larga ajratasan.

Options API (feature tarqalgan):
jamlangan):

```
data: { search, results }  
methods: { doSearch }  
computed: { filtered }  
watch: { search }  
ishlatsa bo'ladi
```

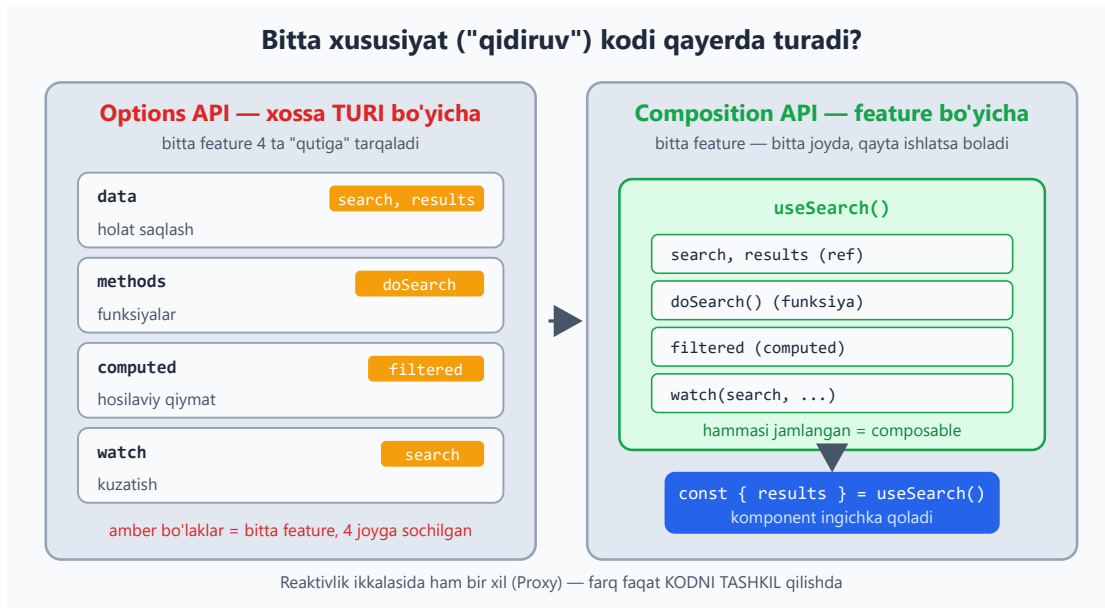
Composition API (feature

```
— useSearch() —  
  search, results | hammasi  
  doSearch       | bir joyda,  
  filtered        | qayta  
                  ]
```

Laravel analogiyasi: Options API — "fat controller" (hamma narsa bitta klassda, metodlarga bo'lingan). Composition API — logikani **service** va **action** klasslariga ajratish (DDD'dagi kabi). Composable = injektsiya qilinadigan reusable xizmat.

Quyidagi diagramma bitta "qidiruv" feature'i ikki uslubda qanday joylashishini taqqoslaydi (reaktivlik ikkalasida ham bir xil Proxy mexanizmida — farq faqat kodni tashkil qilishda):

Bitta xususiyat ("qidiruv") kodi qayerda turadi?



4.1 <script setup> — chuqurroq

<script setup> ichida: - Top-level e'lon qilingan har narsa avtomatik template'ga ochiladi. - defineProps, defineEmits, defineModel, defineExpose — kompilyator makrolari (import shart emas). - Kod komponent yaratilganda bir marta ishlaydi (setup() tanasi kabi).

```
<script setup>
import { ref, computed, onMounted } from 'vue'

const count = ref(0)
const double = computed(() => count.value * 2)

onMounted(() => console.log('DOM tayyor'))

// hammasi avtomatik template uchun ochiq
</script>
```

defineExpose — komponent ichidan tashqariga metod ochish

<script setup> default'da hamma narsani **yopiq** qiladi (ota template ref orqali bola ichiga kira olmaydi). Ataylab ochish kerak bo'lsa:

```
<!-- Child.vue -->
<script setup>
import { ref } from 'vue'
const isOpen = ref(false)
function open() { isOpen.value = true }
```

```
defineExpose({ open }) // ota faqat shularni ko'radi
</script>
```

```
<!-- Parent -->
<script setup>
import { ref } from 'vue'
const childRef = ref(null)
</script>
<template>
  <Child ref="childRef" />
  <button @click="childRef.open()">Bolani och</button>
</template>
```

4.2 Lifecycle hooks (hayot tsikli)

Komponent yaratilishidan yo'q qilinishigacha bo'lgan bosqichlarga "ulanish":

```
<script setup>
import { onMounted, onUpdated, onUnmounted, onBeforeMount,
        onBeforeUnmount, onErrorCaptured } from 'vue'

onBeforeMount(() => {}) // DOM'ga joylashdan oldin
onMounted(() => {
  // DOM tayyor - DOM o'lchash, 3rd-party kutubxona init, fetch,
  addEventListener
})
onUpdated(() => {}) // reaktiv o'zgarish DOM'ga tushgach
onBeforeUnmount(() => {}) // o'chishdan oldin - tozalashga eng yaxshi
joy
onUnmounted(() => {
  // listener'larni olib tashlash, interval clear, socket yopish
})
onErrorCaptured((err) => { /* bola xatosini tutib olish */ return false
})
</script>
```

Eng ko'p ishlatiladigani — **onMounted** (DOM/fetch boshlash) va **onUnmounted** (tozalash).

MUHIM — leak'dan saqlanish:

```
onMounted(() => {
  const id = setInterval(tick, 1000)
  window.addEventListener('resize', onResize)

  onUnmounted(() => { // har doim juftini tozala
    clearInterval(id)
```

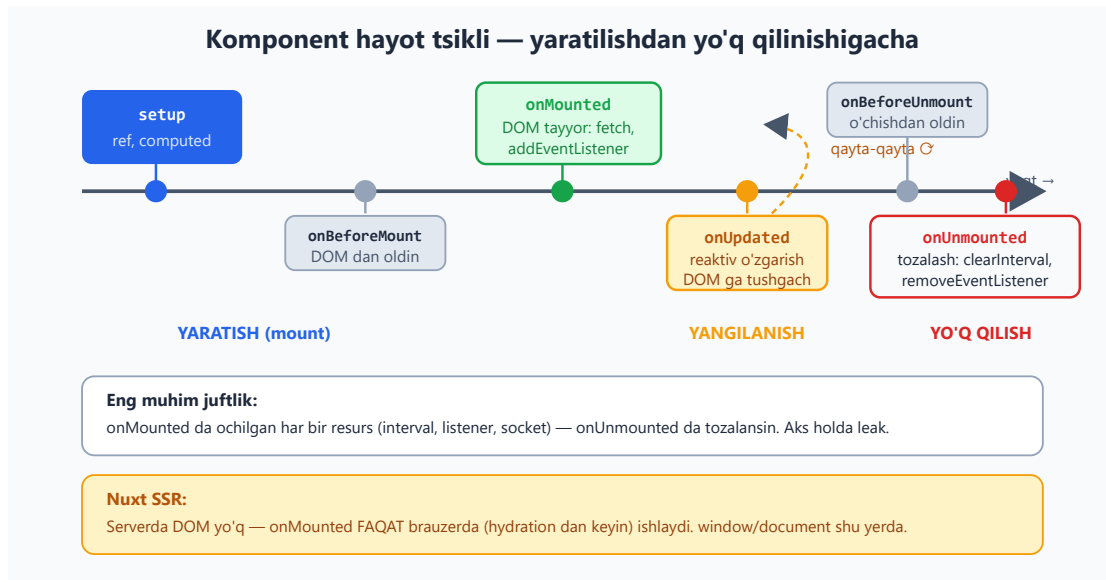
```

    window.removeEventListener('resize', onResize)
  })
})

```

Nuxt SSR'da `onMounted` faqat **brauzerda** ishlaydi (server'da DOM yo'q). Bu — `window / document` ga murojaatni `onMounted` ichida qilish kerakligining sababi.

Quyidagi vaqt o'qi hooklarning chaqirilish tartibini ko'rsatadi (`setup` → `onMounted` → `onUpdated` → `onUnmounted`), jumladan SSR/hydration'dagi xulq:



4.3 Composable — qayta ishlatiladigan logika (ENG MUHIM)

Composable = reaktiv holat + logikani o'rab, qayta ishlatish uchun `use...()` funksiyasi. Vue'ning "custom hook"i.

Qoidalar

1. Nomi `use` bilan boshlanadi: `useCounter`, `useFetch`, `useAuth`.
2. Reaktiv qiymatlar (`ref`, `computed`) va funksiyalarni **qaytaradi**.
3. Odatda `composables/` papkada (Nuxt'da bu papka auto-import qilinadi).

Eng oddiy misol

```

// composables/useCounter.js
import { ref, computed } from 'vue'

```

```
export function useCounter(initial = 0, step = 1) {
  const count = ref(initial)
  const double = computed(() => count.value * 2)

  const inc = () => count.value += step
  const dec = () => count.value -= step
  const reset = () => count.value = initial

  return { count, double, inc, dec, reset } // ref'larni qaytaramiz
}
```

```
<script setup>
import { useCounter } from '@composables/useCounter'

const { count, double, inc, dec, reset } = useCounter(10, 2)
// destructure qilsa ham reaktivlik saqlanadi - chunki ular ref!
</script>

<template>
  <p>{{ count }} (x2 = {{ double }})</p>
  <button @click="inc">+</button>
  <button @click="dec">-</button>
  <button @click="reset">Reset</button>
</template>
```

Eslatma: composable'dan **ref qaytarganing uchun** destructure reaktivlikni buzmaydi (02-modulda `reactive destructure` muammosini ko'rgansan). Shuning uchun composable'lar odatda ref qaytaradi.

Real composable — useFetch (soddalashtirilgan)

```
// composables/useFetch.js
import { ref } from 'vue'

export function useFetch(url) {
  const data = ref(null)
  const error = ref(null)
  const loading = ref(false)

  async function execute() {
    loading.value = true
    error.value = null
    try {
      const res = await fetch(url)
      if (!res.ok) throw new Error(`HTTP ${res.status}`)
      data.value = await res.json()
    } catch (e) {
      error.value = e
    } finally {

```

```

        loading.value = false
    }
}

execute()
return { data, error, loading, refetch: execute }
}

```

```

<script setup>
import { useFetch } from '@composables/useFetch'
const { data, loading, error, refetch } =
useFetch('https://api.example.com/users')
</script>

<template>
  <p v-if="loading">Yuklanmoqda...</p>
  <p v-else-if="error">Xato: {{ error.message }}</p>
  <ul v-else>
    <li v-for="u in data" :key="u.id">{{ u.name }}</li>
  </ul>
  <button @click="refetch">Qayta</button>
</template>

```

Bu **bitta** composable'ni 100 ta komponentda ishlatasan. DRY, testlanadigan, toza. (Nuxt'da `useFetch` allaqachon mavjud — 08-modul.)

Composable ichida lifecycle va cleanup

```

// composables/useMouse.js
import { ref, onMounted, onUnmounted } from 'vue'

export function useMouse() {
  const x = ref(0)
  const y = ref(0)

  function update(e) { x.value = e.clientX; y.value = e.clientY }

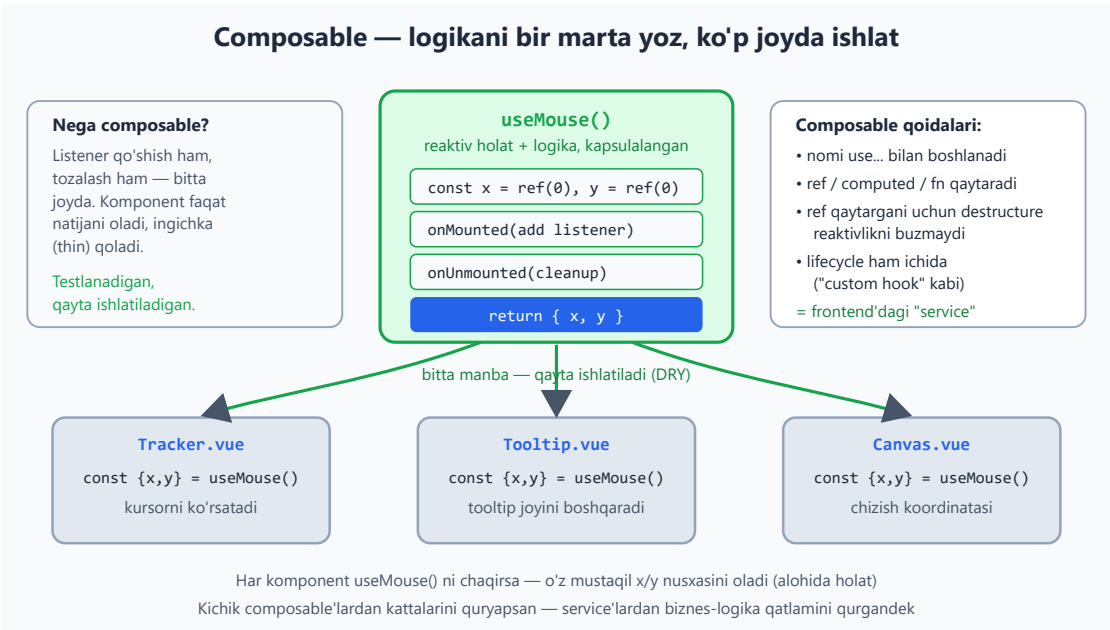
  onMounted(() => window.addEventListener('mousemove', update))
  onUnmounted(() => window.removeEventListener('mousemove', update))

  return { x, y }
}

```

Composable ichida `onMounted` / `onUnmounted` ishlatish mumkin — ular composable'ni chaqirgan komponentga "ulanadi". Bu — logikani **to'liq** kapsulalash: listener qo'shish ham, tozalash ham composable ichida. Komponent faqat `const { x, y } = useMouse()` deydi.

Quyidagi diagramma bitta composable bir nechta komponentda qanday qayta ishlatilishini ko'rsatadi (har komponent o'z mustaqil reaktiv nusxasini oladi):



Composable'larni birga ishlatish (compose qilish)

```
export function useUserProfile(userId) {
  const { data: user, loading } = useFetch(`/api/users/${userId}`)
  const { data: posts } = useFetch(`/api/users/${userId}/posts`)
  const isLoading = computed(() => !!user.value && !!posts.value)
  return { user, posts, loading, isLoading }
}
```

Kichik composable'lardan kattalarini quryapsan — xuddi service'lardan biznes-logika qatlamini qurgandek.

4.4 Composable vs boshqa yondashuvlar

Vosita	Qachon
Composable	Reaktiv holat + logika (stateful). State'ni baham ko'rish/qayta ishlatish
Util funksiya	Sof, holati yo'q yordamchi (<code>formatDate</code> , <code>slugify</code>) — <code>utils/</code>
Komponent	Vizual UI bo'lagi

Vosita	Qachon
Pinia store	Butun ilova bo'ylab bitta global holat (auth, cart)
provide/inject	Daraxtning ma'lum shoxi uchun lokal kontekst

Sezgi: Logika **UI markup'siz** va **qayta ishlatilsa** → composable. Faqat bitta komponentda ishlatilsa va kichik bo'lsa → komponent ichida qoldir.

4.5 Advanced reactivity (composable yozayotganda kerak bo'ladi)

```
import { shallowRef, triggerRef, customRef, toValue, readonly } from 'vue'

// shallowRef - faqat .value almashishini kuzatadi (ichini emas). Katta obyektlar uchun.
const big = shallowRef({ huge: 'data' })
big.value = { huge: 'new' } // kuzatiladi
big.value.huge = 'x' // kuzatilmAYDI (ichki) - triggerRef(big) kerak

// toValue - ref/getter/oddiy qiymatni "yechadi" (composable arg moslashuvchanligi uchun)
function useX(source) { // source: ref | getter | qiymat - barchasini qabul qiladi
  const val = toValue(source)
}

// readonly - o'zgartirib bo'lmaydigan nusxa (store'dan tashqariga immutable berish)
const state = reactive({ count: 0 })
const ro = readonly(state) // ro.count = 1 → warning
```

`toValue` — kuchli composable yozishda muhim: foydalanuvchi `useX(myRef)`, `useX(() => x)` yoki `useX(5)` bersa ham ishlaydi.

4.6 Composable papka strukturasi (EduCore uchun namuna)



```

composables/
├─ useAuth.ts           # login, logout, current user
├─ useApi.ts            # asosiy fetch wrapper (token, baseUrl)
├─ usePagination.ts     # sahifalash logikasi
├─ useDebounce.ts       # debounce util-composable
├─ useToggle.ts         # boolean toggle
├─ useTenant.ts         # multi-tenant: joriy tenant konteksti
├─ useTable.ts          # qidiruv+filter+sort birlashgan jadval
└─ logikasi

```

Bu — frontend'dagi "application layer". Komponentlar ingichka (thin) qoladi, logika composable'larda — xuddi controller'lar ingichka, logika service/action'larda bo'lgani kabi.

Xulosa

- **Composition API** logikani feature bo'yicha jamlaydi (Options API tarqatadi)
- `<script setup>` — eng qisqa, zamonaviy uslub; `defineExpose` bilan tanlab ochish
- **Lifecycle:** `onMounted` (boshlash), `onUnmounted` (tozalash) — leak'dan saqlan
- **Composable** = `use...()`, `ref/computed/fn` qaytaradi, qayta ishlatiladigan reaktiv logika (frontend "service")
- Composable ichida lifecycle + cleanup → to'liq kapsulalash
- `toValue`, `shallowRef`, `readonly` — kuchli composable vositalari
- Toza arxitektura: **ingichka komponent + boy composable**



Masalalar (kamida 22 ta)

Lifecycle (1–5)

1. `onMounted` da `console.log` qil, `onUnmounted` da yana — `v-if` bilan komponentni o'chirib/yoqib ketma-ketlikni kuzat.
2. **Soat (★):** `onMounted` da `setInterval` bilan har soniya vaqtni yangila; `onUnmounted` da `clearInterval`. Tozalashni unutsang nima bo'lishini izohla.
3. **Window resize (★):** Oyna kengligini ekranda ko'rsat (`resize` listener + cleanup).
4. **Scroll position (★):** Sahifa scroll qiymatini kuzatib ko'rsat; tozala.

5. `onErrorCaptured` (★★): Ataylab xato tashlaydigan bola yarat, ota uni tutib "Xatolik yuz berdi" ko'rsatsin.

Asosiy composable'lar (6–13)

1. `useToggle(initial=false)` → `{ value, toggle, setTrue, setFalse }`. Modal/sidebar'da ishlat.
2. `useCounter(initial, step)` → `{ count, inc, dec, reset, double }` (yuqoridagini o'zing qayta yoz).
3. `useLocalStorage(key, default)` (★): `localStorage` bilan sinxron reaktiv ref (`watch` ichida saqla, init'da o'qi).
4. `useDebounce(value, delay)` (★): ref'ni debounce qilingan reaktiv qiymatga aylantir.
5. `useMouse()` → `{ x, y }` (listener + cleanup composable ichida).
6. `useWindowSize()` (★) → `{ width, height }`, reaktiv.
7. `useClipboard()` (★): `{ copy, copied }` — matn nusxalash, 2s "copied" holati.
8. `useInterval(callback, ms)` (★): start/stop boshqaruvi bilan.

Data composable'lari (14–18)

1. `useFetch(url)` (★): yuqoridagini qayta yoz; `loading/error/data/refetch` bilan.
2. `usePagination(items, perPage)` (★★): `currentPage`, `totalPages`, `paginatedItems`, `next/prev/goTo`.
3. `useSearch(items, keys)` (★★): qidiruv matni bo'yicha `keys` lar ichidan filtrlangan ro'yxat qaytarsin.
4. `useSort(items)` (★★): ustun bo'yicha `asc/desc` saralash, holat boshqaruvi bilan.
5. `useTable` (★★★): 15+16+17 ni birlashtir — bitta composable qidiruv+filter+sort+pagination beradigan. EduCore jadvallari uchun asos.

Arxitektura va kompozitsiya (19–24)

1. **Compose qilish** (★★): `useUserProfile(id)` ni `useFetch` ustiga qur (user + posts birga).
2. **toValue** (★★): `useDoubled(source)` yoz — `source` ref, getter yoki oddiy son bo'lsa ham ishlasin (`toValue`).
3. **defineExpose** (★★): `VideoPlayer` komponenti `play()` / `pause()` metodlarini `defineExpose` qilsin; ota tugma orqali boshqarsin.

4. **Composable + komponent (★★):** `useFetch` + `DataList` (03-modul scoped slot) ni birlashtirib, har qanday API ro'yxatini ko'rsatadigan reusable blok yarat.
 5. **Refactor (★★★):** Quyidagi "fat" komponentni composable(lar)ga ajrat: > Bitta komponentda: qidiruv inputi + API'dan ro'yxat olish + filterlash + pagination + `localStorage`'ga oxirgi qidiruvni saqlash — hammasi aralashgan. Buni `useSearch`, `useFetch`, `usePagination`, `useLocalStorage` ga bo'lib, komponentni ingichka qil.
 6. **EduCore useAuth skeleti (★★★):** `user` (ref), `isAuthenticated` (computed), `login(creds)`, `logout()`, `fetchUser()` — token'ni `localStorage` bilan sinxronla (hozircha API'ni soxta qil).
-

✓ Tanlangan yechimlar

6 — useToggle

```
// composables/useToggle.js
import { ref } from 'vue'
export function useToggle(initial = false) {
  const value = ref(initial)
  const toggle = () => value.value = !value.value
  const setTrue = () => value.value = true
  const setFalse = () => value.value = false
  return { value, toggle, setTrue, setFalse }
}
```

8 — useLocalStorage

```
// composables/useLocalStorage.js
import { ref, watch } from 'vue'
export function useLocalStorage(key, defaultValue) {
  const stored = localStorage.getItem(key)
  const data = ref(stored ? JSON.parse(stored) : defaultValue)

  watch(data, (val) => {
    localStorage.setItem(key, JSON.stringify(val))
  }, { deep: true })

  return data
}
// Ishlatish: const theme = useLocalStorage('theme', 'dark')
```



15 — usePagination



```
// composables/usePagination.js
import { ref, computed, toValue } from 'vue'
export function usePagination(items, perPage = 10) {
  const currentPage = ref(1)
  const list = computed(() => toValue(items)) // ref yoki oddiy massiv bo'lsa ham

  const totalPages = computed(() =>
    Math.max(1, Math.ceil(list.value.length / perPage))
  )
  const paginatedItems = computed(() => {
    const start = (currentPage.value - 1) * perPage
    return list.value.slice(start, start + perPage)
  })

  const next = () => { if (currentPage.value < totalPages.value)
    currentPage.value++ }
  const prev = () => { if (currentPage.value > 1) currentPage.value-- }
  const goTo = (p) => { currentPage.value = Math.min(Math.max(1, p),
    totalPages.value) }

  return { currentPage, totalPages, paginatedItems, next, prev, goTo }
}
```



18 — useTable (qisqartirilgan tuzilma)



```
// composables/useTable.js
import { ref, computed } from 'vue'
export function useTable(source, { searchKeys = [], perPage = 10 } = {}) {
  const search = ref('')
  const sortKey = ref(null)
  const sortDir = ref('asc')
  const page = ref(1)

  const searched = computed(() => {
    if (!search.value) return source.value
    const q = search.value.toLowerCase()
    return source.value.filter(row =>
      searchKeys.some(k => String(row[k]).toLowerCase().includes(q))
    )
  })
  const sorted = computed(() => {
    if (!sortKey.value) return searched.value
    return [...searched.value].sort((a, b) => {
      const r = a[sortKey.value] > b[sortKey.value] ? 1 : -1
      return sortDir.value === 'asc' ? r : -r
    })
  })
  const totalPages = computed(() => Math.max(1,
    Math.ceil(sorted.value.length / perPage)))
  const rows = computed(() => {
    const start = (page.value - 1) * perPage
    return sorted.value.slice(start, start + perPage)
  })
  function setSort(key) {
    if (sortKey.value === key) sortDir.value = sortDir.value === 'asc' ?
    'desc' : 'asc'
    else { sortKey.value = key; sortDir.value = 'asc' }
    page.value = 1
  }
  return { search, sortKey, sortDir, page, rows, totalPages, setSort }
}
```

➡ Keyingi: 05 — Vue Router

05 — Vue Router

← Oldingi: 04 — Composition API · 🏠 README · Keyingi: 06 — Pinia →

SPA (Single Page Application)'da sahifa **qayta yuklanmaydi** — URL o'zgaradi, JS kerakli komponentni almashtiradi. Vue Router shuni boshqaradi.

Laravel analogiyasi: `routes/web.php` — URL'ni controller'ga bog'laysan. Vue Router — URL'ni **komponentga** bog'laydi. Route guard'lar — Laravel **middleware** (`auth`, `can`) ning frontend ekvivalenti.

Eslatma: **Nuxt'da Vue Router'ni qo'lda sozlamaysan** — fayl tuzilishi avtomatik route yasaydi (07-modul). Lekin ostidagi mexanizm shu Vue Router. Buni bilish — Nuxt'ni tushunish uchun shart.

5.1 O'rnatish va asosiy setup

```
npm install vue-router@4
```

```
// router/index.js
import { createRouter, createWebHistory } from 'vue-router'
import Home from '@views/Home.vue'
import About from '@views/About.vue'

const routes = [
  { path: '/', name: 'home', component: Home },
  { path: '/about', name: 'about', component: About },
]

export const router = createRouter({
  history: createWebHistory(), // toza URL (/about).
  createWebHashHistory() - /#/about
  routes,
})
```

```
// main.js
import { createApp } from 'vue'
import App from './App.vue'
import { router } from './router'

createApp(App).use(router).mount('#app')
```

```

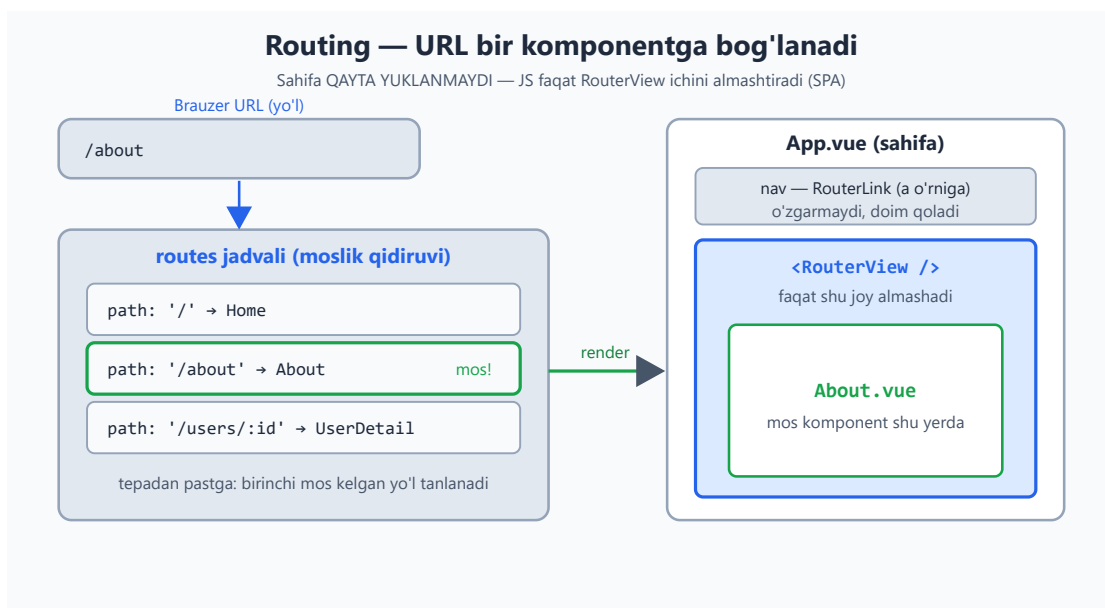
<!-- App.vue -->
<template>
  <nav>
    <RouterLink to="/">Bosh sahifa</RouterLink>
    <RouterLink :to="{ name: 'about' }">Biz haqimizda</RouterLink>
  </nav>

  <RouterView />    <!-- bu yerda joriy route komponenti render bo'ladi -->
</template>

```

- **<RouterLink>** — **<a>** o'rniga (sahifa qayta yuklanmaydi). Faol linkga avtomatik `.router-link-active` klassi qo'shiladi.
- **<RouterView>** — joriy route'ga mos komponent shu yerda chiqadi (Laravel `@yield('content')` kabi).
- **to** — string (`/about`) yoki obyekt (`{ name: 'about' }`). **Nomli route afzal** — URL o'zgarsa, link buzilmaydi.

Umumiy mexanizm quyidagicha: URL `routes` jadvaliga solishtiriladi, mos komponent `<RouterView>` ichida render bo'ladi, sahifaning qolgan qismi o'zgarmaydi:



5.2 Dynamic routes (parametrlar)

```

const routes = [
  { path: '/users/:id', name: 'user', component: UserDetails },
  { path: '/posts/:category/:slug', component: Post },
  { path: '/files/:path(.*)', component: File },    // wildcard (qolgan

```

```
hammasi)
]
```

```
<!-- UserDetails.vue -->
<script setup>
import { useRoute } from 'vue-router'
const route = useRoute()

console.log(route.params.id)      // URL'dagi :id
console.log(route.query.tab)      // ?tab=info
console.log(route.path)           // /users/5
</script>

<template>
  <h1>User #{{ route.params.id }}</h1>
</template>
```

MUHIM — param o'zgarganda komponent qayta yaratilmaydi: `/users/1` → `/users/2` ga o'tilganda Vue komponentni **qayta ishlatadi** (qayta create qilmaydi), shuning uchun `onMounted` qaytadan ishlamaydi. Param o'zgarishini **kuzatish** kerak:

```
import { watch } from 'vue'
import { useRoute } from 'vue-router'
const route = useRoute()

watch(() => route.params.id, (newId) => {
  fetchUser(newId) // har param o'zgarganda qayta yukla
}, { immediate: true })
```

Bu — eng ko'p uchraydigan router bug'i. Yodda tut.

5.3 Dasturiy navigatsiya (programmatic)

```
<script setup>
import { useRouter } from 'vue-router'
const router = useRouter()

function goToProfile(id) {
  router.push({ name: 'user', params: { id } })
}

router.push('/about') // string
router.push({ path: '/users/5', query: { tab: 'posts' } })
router.replace('/login') // tarixga qo'shmaydi (back ishlamaydi)
router.back() // brauzer "orqaga"
router.go(-2)
</script>
```

`useRoute()` (joriy route ma'lumoti, **o'qish**) va `useRouter()` (navigatsiya, **harakat**) — ikkisini chalkashtirma. Route = ma'lumot, Router = boshqaruv.

5.4 Nested routes (ichma-ich)

Layout ichida o'zgaruvchi qism (masalan, dashboard'da yon menyu doim qoladi, o'ng tomon o'zgaradi):

```
const routes = [
  {
    path: '/dashboard',
    component: DashboardLayout,
    children: [
      { path: '', name: 'dash-home', component: DashHome }, //
    ],
  },
  { path: '/dashboard/profile', component: Profile }, //
  { path: '/dashboard/settings', component: Settings }, //
]
```

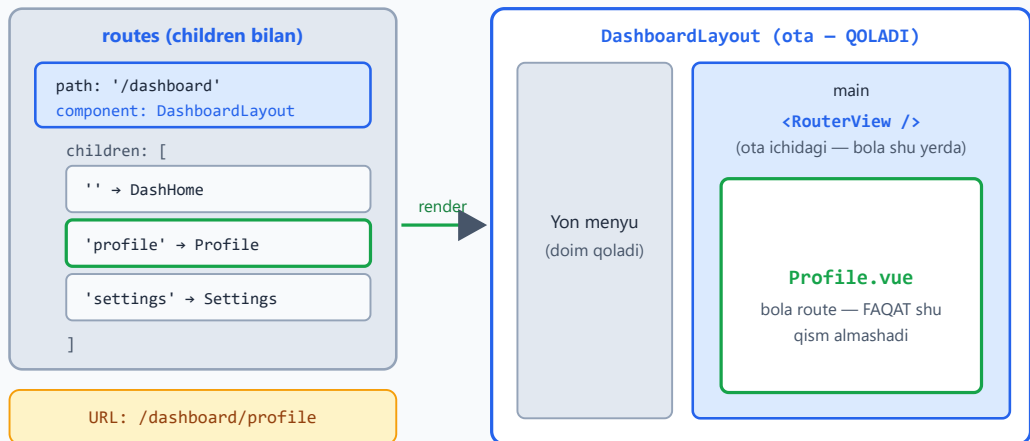
```
<!-- DashboardLayout.vue -->
<template>
  <aside>Yon menyu (doim qoladi)</aside>
  <main>
    <RouterView /> <!-- bola route shu yerda -->
  </main>
</template>
```

Har nest darajasiga bittadan `<RouterView>`. Ota komponent qoladi, ichidagi `<RouterView>` bola route bilan almashadi.

Buni vizual ko'rinishda quyidagicha tasavvur qil:

Nested routes — ichma-ich RouterView

Har nest darajasiga bittadan RouterView; ota qoladi, ichidagi bola almashadi



/profile dan /settings ga o'tsa: ota va yon menyu o'sha-o'sha, ichidagi bola almashadi

5.5 Lazy loading (route-level code splitting)

Har komponentni boshidan yuklash o'rniga — kerak bo'lganda yuklash. Bundle hajmi kichrayadi, ilk yuklanish tezlashadi:

```
const routes = [
  { path: '/', component: Home }, // bu darrov kerak - eager
  {
    path: '/admin',
    component: () => import('@views/Admin.vue'), // lazy - alohida chunk
  },
]
```

`() => import(...)` — Vite alohida `.js` chunk yasaydi, faqat `/admin` ga kirilganda yuklanadi. Katta ilovalarda **doim** og'ir/kam ishlatiladigan sahifalarni lazy qil.

5.6 Navigation guards — middleware ekvivalenti

Navigatsiyani to'xtatish/yo'naltirish (auth tekshirish, ruxsat, "saqlanmagan o'zgarishlar" ogohlantirishi):

Global guard (`beforeEach`)

```
// router/index.js
router.beforeEach((to, from) => {
  const isAuth = !!localStorage.getItem('token')

  if (to.meta.requiresAuth && !isAuth) {
    return { name: 'login', query: { redirect: to.fullPath } } //
    yo'naltir
  }
  // return true yoki hech narsa – davom et
})
```

Route'ga `meta` qo'shasan:

```
{ path: '/admin', component: Admin, meta: { requiresAuth: true, role:
'admin' } }
```

Per-route guard (`beforeEnter`)

```
{
  path: '/admin',
  component: Admin,
  beforeEnter: (to, from) => {
    if (!isAdmin()) return { name: 'home' }
  },
}
```

In-component guard (`onBeforeRouteLeave`)

```
import { onBeforeRouteLeave } from 'vue-router'

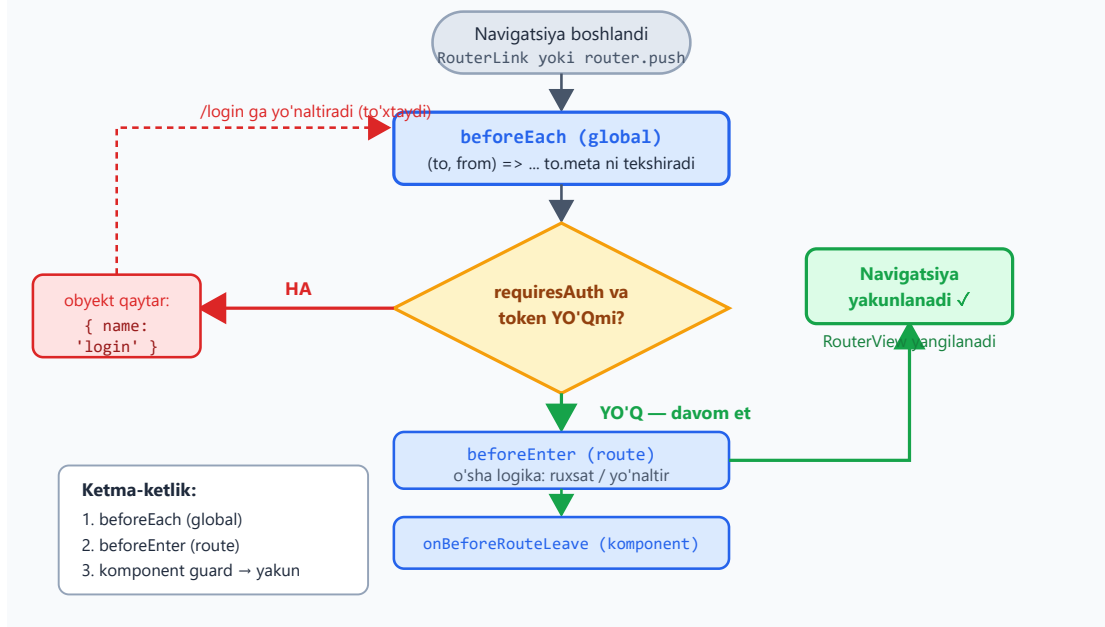
onBeforeRouteLeave((to, from) => {
  if (hasUnsavedChanges.value) {
    return window.confirm('Saqlanmagan o'zgarishlar bor. Chiqasizmi?')
  }
})
```

Guard'lar ketma-ketligi (soddalashtirilgan): `beforeEach` (global) → `beforeEnter` (route) → komponent ichidagi guard'lar → navigatsiya yakunlanadi.

Quyidagi diagramma bu oqimni ko'rsatadi — har bosqich "davom et yoki to'xtat/yo'naltir" deb qaror qiladi:

Navigation guard oqimi (middleware ekvivalenti)

Har bosqich qaror qiladi: davom et (true) yoki to'xtat/yo'naltir (obyekt qaytar)



Bu Laravel middleware pipeline'iga juda o'xshaydi: har bosqich "davom et yoki to'xtat/yo'naltir" deb qaror qiladi.

5.7 Foydali narsalar

```
// Scroll xulqi (yangi sahifada tepaga)
const router = createRouter({
  history: createWebHistory(),
  routes,
  scrollBehavior(to, from, savedPosition) {
    if (savedPosition) return savedPosition // back/forward'da eski
    joyga
    if (to.hash) return {el: to.hash} // #section ga
    return {top: 0}
  },
})

// 404
{ path: '/*', name: 'not-found', component: NotFound }

// Redirect & alias
{ path: '/home', redirect: '/' }
{ path: '/', component: Home, alias: '/bosh' }

// Props orqali param uzatish (route'ga bog'liqlikni kamaytiradi)
{ path: '/users/:id', component: UserDetail, props: true }
// UserDetail endi route'ni emas, oddiy `id` prop'ini qabul qiladi
```

<RouterView> da transition + KeepAlive:

```
<RouterView v-slot="{ Component }">
  <Transition name="fade" mode="out-in">
    <KeepAlive>
      <component :is="Component" />
    </KeepAlive>
  </Transition>
</RouterView>
```

Xulosa

- `createRouter` + `createWebHistory`, `<RouterLink>` + `<RouterView>`
- **Dynamic param:** `:id` → `route.params.id`; param o'zgarsa **watch qil** (komponent qayta yaratilmaydi)
- `useRoute()` = o'qish, `useRouter()` = harakat (`push/replace/back`)
- **Nested routes** — layout + bola `<RouterView>`
- **Lazy loading** — `() => import(...)`, bundle splitting
- **Guards** = middleware: global `beforeEach`, route `beforeEnter`, komponent `onBeforeRouteLeave`; meta bilan auth/role
- Nomli route + `props: true` — toza, barqaror

Masalalar (kamida 20 ta)

Asosiy (1–7)

1. 3 sahifali sayt: Home, About, Contact. Nav + `<RouterView>`. `<RouterLink>` bilan o'tib ko'r, sahifa qayta yuklanmasligini tasdiqla.
2. Faol linkni `.router-link-active` orqali boshqacha rangga bo'ya.
3. Nomli route'lar (`name`) qo'shib, `:to="{ name: '...' }"` bilan navigatsiya qil.
4. `router.push` bilan dasturiy o'tish: tugma bosilganda `/about` ga o't.
5. 404 sahifa qo'sh (`/:pathMatch(.*)*`); mavjud bo'lmagan URL'ga kirib sina.
6. `/home` → `/` redirect qo'sh.
7. `scrollBehavior` bilan har yangi sahifada tepaga scroll qil.

Dynamic & data (8–13)

1. `/users/:id` route; sahifada `route.params.id` ni ko'rsat.
2. **Param watch (★)**: Foydalanuvchilar ro'yxatidan `/users/1`, `/users/2` ga o'tganda sahifa ma'lumoti yangilansin (`watch(() => route.params.id, ..., {immediate:true})`).
3. Query string (★): `/search?q=vue` — `route.query.q` ni ko'rsat; inputga yozsang URL query'ni `router.push` bilan yangila.
4. `props: true` (★): `UserDetail` ni route'ga bog'lamasdan, `id` ni prop sifatida qabul qildir.
5. **Master-detail (★★)**: `/products` ro'yxat → element bosilsa `/products/:id` detail sahifa; "Orqaga" tugmasi `router.back()`.
6. **Pagination URL'da (★★)**: `/posts?page=2` — sahifa raqami query'da saqlansin, refresh'da ham qolsin.

Nested & lazy (14–17)

1. **Dashboard layout (★★)**: `/dashboard` layout + `children: ''` (home), `profile`, `settings`. Yon menyu doim qolsin, o'ng tomon almashsin.
2. Ikki darajali nested (★★): `/dashboard/users` ichida yana `/dashboard/users/:id` nest qil.
3. Lazy loading (★): `Admin` sahifasini `() => import()` bilan yukla; `Network` tab'da alohida chunk yuklanishini ko'rsat.
4. Transition (★★): `<RouterView v-slot>` bilan sahifalar orasiga fade animatsiya qo'sh.

Guards (18–23)

1. **Auth guard (★★)**: `meta.requiresAuth` bo'lgan `/dashboard` ni himoyala; token yo'q bo'lsa `/login` ga yo'naltir (`beforeEach`).
2. **Redirect after login (★★)**: Login'da `query.redirect` ni ishlatib, foydalanuvchini kelgan sahifasiga qaytar.
3. **Role guard (★★)**: `meta.role: 'admin'` — admin bo'lmasa `/403` ga.
4. **Leave guard (★★)**: Forma sahifasida o'zgarish bo'lsa, `onBeforeRouteLeave` bilan "Saqlanmagan o'zgarishlar bor" deb ogohlantir.
5. **Title (★)**: `beforeEach` da `to.meta.title` ni `document.title` ga yoz.
6. **EduCore router skeleti (★★★)**: `/login` (public), `/app` (auth layout + children: dashboard, students, payments), `/admin` (role: admin). Guard'larni `meta` +

`beforeEach` bilan to'liq qur. (Bu Nuxt'da fayl-tuzilish bilan ko'rinishini 07-modulda solishtirasan.)

✓ Tanlangan yechimlar

9 — Param watch bilan data yangilash

```
<!-- UserDetails.vue -->
<script setup>
import { ref, watch } from 'vue'
import { useRoute } from 'vue-router'
const route = useRoute()
const user = ref(null)

async function load(id) {
  // soxta "API"
  user.value = { id, name: `User ${id}` }
}

watch(() => route.params.id, (id) => load(id), { immediate: true })
</script>

<template>
<h1 v-if="user">{{ user.name }} (#{{ user.id }})</h1>
</template>
```



18 — Auth guard + redirect



```
// router/index.js
router.beforeEach((to) => {
  const isAuth = !!localStorage.getItem('token')
  if (to.meta.requiresAuth && !isAuth) {
    return { name: 'login', query: { redirect: to.fullPath } }
  }
})
```

```
// routes
{ path: '/login', name: 'login', component: Login },
{ path: '/dashboard', component: Dashboard, meta: { requiresAuth: true } },
```

```
<!-- Login.vue: muvaffaqiyatdan keyin -->
<script setup>
import { useRoute, useRouter } from 'vue-router'
const route = useRoute()
const router = useRouter()

function onLoginSuccess() {
  localStorage.setItem('token', 'xxx')
  router.push(route.query.redirect || '/dashboard')
}
</script>
```



Keyingi: 06 — Pinia

06 — Pinia (State Management)

← Oldingi: 05 — Vue Router · 🏠 README · Keyingi: 07 — Nuxt asoslari →

`ref` /composable lokal holat uchun yetadi. Lekin **butun ilova bo'ylab** baham ko'riladigan holat (joriy foydalanuvchi, savat, til, EduCore'da joriy tenant) kerak bo'lsa — **Pinia**.

Pinia — Vue'ning rasmiy state-management kutubxonasi (Vuex'ning vorisi). Soddarog, TypeScript-do'st, DevTools bilan zo'r.

Laravel analogiyasi: Pinia store — **singleton service**. Bir marta yaratiladi, butun ilova bir xil instansiyani ishlatadi. State — service property'lari, getters — accessor'lar, actions — service metodlari (biznes-logika shu yerda).

Qachon Pinia, qachon yo'q?

Holat	Yechim
Bitta komponentga tegishli	lokal <code>ref</code>
Bir necha qo'shni komponent	props/emit yoki composable
Daraxtning ma'lum shoxi	<code>provide/inject</code>
Butun ilova , ko'p joydan o'qiladi/yoziladi	Pinia

Hamma narsani store'ga tiqishtirma. Pinia — *global* holat uchun. Lokal narsa lokal qolsin.

6.1 O'rnatish

```
npm install pinia
```

```
// main.js
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import App from './App.vue'
```



```
createApp(App).use(createPinia()).mount('#app')
```

Nuxt'da: `npm i @pinia/nuxt`, keyin `nuxt.config` modules'ga qo'shasan — `createPinia` qo'lda kerak emas (07-modul).

6.2 Store yaratish — Setup syntax (tavsiya)

Composition API uslubi — eng moslashuvchan, `<script setup>` ga o'xshaydi:

```
// stores/counter.js
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'

export const useCounterStore = defineStore('counter', () => {
  // STATE → ref
  const count = ref(0)
  const history = ref([])

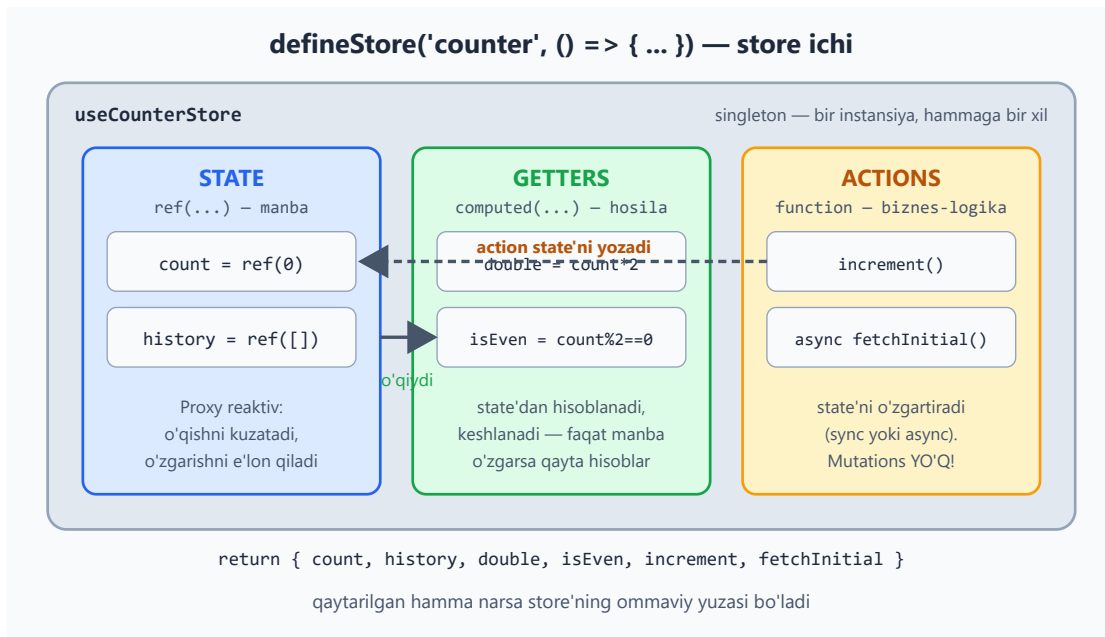
  // GETTERS → computed
  const double = computed(() => count.value * 2)
  const isEven = computed(() => count.value % 2 === 0)

  // ACTIONS → funksiyalar (sync yoki async)
  function increment() {
    count.value++
    history.value.push(count.value)
  }
  async function fetchInitial() {
    const res = await fetch('/api/counter')
    count.value = (await res.json()).value
  }

  return { count, history, double, isEven, increment, fetchInitial }
})
```

Moslik: - `ref` = state - `computed` = getter - `function` = action

Quyidagi diagramma store'ning uch qismi (state / getters / actions) o'zaro qanday bog'lanishini ko'rsatadi:



```

<script setup>
import { useCounterStore } from '@/stores/counter'
const counter = useCounterStore()
</script>

<template>
  <p>{{ counter.count }} (x2 = {{ counter.double }})</p>
  <button @click="counter.increment()"></button>
</template>

```

Option syntax (Vuex'ga o'xshash, alternativa)

```

export const useCounterStore = defineStore('counter', {
  state: () => ({ count: 0 }),
  getters: {
    double: (state) => state.count * 2,
  },
  actions: {
    increment() { this.count++; }, // this - store
    async fetchInitial() { /* await ... */ },
  },
})

```

Ikkalasi ham ishlaydi. **Setup syntax** zamonaviyroq va composable'lar bilan yaxshiroq birikadi. Bu qo'llanmada uni ishlatamiz.

6.3 Store'dan foydalanish — destructure tuzog'i

```

<script setup>
import { storeToRefs } from 'pinia'
import { useCounterStore } from '@/stores/counter'

const counter = useCounterStore()

// ✗ XATO – reaktivlikni uzadi
const { count, double } = counter

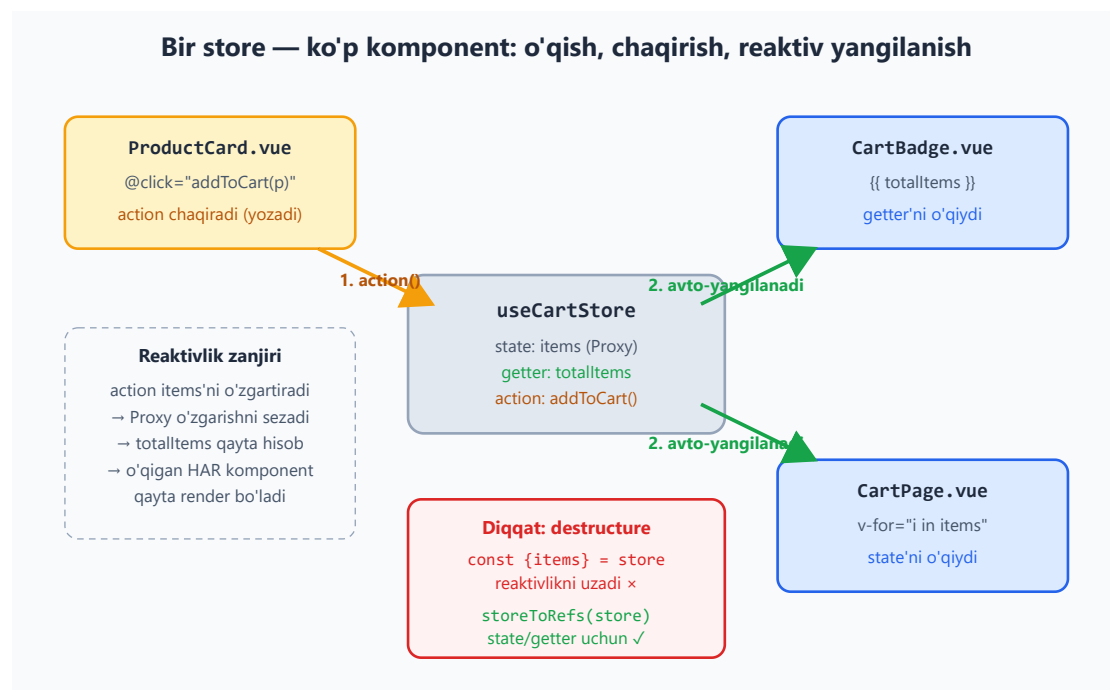
// ✓ State/getters uchun storeToRefs
const { count, double } = storeToRefs(counter)

// ✓ Actions'ni to'g'ridan-to'g'ri destructure qilsa bo'ladi (ular
funksiya, reaktiv emas)
const { increment } = counter
</script>

```

Nega? Store — `reactive` obyekt (02-modul!). Uni to'g'ridan-to'g'ri destructure qilsang, reaktivlik uziladi. `storeToRefs` state va getter'larni reaktiv ref'larga o'raydi. **Actions** esa shunchaki funksiya — to'g'ridan-to'g'ri olsa bo'ladi.

Quyidagi diagramma bitta store'ni ko'p komponent qanday baham ko'rishini ko'rsatadi: biri action chaqirib state'ni o'zgartiradi, qolganlari esa Proxy reaktivligi tufayli avtomatik yangilanadi:



6.4 State'ni o'zgartirish usullari

```
const store = useCounterStore()

// 1) action orqali (TAVSIYA – logika bir joyda, DevTools'da kuzatiladi)
store.increment()

// 2) to'g'ridan-to'g'ri (mumkin, lekin oddiy holatlar uchun)
store.count++

// 3) $patch – bir nechta o'zgarishni birga (bitta yangilanish)
store.$patch({ count: 10, name: 'x' })
store.$patch((state) => { // murakkab (massivga push va h.k.)
  state.items.push(newItem)
  state.count++
})

// 4) $reset – boshlang'ich holatga (faqat Option syntax'da avtomatik;
// Setup syntax'da o'zing reset action yozasan)
```

Vuex'dan farqli: Pinia'da **mutations yo'q**. To'g'ridan-to'g'ri o'zgartirish yoki action. Bu — kamroq boilerplate.

6.5 Store'lar bir-birini ishlatishi

```
// stores/cart.js
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'
import { useAuthStore } from '../auth'

export const useCartStore = defineStore('cart', () => {
  const auth = useAuthStore() // boshqa store'ni chaqir
  const items = ref([])

  const canCheckout = computed(() =>
    auth.isAuthenticated && items.value.length > 0
  )
  return { items, canCheckout }
})
```

Store'lar bir-birini bimalol ishlatadi — service'lar bir-birini DI orqali chaqirgani kabi. Faqat **aylanma bog'liqlik** ($A \rightarrow B \rightarrow A$) dan ehtiyot bo'l.

6.6 Real store — useAuthStore (EduCore namuna)

```
// stores/auth.js
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'

export const useAuthStore = defineStore('auth', () => {
  const user = ref(null)
  const token = ref(localStorage.getItem('token') || null)

  const isAuthenticated = computed(() => !!token.value)
  const isAdmin = computed(() => user.value?.role === 'admin')

  async function login(credentials) {
    const res = await fetch('/api/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(credentials),
    })
    if (!res.ok) throw new Error('Login xato')
    const data = await res.json()
    token.value = data.token
    user.value = data.user
    localStorage.setItem('token', data.token)
  }

  async function fetchUser() {
    if (!token.value) return
    const res = await fetch('/api/me', {
      headers: { Authorization: `Bearer ${token.value}` },
    })
    user.value = await res.json()
  }

  function logout() {
    token.value = null
    user.value = null
    localStorage.removeItem('token')
  }

  return { user, token, isAuthenticated, isAdmin, login, fetchUser,
    logout }
})
```

Router guard bilan birlashtirish (05-modul):

```
router.beforeEach((to) => {
  const auth = useAuthStore()
  if (to.meta.requiresAuth && !auth.isAuthenticated) {
    return { name: 'login' }
  }
})
```

6.7 Persist (saqlash) va pluginlar

State'ni refresh'da yo'qotmaslik uchun `localStorage` ga saqlash:

Qo'lda (oddiy):

```
import { watch } from 'vue'
// store ichida:
watch(token, (val) => {
  val ? localStorage.setItem('token', val) :
  localStorage.removeItem('token')
})
```

Plugin bilan (avtomatik, ko'p store uchun):

```
npm install pinia-plugin-persistedstate
```

```
// main.js
import piniaPersist from 'pinia-plugin-persistedstate'
const pinia = createPinia()
pinia.use(piniaPersist)
```

```
// store (Option syntax'da):
export const useAuthStore = defineStore('auth', {
  state: () => ({ token: null }),
  persist: true, // butun store localStorage'ga
})
```

O'z plugging (DI/logging uchun)

```
pinia.use(({ store }) => {
  // har store yaratilganda ishlaydi
  store.$subscribe((mutation, state) => {
    console.log(`[${store.$id}] o'zgardi`, state) // global logging
  })
})
```

`$subscribe` — store o'zgarishini global tinglash. Logging, analytics, sync uchun qulay.

6.8 Arxitektura: store nima qiladi, nima qilmaydi

Store ichiga: - Global state (auth, ui sozlamalari, savat, tenant) - O'sha state'ni o'zgartiruvchi biznes-logika (action) - Hosilaviy qiymatlar (getter)

Store ichiga EMAS: - Vizual/UI logika (komponentda qolsin) - Faqat bitta komponentga kerakli vaqtinchalik holat (lokal `ref`) - Og'ir API qatlami — uni alohida `services/api.js` ga ajratib, store action'i undan foydalansin (DDD'dagi repository/service ajratimiga o'xshash)

```
Komponent (UI) → Store (action) (state+biznes) → API service (HTTP) → Backend
```



Bu qatlamlash — backend'dagi controller → service → repository ga to'g'ridan-to'g'ri mos keladi. Sen buni allaqachon bilasan.

Xulosa

- **Pinia** — global holat (singleton service kabi); lokal narsani store'ga tiqma
- **Setup syntax:** `ref` =state, `computed` =getter, `function` =action
- **storeToRefs** — state/getter destructure uchun (actions'ni to'g'ridan-to'g'ri ol)
- O'zgartirish: action (afzal), to'g'ridan-to'g'ri, `$patch`. **Mutations yo'q**
- Store'lar bir-birini ishlatadi (DI kabi)
- **Persist** — qo'lda `watch` yoki plugin
- Qatlam: Komponent → Store → API service → Backend



Masalalar (kamida 20 ta)

Asosiy (1–7)

1. `useCounterStore` yarat (`count`, `double`, `increment`, `decrement`, `reset`). Ikki alohida komponentda ishlatib, **bir xil** state ko'rsatishini tasdiqla.
2. `storeToRefs` (★): yuqoridagi store'dan `count`, `double` ni destructure qil; to'g'ridan-to'g'ri destructure bilan farqini (reaktivlik yo'qolishini) ko'rsat.
3. `useThemeStore` (★): `theme` state + `toggle` action; har joyda joriy theme'dan foydalan.
4. `useUiStore`: `sidebarOpen`, `toggleSidebar` — header'dagi tugma va sidebar komponenti bir holatni baham ko'rsin.
5. `$patch` (★): bir nechta state'ni bitta `$patch` bilan yangila.
6. Getter parametrli (★): `getById` getter — `(id) => items.find(...)`. (Eslatma: getter funksiya qaytaradi.)

7. Store'lar bog'liqligi (★): `useCartStore` `useAuthStore` ni ishlatib, `canCheckout` (computed) ni hisoblasin.

Savat (cart) — to'liq misol (8–12)

1. **Cart store (★★)**: `items` (`[{id, name, price, qty}]`), `getter`'lar: `totalItems`, `totalPrice`, `isEmpty`.
2. `addToCart(product)` (★): mahsulot bor bo'lsa `qty++`, yo'q bo'lsa qo'sh.
3. `removeFromCart(id)`, `updateQty(id, qty)` (`qty` 0 bo'lsa o'chir), `clear()`.
4. Ul'ga ulang (★★): mahsulot ro'yxati + "Savatga" tugmasi + savat badge (`totalItems`) + savat sahifasi.
5. **Persist (★★)**: savatni `localStorage` ga saqla (qo'lda `watch` yoki plugin); refresh'da qolsin.

Auth — to'liq (13–17)

1. **Auth store (★★)**: `user`, `token`, `isAuthenticated`, `login(creds)`, `logout()` (API'ni soxta qil).
2. Router guard (★★): `isAuthenticated` ga qarab `/dashboard` ni himoyala (05-modul bilan birlashtir).
3. `isAdmin` `getter` (★) + admin-only tugma faqat adminga ko'rinsin.
4. Token persist (★★): refresh'da login holati saqlansin; ilova ochilganda `fetchUser` chaqirilsin.
5. `logout` (★): chiqishda state tozalansin va `/login` ga yo'naltirilsin.

Async & arxitektura (18–24)

1. **Async action (★★)**: `useProductsStore` — `fetchProducts()` (loading/error/data state bilan); komponent loading/error/ro'yxat holatlarini ko'rsatsin.
2. **API qatlamini ajratish (★★★)**: 18-da fetch'ni to'g'ridan-to'g'ri store'da yozmasdan, `services/api.js` (yoki composable `useApi`) ga ajrat; store o'shani chaqirsin.
3. **\$subscribe plugin (★★)**: Har store o'zgarishini console'ga loglaydigan global plugin yoz.
4. **Optimistic update (★★★)**: "Like" tugmasi — darrov UI'da `liked` qil, keyin "API" xato bersa orqaga qaytar (rollback).
5. **useTenantStore (EduCore) (★★★)**: Joriy tenant (`currentTenant`, `setTenant`, `tenantId` `getter`); barcha API so'rovlarga tenant kontekstini qo'shadigan tuzilma o'ylab top (multi-tenant frontend asosi).

6. **Notifications store (★★):** `notifications` massivi, `notify(msg, type)` (auto-dismiss 3s), `dismiss(id)`; global toast komponenti store'ni o'qisin.
7. **To-Do'ni store'ga ko'chir (★★):** Oldingi modullardagi To-Do'ni Pinia store'ga ko'chir (`todos`, `add/toggle/remove`, `activeCount / completedCount` `getter`, `persist`). Komponent endi ingichka bo'lsin.
-

✓ Tanlangan yechimlar

8-10 — Cart store

```
// stores/cart.js
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'

export const useCartStore = defineStore('cart', () => {
  const items = ref([])

  const totalItems = computed(() =>
    items.value.reduce((s, i) => s + i.qty, 0)
  )
  const totalPrice = computed(() =>
    items.value.reduce((s, i) => s + i.price * i.qty, 0)
  )
  const isEmpty = computed(() => items.value.length === 0)

  function addToCart(product) {
    const existing = items.value.find(i => i.id === product.id)
    if (existing) existing.qty++
    else items.value.push({ ...product, qty: 1 })
  }
  function updateQty(id, qty) {
    const item = items.value.find(i => i.id === id)
    if (!item) return
    if (qty <= 0) removeFromCart(id)
    else item.qty = qty
  }
  function removeFromCart(id) {
    items.value = items.value.filter(i => i.id !== id)
  }
  function clear() { items.value = [] }

  return { items, totalItems, totalPrice, isEmpty,
    addToCart, updateQty, removeFromCart, clear }
})
```



21 — Optimistic update (rollback)



```
// stores/posts.js (action ichida)
async function toggleLike(post) {
  const prev = post.liked
  post.liked = !post.liked // 1) darrov UI yangilanadi
  post.likes += post.liked ? 1 : -1
  try {
    await fakeApiToggleLike(post.id) // 2) serverga
  } catch (e) {
    post.liked = prev // 3) xato bo'lsa orqaga
    post.likes += post.liked ? 1 : -1
    throw e
  }
}
```



20 — Global logging plugin



```
// main.js
const pinia = createPinia()
pinia.use(({ store }) => {
  store.$subscribe((mutation, state) => {
    console.log(`[${store.$id}]`, mutation.type, state)
  })
})
```



Keyingi: [07 — Nuxt asoslari](#) — endi Vue ustiga "Laravel"ni qo'yamiz.

07 — Nuxt asoslari

← Oldingi: 06 — Pinia · 🏠 README · Keyingi: 08 — Data & Server →

Vue — kutubxona/freymvork. **Nuxt** — Vue ustiga qurilgan **meta-framework**: routing, SSR, server API, auto-import, SEO — hammasini "convention" bilan tayyor beradi.

Eng aniq analogiya: Vue → React kabi, Nuxt → **Laravel** kabi. Sen Laravel'da har safar router, ORM, middleware'ni noldan yozmaysan — convention bor. Nuxt ham xuddi shunday: papkaga fayl tashlaysan, route o'zi paydo bo'ladi.

Nuxt nima beradi (Vue'da o'zing qilishing kerak bo'lganini)

- **File-based routing** — `router/index.js` yozmaysan (05-modulni eslagin — endi avtomatik)
- **SSR/SSG** — server'da render (SEO, tezlik)
- **Auto-imports** — `import { ref }` yozmaysan; composable/komponentlar avtomatik
- **Server engine (Nitro)** — `/api/...` backend endpointlar shu loyihaning ichida
- **Layouts, middleware, plugins** — Laravel'dagidek tuzilma

Bu modul **Nuxt 4** strukturasiga asoslangan. Nuxt 3 farqlari belgilab ketiladi.

7.1 O'rnatish

```
npx nuxi@latest init my-nuxt-app
cd my-nuxt-app
npm install
npm run dev      # http://localhost:3000
```

Nuxt 4 papka strukturasini

```
my-nuxt-app/
├── app/                # ← brauzerga ketadigan HAMMA narsa shu yerda
  (Nuxt 4)
  ├── app.vue          # ildiz komponent (Laravel
  layouts/app.blade.php kabi)
  └── pages/            # routing — har fayl = route
```

```

|   |   | components/      # auto-import komponentlar
|   |   | composables/    # auto-import composable'lar
|   |   | layouts/        # qayta ishlatiladigan sahifa qoliplari
|   |   | middleware/      # route middleware (auth va h.k.)
|   |   | plugins/         # ilova boot'ida ishlaydigan kod
|   |   | assets/          # build qilinadigan CSS/rasm
|   |   | utils/           # auto-import sof funksiyalar
|   |   | server/          # ← Nitro backend (root'da qoladi)
|   |   |   | api/         # /api/... endpointlar
|   |   |   | ...
|   |   | shared/          # app + server BIRGA ishlatadigan kod (types,
utils)
|   |   | public/          # to'g'ridan-to'g'ri xizmat (favicon,
robots.txt)
|   |   | nuxt.config.ts    # asosiy konfiguratsiya
|   |   | package.json

```

Nuxt 3 da farq: `app/` papkasi yo'q edi — `pages/`, `components/`, `composables/`, `app.vue` to'g'ridan-to'g'ri **ildizda** turardi. Nuxt 4 ularni `app/` ichiga ko'chirdi (toza ajratish + tezroq startup). Eski tuzilma hali ishlaydi (backward compat), lekin yangi loyihada `app/` ishlat.

Nega `app/` va `server/` ajratilgan? `app/` — brauzer konteksti, `server/` — Node/Nitro konteksti. Ikki-sida turli global'lar, turli import'lar bor. Ajratish IDE type-safety va xatolarning oldini oladi. Backend dasturchi sifatida bu mantiq senga tushunarli — frontend va backend kodi aralashmasligi kerak.

7.2 `app.vue` — ildiz komponent

Eng minimal Nuxt ilovasi — faqat `app/app.vue`:

```

<!-- app/app.vue -->
<template>
  <div>
    <h1>Salom Nuxt!</h1>
  </div>
</template>

```

Routing kerak bo'lsa — `<NuxtPage />` qo'shasan (bu Vue Router'dagi `<RouterView>` ning Nuxt versiyasi):

```

<!-- app/app.vue -->
<template>
  <div>
    <AppHeader />          <!-- auto-import! import yozish shart emas -
->
    <NuxtPage />           <!-- joriy sahifa shu yerda render bo'ladi -

```

```
->
  <AppFooter />
</div>
</template>
```

`app/pages/` papkasi bo'lmasa, Nuxt `vue-router` ni umuman qo'shmaydi (faqat bitta `app.vue` — landing page uchun). Routing kerak bo'lganda `pages/` ochasan.

7.3 File-based routing — eng katta "wow"

`app/pages/` ichidagi har fayl avtomatik route bo'ladi. **`router/index.js` YO'Q.**

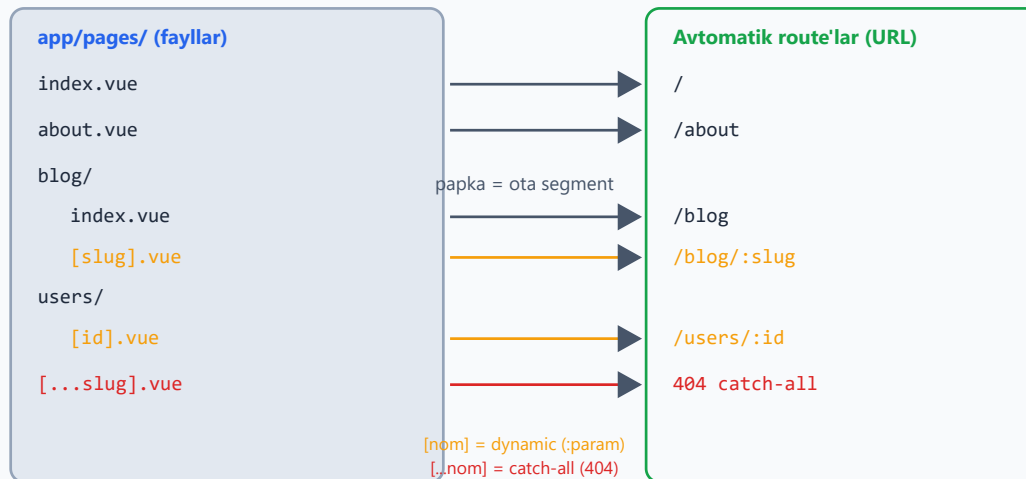
```
app/pages/
├─ index.vue           → /
├─ about.vue          → /about
├─ contact.vue        → /contact
├─ blog/
│   ├─ index.vue      → /blog
│   └─ [slug].vue     → /blog/:slug      (dynamic)
├─ users/
│   └─ [id].vue       → /users/:id
└─ [...slug].vue     → 404 catch-all
```

05-modul bilan solishtir: o'sha yerda `routes` massivini qo'lda yozgansan. Nuxt'da — papka tuzilishi = route. Bu Laravel'ning `php artisan route:list` mantiqiga o'xshaydi: convention bor, qo'lda ro'yxat yo'q.

Quyidagi diagramma `app/pages/` papkasidagi har bir fayl qanday qilib avtomatik URL route'iga aylanishini ko'rsatadi:

File-based routing: fayl tuzilishi = route ro'yxati

router/index.js YO'Q — Nuxt papkani o'qib route'larni o'zi yasaydi



Dynamic route va parametr

```
<!-- app/pages/users/[id].vue → /users/123 -->
<script setup>
const route = useRoute() // auto-import - vue-router'dan
import yo'q
const id = route.params.id
</script>

<template>
  <h1>User #{{ id }}</h1>
</template>
```

Navigatsiya — <NuxtLink>

```
<template>
  <nav>
    <NuxtLink to="/">Bosh</NuxtLink>
    <NuxtLink to="/about">Haqida</NuxtLink>
    <NuxtLink :to="/users/${user.id}">{{ user.name }}</NuxtLink>
  </nav>
</template>
```

<NuxtLink> — <RouterLink> ning aqlli versiyasi: prefetch (ko'rinishga kirganda sahifani oldindan yuklaydi), tashqi linklarni avtomatik aniqlaydi. Dasturiy navigatsiya: `navigateTo('/about')` (auto-import).

Nested routes

app/pages/dashboard.vue + app/pages/dashboard/ papka:

```
app/pages/
├── dashboard.vue      ← ota (ichida <NuxtPage/> bo'lishi kerak)
└── dashboard/
    ├── index.vue      → /dashboard
    ├── profile.vue    → /dashboard/profile
    └── settings.vue   → /dashboard/settings
```

```
<!-- app/pages/dashboard.vue -->
<template>
  <div>
    <aside>Yon menyu (doim qoladi)</aside>
    <NuxtPage />      <!-- bola sahifa shu yerda -->
  </div>
</template>
```

7.4 Layouts — qayta ishlatiladigan qoliplar

Bir nechta sahifa bir xil ramkani (header/footer/sidebar) baham ko'rsa — layout.

Laravel `@extends('layouts.app')` ning aynan o'zi.

```
<!-- app/layouts/default.vue -->
<template>
  <div>
    <AppHeader />
    <main>
      <slot />      <!-- sahifa kontenti shu yerga tushadi -->
    </main>
    <AppFooter />
  </div>
</template>
```

```
<!-- app/layouts/admin.vue -->
<template>
  <div class="admin">
    <AdminSidebar />
    <slot />
  </div>
</template>
```

Sahifada layout tanlash:

```
<!-- app/pages/admin/index.vue -->
<script setup>
```

```
definePageMeta({ layout: 'admin' }) // default'dan boshqa
</script>
```

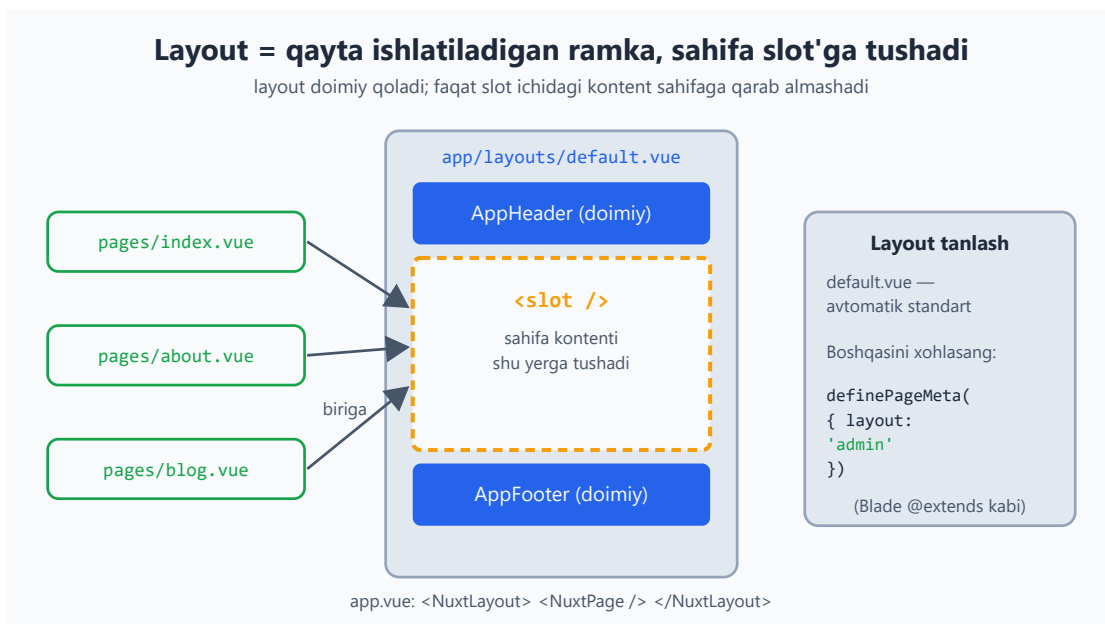
app/app.vue da `<NuxtLayout>` bo'lishi kerak (yoki Nuxt avtomatik o'raydi):

```
<!-- app/app.vue -->
<template>
  <NuxtLayout>
    <NuxtPage />
  </NuxtLayout>
</template>
```

default.vue nomli layout — avtomatik standart. Boshqasini xohlasang

```
definePageMeta({ layout: 'admin' })
```

Diagrammada layout doimiy ramka bo'lib qolishi va `<slot />` ichiga har bir sahifa kontenti tushishi tasvirlangan:



7.5 Auto-imports — "import yo'q" sehri

Nuxt avtomatik import qiladi: - **Vue API:** `ref`, `computed`, `watch`, `onMounted` — import yozma - `app/components/` — har komponent global ishlatishga tayyor - `app/composables/` — `useX()` avtomatik - `app/utils/` — sof funksiyalar avtomatik - **Nuxt composable'lari:** `useRoute`, `useRouter`, `useFetch`, `useState`, `navigateTo`, ...

```
<script setup>
// HECH QANDAY import yo'q — hammasi avtomatik
const count = ref(0)
```



```
const route = useRoute()
const { data } = await useFetch('/api/users')
</script>

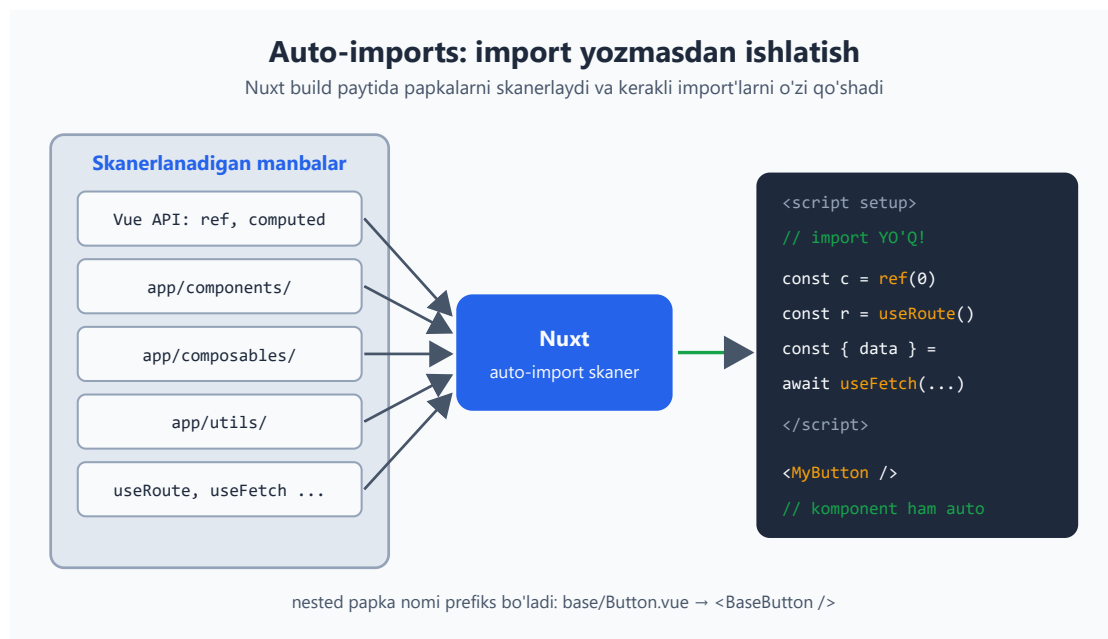
<template>
  <MyButton @click="count++">Bosildi: {{ count }}</MyButton>
  <!-- MyButton = app/components/MyButton.vue, import qilinmagan! -->
</template>
```

Komponent nomlash (nested)

```
app/components/
├─ AppHeader.vue          → <AppHeader />
├─ base/
│   └─ Button.vue         → <BaseButton />      (papka + fayl
nomi)
└─ user/
    └─ Card.vue           → <UserCard />
```

Nuxt papka nomini prefiks qiladi: `base/Button.vue` → `<BaseButton>`. Bu — komponentlarni guruhlash + nom to'qnashuvini oldini oladi.

Quyidagi diagramma Nuxt build paytida belgilangan papkalarni skanerlab, kerakli import'larni o'zi qanday qo'shishini ko'rsatadi:



Auto-import yoqdimi-yo'qmi? Ko'pchilik yoqtiradi (kam boilerplate). Aniq import xohlasang `nuxt.config` da o'chirsa bo'ladi, lekin tavsiya — convention'ga ergash.

7.6 definePageMeta va useHead (SEO)

```
<script setup>
definePageMeta({
  layout: 'admin',
  middleware: 'auth',          // app/middleware/auth.ts
  title: 'Boshqaruv paneli',
})

// SEO/meta teglar
useHead({
  title: 'EduCore – Boshqaruv',
  meta: [
    { name: 'description', content: 'O\'quv markazi boshqaruv tizimi' },
  ],
})

// Yoki qulayroq:
useSeoMeta({
  title: 'EduCore',
  description: 'O\'quv markazlari uchun SaaS',
  ogImage: '/og.png',
})
</script>
```

`useSeoMeta` / `useHead` — har sahifaga unikal SEO. **SSR** tufayli bu meta teglar server HTML'ida bo'ladi → Google/ijtimoiy tarmoq to'g'ri o'qiydi. (Oddiy Vue SPA'da bu muammo, Nuxt hal qiladi.)

7.7 nuxt.config.ts — markaziy konfiguratsiya

```
// nuxt.config.ts
export default defineNuxtConfig({
  devtools: { enabled: true },

  modules: [
    '@pinia/nuxt',
    '@nuxtjs/tailwindcss',
    '@nuxt/image',
  ],

  css: ['~/assets/css/main.css'],

  runtimeConfig: {
    apiSecret: '', // faqat server (maxfiy)
    public: {
      apiBase: 'https://api.educore.uz', // brauzerga ham ochiq
    }
  }
})
```

```

    },
  },
  app: {
    head: {
      title: 'EduCore',
      htmlAttrs: { lang: 'uz' },
    },
  },
})

```

~/ yoki @/ — app/ papkani bildiradi (Nuxt 4). runtimeConfig — .env qiymatlarni xavfsiz boshqarish (apiSecret brauzerga chiqmaydi, public chiqadi). Laravel'dagi config() + .env ajratimiga o'xshaydi.

7.8 Pinia'ni Nuxt'ga ulash

```
npm install pinia @pinia/nuxt
```

```

// nuxt.config.ts
export default defineNuxtConfig({
  modules: ['@pinia/nuxt'],
})

```

Endi app/stores/ dagi store'lar auto-import bo'ladi — createPinia() qo'lda kerak emas (06-moduldagi setup Nuxt qiladi). Store'larni xuddi 06-moduldagidek yozasan.

Xulosa

- Nuxt = "Vue uchun Laravel" — convention over configuration
- **Nuxt 4:** ilova kodi app/ da, backend server/ da, umumiy kod shared/ da
- **app/pages/** = file-based routing ([id].vue = dynamic, [...slug].vue = catch-all) — qo'lda router yo'q
- <NuxtPage> (= RouterView), <NuxtLink> (= aqlli RouterLink, prefetch), navigateTo()
- **Layouts** (app/layouts/) = Blade @extends; <slot/> ga sahifa tushadi
- **Auto-imports:** Vue API, komponent, composable, util — import yozmaysan
- definePageMeta (layout/middleware), useSeoMeta / useHead (SSR SEO)
- nuxt.config.ts + runtimeConfig (maxfiy/public ajratish)

Masalalar (kamida 20 ta)

Setup & routing (1–8)

1. `nuxt init` bilan loyiha yarat, ishga tushir, `app/app.vue` ni o'zgartirib brauzerda ko'r.
2. `app/pages/index.vue`, `about.vue`, `contact.vue` yarat; `<NuxtLink>` bilan nav qil. `router/index.js` yo'qligiga e'tibor ber.
3. Dynamic route (★): `app/pages/users/[id].vue`; `/users/5`, `/users/42` ga kirib `route.params.id` ni ko'rsat.
4. Catch-all (★): `app/pages/[...slug].vue` 404 sahifa yasa; mavjud bo'lmagan URL'ni ushla.
5. `<NuxtLink>` prefetch (★): Network tab'da, linkga hover qilganda sahifa oldindan yuklanishini kuzat.
6. `navigateTo` (★): tugma bosilganda dasturiy ravishda `/about` ga o't.
7. Nested route (★★): `app/pages/dashboard.vue` + `dashboard/profile.vue`, `dashboard/settings.vue`; yon menyu doim qolsin.
8. Blog (★★): `app/pages/blog/index.vue` (ro'yxat) + `blog/[slug].vue` (post); ro'yxatdan postga `<NuxtLink>` bilan o't.

Layouts (9–12)

1. `app/layouts/default.vue` (header+footer+slot) yarat; barcha sahifalar undan foydalansin.
2. `app/layouts/admin.vue` (★) yarat; `definePageMeta({ layout: 'admin' })` bilan faqat admin sahifalarga qo'lla.
3. **Layout'siz sahifa** (★): Login sahifasiga `layout: false` yoki bo'sh layout ber.
4. Dinamik layout (★★): foydalanuvchi rolega qarab layout almashtirish (`definePageMeta` + `computed/middleware`).

Auto-import & komponentlar (13–17)

1. `app/components/AppHeader.vue` yarat; **import qilmasdan** `app.vue` da ishlat.
2. Nested komponent (★): `app/components/base/Button.vue` → `<BaseButton>` sifatida ishlat.
3. Composable auto-import (★): `app/composables/useCounter.ts` yarat (04-moduldan); import yozmasdan sahifada ishlat.

4. Util auto-import (★): `app/utils/formatDate.ts` yarat; sahifada to'g'ridan-to'g'ri chaqir.
5. 04-moduldagi `useToggle` ni `app/composables/` ga ko'chir va modal komponentida ishlat.

Meta, config, integratsiya (18–24)

1. SEO (★): har sahifaga `useSeoMeta` bilan unikal title/description ber; View Source'da meta'lar **HTML'da** ekanini tasdiqla (SSR isboti).
2. `definePageMeta({ title })` (★): `app/layouts` yoki plugin orqali sahifa title'ini boshqar.
3. TailwindCSS (★): `@nuxtjs/tailwindcss` modulini ulab, sahifani Tailwind bilan stilla.
4. Pinia (★★): `@pinia/nuxt` ni ulab, 06-moduldagi `useCartStore` ni Nuxt'da ishlat (auto-import store).
5. `runtimeConfig` (★★): `public.apiBase` qo'sh; sahifada `useRuntimeConfig().public.apiBase` ni o'qib ko'rsat.
6. **05↔07 solishtiruv** (★★): 05-moduldagi qo'lda Vue Router setup'ini Nuxt file-based routing bilan taqqosla; xulosani izoh sifatida yoz (qaysi convention nimani avtomatlashtirdi).
7. **EduCore skeleti** (★★★): `app/pages/` da `index`, `login`, `app/` (dashboard, students, payments — nested), `admin/`; `default + admin + auth` layout'lar; auto-import header/sidebar komponentlari. (Middleware/auth — keyingi modulda to'liq.)

✓ Tanlangan yechimlar



7 — Nested dashboard



```
<!-- app/pages/dashboard.vue (ota) -->
<template>
  <div style="display:flex; gap:1rem">
    <aside>
      <NuxtLink to="/dashboard/profile">Profil</NuxtLink>
      <NuxtLink to="/dashboard/settings">Sozlamalar</NuxtLink>
    </aside>
    <main>
      <NuxtPage />    <!-- bola sahifalar shu yerda -->
    </main>
  </div>
</template>
```

```
<!-- app/pages/dashboard/profile.vue -->
<template><h2>Profil sahifasi</h2></template>
```



10 — Admin layout



```
<!-- app/layouts/admin.vue -->
<template>
  <div class="admin-wrap">
    <AdminSidebar />
    <section class="content"><slot /></section>
  </div>
</template>
```

```
<!-- app/pages/admin/index.vue -->
<script setup>
definePageMeta({ layout: 'admin' })
</script>
<template><h1>Admin Dashboard</h1></template>
```

➡ Keyingi: 08 — Data Fetching & Server (Nitro)

08 — Data Fetching & Server (Nitro)

← Oldingi: 07 — Nuxt asoslari · 🏠 README · Keyingi: 09 — Nuxt UI →

Bu modul Nuxt'ning eng kuchli va eng "fullstack" qismi. Backend odam uchun aynan shu yer qiziq: Nuxt faqat frontend emas — uning ichida **Nitro** degan to'liq server engine bor. Ya'ni bitta loyihada ham Vue app, ham API yozasan. Laravel termini bilan: bu deyarli **mini-Laravel frontend bilan birga keladi**.

Modul 2 qismdan iborat: 1. **Client/Universal data fetching** — `useFetch`, `useAsyncData`, `$fetch` 2. **Server qismi (Nitro)** — `server/api/`, `middleware`, `useState`

1. Muammo: nega oddiy `fetch` yetarli emas?

SSR'da kod **ikki marta** ishlaydi: bir marta serverda (birinchi HTML render), bir marta clientda (hydration va keyingi navigatsiya). Endi savol: ma'lumotni qayerda olamiz?

Agar React'dagidek `onMounted` ichida `fetch` qilsang:

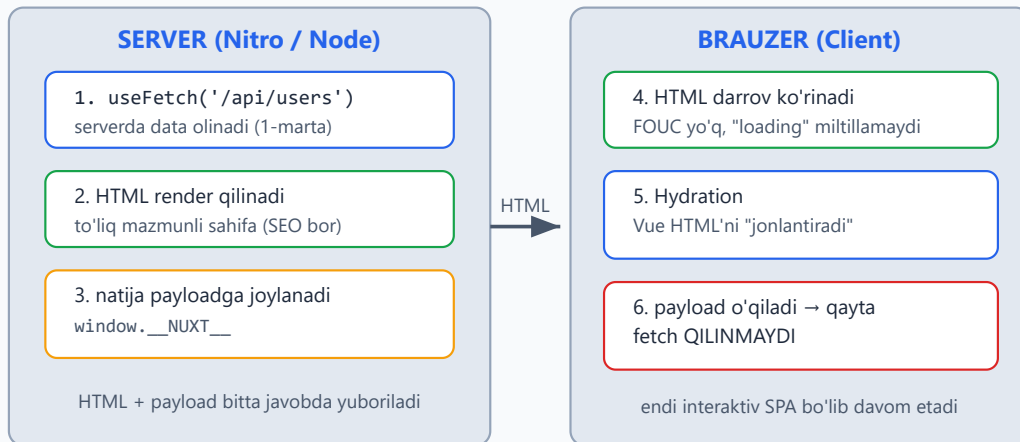
```
<script setup>
const users = ref([])
onMounted(async () => {
  users.value = await fetch('/api/users').then(r => r.json())
})
</script>
```

Muammolar: - `onMounted` **faqat clientda** ishlaydi → serverda HTML bo'sh keladi → SEO yo'q, "loading..." miltillaydi (FOUC). - Bot/Google sahifani ko'rganda ma'lumot yo'q. - Server qilgan ishni client yana takrorlaydi.

Nuxt buni `useFetch` / `useAsyncData` bilan hal qiladi: **kod serverda ishlaydi, natija HTML bilan birga clientga "payload" sifatida uzatiladi, client uni qayta yuklamaydi**. Bu — universal (isomorphic) data fetching.

Quyidagi diagramma bu oqimni bosqichma-bosqich ko'rsatadi: server data oladi va HTML render qiladi, brauzer uni darrov ko'rsatadi, keyin hydration paytida payloaddan o'qib qayta fetch qilmaydi.

SSR oqimi: useFetch serverda ishlaydi, payload bilan clientga uzatiladi



Universal (isomorphic):

data faqat BIR marta (serverda) olinadi, client payloaddan o'qiydi — ikki marta so'rov yo'q.

Laravel analogiyasi. Controller `User::all()` ni olib, `view()` ga uzatadi — server ma'lumot bilan tayyor HTML yuboradi. `useFetch` ham xuddi shu, lekin keyin SPA sifatida client-side davom etadi. Ikki dunyoning yaxshisi.

2. useFetch — asosiy ish quroli

Eng ko'p ishlatadigan composable. Setup'ning yuqori qismida (top-level) chaqiriladi.

```
<script setup>
const { data, pending, error, refresh } = await useFetch('/api/users')
</script>

<template>
  <p v-if="pending">Yuklanmoqda...</p>
  <p v-else-if="error">Xato: {{ error.message }}</p>
  <ul v-else>
    <li v-for="u in data" :key="u.id">{{ u.name }}</li>
  </ul>
</template>
```

Qaytaradi:

Maydon	Nima
<code>data</code>	natija (Ref)

Maydon	Nima
<code>pending</code>	<code>true</code> — yuklanmoqda (boolean Ref)
<code>error</code>	xato bo'lsa error obyekti, aks holda <code>null</code>
<code>refresh()</code>	qayta yuklash funksiyasi
<code>status</code>	<code>'idle' \ 'pending' \ 'success' \ 'error'</code>
<code>execute()</code>	<code>lazy / immediate:false</code> bo'lganda qo'lda ishga tushirish
<code>clear()</code>	data'ni tozalash

Muhim opsiyalar

```
const { data } = await useFetch('/api/products', {
  query: { page: 1, limit: 20 }, // ?page=1&limit=20
  method: 'GET',
  headers: { 'X-Custom': 'value' },
  lazy: false, // false = navigatsiyani bloklab kutadi; true =
  bloklamaydi
  server: true, // false = faqat clientda fetch qil (SSR'da
  o'tkazib yubor)
  immediate: true, // false = avtomatik chaqirmaydi, execute()
  kutadi
  default: () => [], // data hali yo'q paytda boshlang'ich qiymat
  watch: [page], // shu ref o'zgarsa avtomatik refetch
  key: 'products', // dedupe / cache kaliti
})
```

`transform` va `pick` — payload'ni kichraytirish

`useFetch` natijasi **HTML bilan clientga uzatiladi** (payload). Agar API katta obyekt qaytarsa, hammasi client'ga ketadi — sahifa og'irlashadi. Faqat kerakligini olib qol:

```
// pick — faqat shu maydonlar payloadga tushadi
const { data } = await useFetch('/api/user', {
  pick: ['id', 'name', 'avatar'],
})

// transform — server javobini o'zgartirib, kichraytirib saqlash
const { data } = await useFetch('/api/products', {
  transform: (res) => res.items.map(p => ({ id: p.id, title: p.title
```

```
))),  
})
```

Why. Bu performance uchun muhim: API'da 50 ta ustun bo'lsa-yu, sahifada 3 tasi kerak bo'lsa — qolgan 47 tasini client'ga uzatish behuda trafik. `pick / transform` payloadni qisqartiradi.

3. `useAsyncData` — nazorat kerak bo'lganda

`useFetch` aslida `useAsyncData` ustiga sugar:

```
// Bu ikkalasi deyarli bir xil  
useFetch('/api/users')  
useAsyncData('users', () => $fetch('/api/users'))
```

`useAsyncData(key, handler)` ni qachon ishlatasan: - Handler ichida **bir nechta** so'rov birlashtirish kerak bo'lsa. - HTTP bo'lmagan async logika (masalan, local DB, SDK chaqiruvi). - O'zing `$fetch` chaqiruvini to'liq nazorat qilmoqchi bo'lsang.

```
const { data } = await useAsyncData('dashboard', async () => {  
  const [stats, recent] = await Promise.all([  
    $fetch('/api/stats'),  
    $fetch('/api/recent'),  
  ])  
  return { stats, recent }  
})
```

Mental model. `useFetch` = "shu URL'ni ur". `useAsyncData` = "men o'zim yozaman, sen faqat SSR/cache/dedupe'ni boshqar". Birinchi argument **key** — u bo'yicha dedupe va cache ishlaydi.

4. `$fetch` — action'lar uchun

`$fetch` — haqiqiy HTTP client (`ofetch` kutubxonasi). JSON'ni avtomatik parse qiladi, base URL biladi.

Qoida (juda muhim):

Holat	Nima ishlatasan
Sahifa yuklanganda data olish (top-level)	<code>useFetch</code> / <code>useAsyncData</code>
Foydalanuvchi harakati: tugma bosish, forma yuborish (POST/PUT/DELETE)	<code>\$fetch</code>

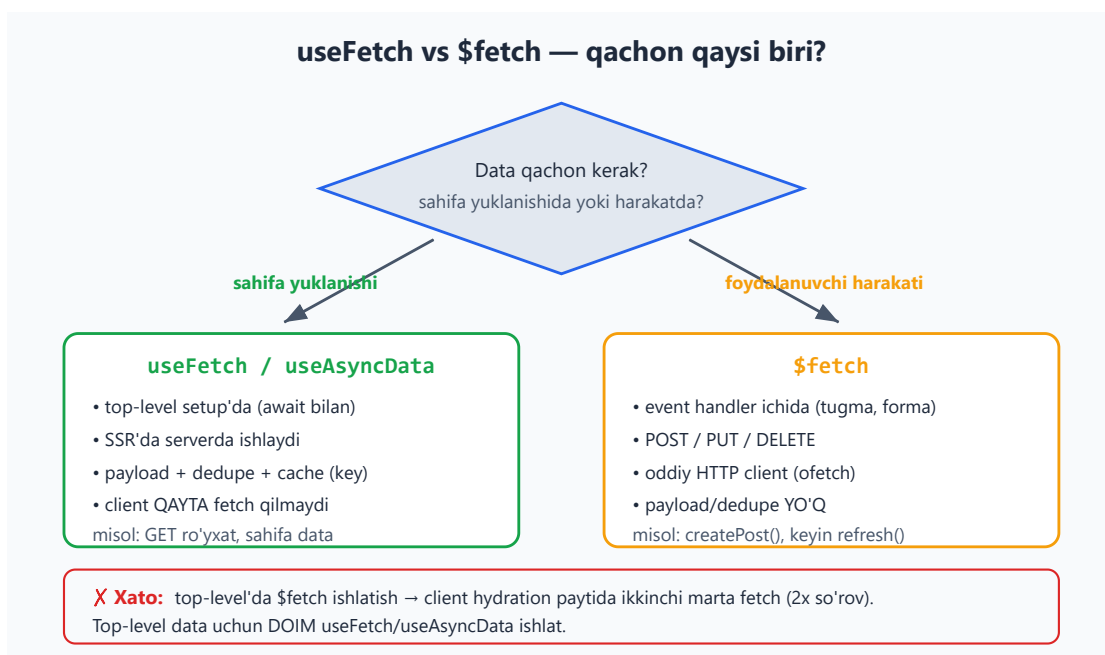
```

<script setup>
async function createPost() {
  const post = await $fetch('/api/posts', {
    method: 'POST',
    body: { title: title.value, content: content.value },
  })
  // ro'yxatni yangilash
  await refresh()
}
</script>

```

❌ **Xato:** setup'ning yuqorisida `const data = await $fetch('/api/users')` yozish. Bu SSR'da ishlaydi-yu, lekin **dedupe/payload** bo'lmaydi → client hydration paytida **yana** fetch qiladi (ikki marta!). Top-level data uchun **doim** `useFetch` / `useAsyncData`. `$fetch` — faqat event handler ichida.

Bu qarorni quyidagi diagramma bilan mustahkamlab oling — qaysi vaziyatda qaysi quroldan foydalanish kerakligini ko'rsatadi.



5. ⚠️ Nuxt 4 gotcha: `data` endi `shallowRef`

Nuxt 4'da `useFetch` / `useAsyncData` qaytaradigan `data` — `shallowRef` (ilgari oddiy `ref` edi). Ya'ni faqat `.value` ni **qayta tayinlash** reaktivlikni ishga soladi; ichidagi nested propertyni o'zgartirish **ishlamaydi**:

```
const { data } = await useFetch('/api/todos')

// ❌ Ishlamaydi (shallowRef nested o'zgarishni kuzatmaydi)
data.value.push(newTodo)

// ✅ To'g'ri — qayta tayinla
data.value = [...data.value, newTodo]
```

Why. Bu ataylab qilingan performance optimizatsiyasi — katta data obyektini deep-reactive qilish qimmat. Agar deep reaktivlik kerak bo'lsa: `useFetch(url, { deep: true })`.

6. Xatolarni boshqarish

```
<script setup>
const { data, error } = await useFetch('/api/user/123')

// Agar 404 kerak bo'lsa — error.vue sahifasiga o'tkazish
if (error.value) {
  throw createError({
    statusCode: 404,
    statusMessage: 'Foydalanuvchi topilmadi',
    fatal: true, // fatal:true → error.vue ko'rsatiladi
  })
}
</script>
```

`app/error.vue` — global xato sahifasi (Laravel'dagi `resources/views/errors/404.blade.php` analogi):

```
<!-- app/error.vue -->
<script setup>
defineProps({ error: Object })
const handleError = () => clearError({ redirect: '/' })
</script>
<template>
  <div>
    <h1>{{ error.statusCode }}</h1>
    <p>{{ error.statusMessage }}</p>
    <button @click="handleError">Bosh sahifaga</button>
```

```
</div>
</template>
```

7. Cookie va SSR autentifikatsiya

SSR'da bitta nozik masala bor: server `useFetch` qilganda **brauzer cookie'larini avtomatik uzatmaydi**. Ya'ni login bo'lgan userning tokeni serverdagi so'rovga ilashmaydi. Qo'lda uzatish kerak:

```
<script setup>
// SSR paytida kelgan cookie'ni keyingi API so'roviga uzatamiz
const headers = useRequestHeaders(['cookie'])
const { data } = await useFetch('/api/me', { headers })
</script>
```

`useCookie` — SSR-safe reaktiv cookie (ham serverda, ham clientda ishlaydi):

```
const token = useCookie('auth_token', {
  maxAge: 60 * 60 * 24 * 7, // 7 kun
  httpOnly: false,          // JS o'qishi kerak bo'lsa
  sameSite: 'lax',
})
token.value = 'eyJ...' // o'rnatish
```

Laravel analogiyasi. `useCookie` $\approx$ `cookie()` helper + reaktiv.

`useRequestHeaders(['cookie'])` $\approx$ kiruvchi `Request` headerlarini keyingi internal so'rovga forward qilish (gateway/proxy pattern).

2-QISM: Server (Nitro)

Mana eng qiziq joy. `server/` papkasi — bu **Nitro** engine ostida ishlaydigan to'liq backend. Bu yerda API endpoint, middleware, hatto WebSocket yozasan. Nitro standalone deploy bo'ladi: Node, serverless, edge (Cloudflare Workers, Vercel Edge) — hamma joyda.

```
server/
├─ api/           → /api/* endpointlar
├─ routes/        → /* (api prefiksisiz route'lar, masalan
/sitemap.xml)
└─ middleware/    → har bir so'rovda ishlaydigan server middleware
```

└─ plugins/ → Nitro plugin (startup hooks)
└─ utils/ → auto-import bo'ladigan server helperlar

Why bu backend odamga muhim. EduCore'da haqiqiy backend Laravel'da bo'lishi mumkin. Lekin Nuxt server qismi **BFF (Backend-for-Frontend)** sifatida ishlaydi: Laravel API'ni proxy qiladi, maxfiy kalitlarni yashiradi, auth cookie'larini boshqaradi, bir nechta so'rovni birlashtiradi. Frontend hech qachon to'g'ridan-to'g'ri tashqi API kalitini ko'rmaydi.

8. Server API routes — `server/api/`

File-based, xuddi sahifalar kabi:

```
// server/api/hello.ts → GET /api/hello
export default defineEventHandler((event) => {
  return { message: 'Salom EduCore' } // avtomatik JSON bo'ladi
})
```

`defineEventHandler` = controller method. `event` = Request / Response o'rami.

Event utility'lari (eng kerakli)

```
export default defineEventHandler(async (event) => {
  const query = getQuery(event) // ?page=1 → { page: '1' }
  const id = getRouterParam(event, 'id') // /api/users/5 → '5'
  const body = await readBody(event) // POST body (JSON)
  const auth = getHeader(event, 'authorization')
  const token = getCookie(event, 'auth_token')

  setResponseStatus(event, 201)
  setCookie(event, 'session', 'abc', { httpOnly: true })

  return { ok: true }
})
```

Laravel ↔ Nitro lug'ati

Nitro	Laravel ekvivalenti
<code>server/api/users.ts</code>	<code>routes/api.php</code> + Controller (bitta faylda)
<code>defineEventHandler(fn)</code>	Controller <code>__invoke()</code>

Nitro	Laravel ekvivalenti
<code>getQuery(event)</code>	<code>\$request->query()</code>
<code>readBody(event)</code>	<code>\$request->all() / \$request->json()</code>
<code>getRouterParam(event, 'id')</code>	<code>route {id} parametri</code>
<code>getHeader(event, 'x')</code>	<code>\$request->header('x')</code>
<code>setResponseStatus(event, 201)</code>	<code>response()->json(\$d, 201)</code>
<code>createError({ statusCode })</code>	<code>abort(404)</code>

Dynamic va method-based routing

```
// server/api/users/[id].ts → GET /api/users/:id
export default defineEventHandler((event) => {
  const id = getRouterParam(event, 'id')
  return { id }
})
```

Fayl nomiga method qo'shib, REST resource yasaysan:

```
server/api/posts/
├─ index.get.ts      → GET    /api/posts      (ro'yxat)
├─ index.post.ts     → POST   /api/posts      (yaratish)
├─ [id].get.ts       → GET    /api/posts/:id  (bitta)
├─ [id].put.ts       → PUT    /api/posts/:id  (yangilash)
└─ [id].delete.ts    → DELETE /api/posts/:id  (o'chirish)
```

Bu Laravel'ning `Route::apiResource('posts', ...)` ning fayl-tizimdagi ko'rinishi. Har bir verb — alohida fayl, alohida handler.

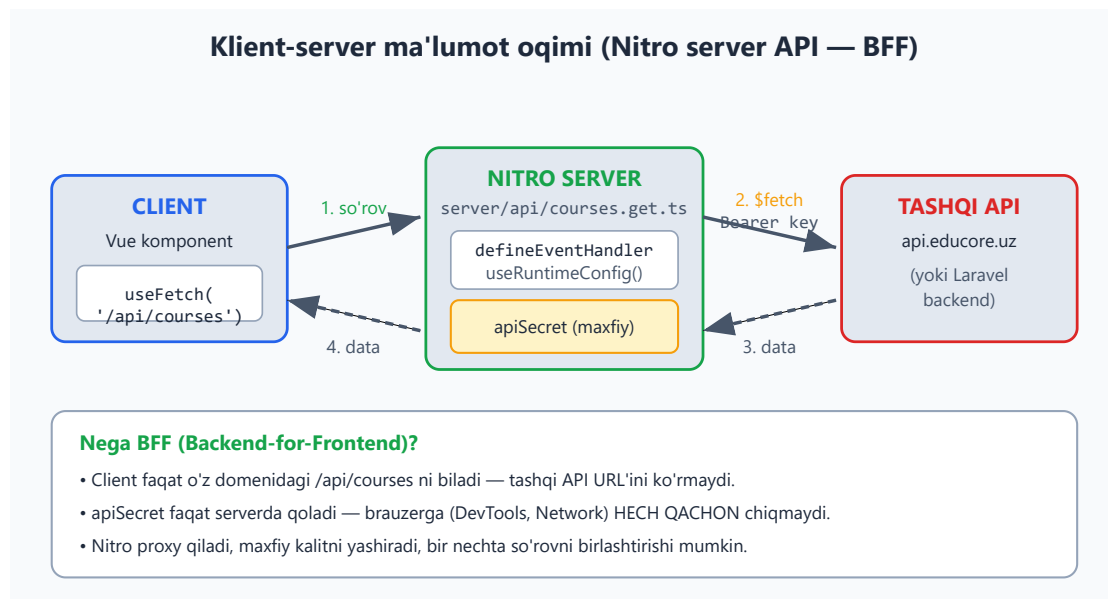
BFF misoli — maxfiy kalit bilan tashqi API'ni proxy qilish

```
// server/api/courses.get.ts
export default defineEventHandler(async (event) => {
  const config = useRuntimeConfig() // serverda private kalitlar mavjud
  const data = await $fetch('https://api.educore.uz/courses', {
    headers: { Authorization: `Bearer ${config.apiSecret}` },
  })
})
```

```
return data
})
```

Endi clientdagi `useFetch('/api/courses')` shu Nitro endpointga uradi, `apiSecret` esa **hech qachon brauzerga chiqmaydi**. Bu — to'g'ri arxitektura.

Quyidagi diagramma klient-server ma'lumot oqimini ko'rsatadi: client faqat o'z `/api/courses` endpointini biladi, Nitro esa maxfiy kalit bilan tashqi API'ni proxy qiladi.



9. Server middleware — `server/middleware/`

Har bir server so'rovida, route handlerdan **oldin** ishlaydi. Hech narsa `return` qilmaydi — faqat `event.context` ga ma'lumot biriktiradi yoki xato tashlaydi.

```
// server/middleware/auth.ts
export default defineEventHandler((event) => {
  const token = getCookie(event, 'auth_token')
  if (token) {
    event.context.user = verifyToken(token) // keyingi handlerlar o'qiy oladi
  }
})
```

Keyin istalgan API handlerda:

```
// server/api/me.get.ts
export default defineEventHandler((event) => {
  if (!event.context.user) {
    throw createError({ statusCode: 401, statusMessage: 'Avtorizatsiya'
  
```



```
yo\'q\' }})
}
return event.context.user
})
```

Laravel analogiyasi. Server middleware ≈ global middleware (app/Http/Kernel.php da ro'yxatdan o'tgan). event.context ≈ request'ga bog'langan ma'lumot (\$request->user() ni middleware o'rnatgani kabi). Har bir so'rovda ishlaydi.

10. Route middleware — app/middleware/ (server middleware EMAS!)

Diqqat: bu **boshqa narsa**. Route middleware — navigatsiya guard'i (05-moduldagi Vue Router guard'ining Nuxt versiyasi). Ham serverda, ham clientda, sahifaga **kirishdan oldin** ishlaydi.

```
// app/middleware/auth.ts
export default defineNuxtRouteMiddleware((to, from) => {
  const user = useAuthStore() // yoki useCookie/useState
  if (!user.isLoggedIn) {
    return navigateTo('/login')
  }
})
```

Sahifada ulash:

```
<script setup>
definePageMeta({ middleware: 'auth' })
</script>
```

Turlari: - **Named** — app/middleware/auth.ts, definePageMeta orqali ulanadi. - **Global** — app/middleware/analytics.global.ts, har bir navigatsiyada avtomatik. - **Inline** — definePageMeta({ middleware: [(to) => {...}] }).

Ikkalasini ADASHTIRMA — eng muhim jadval

	Route middleware (app/middleware/)	Server middleware (server/middleware/)
Qachon	Sahifaga navigatsiyadan oldin	Har bir HTTP so'rovda

	Route middleware (<code>app/middleware/</code>)	Server middleware (<code>server/middleware/</code>)
Qayerda	Client + server (navigatsiya)	Faqat server
Funksiya	<code>defineNuxtRouteMiddleware</code>	<code>defineEventHandler</code>
Vazifa	Redirect, access control (UI)	Auth context, logging, CORS, headers
Laravel analogi	<code>route</code> 'ga ulangan middleware (<code>->middleware('auth')</code>)	global HTTP middleware (Kernel)

Why ikkitaligi. Sahifa access controli (login bo'lmasa `/login` ga ot) — bu navigatsiya darajasi → route middleware. API so'rovni himoyalash, har request'da token tekshirish — bu server darajasi → server middleware. Ko'pincha ikkalasi birga kerak.

11. `useState` — SSR-safe shared state

Bu yer backend odam uchun **xavfsizlik** nuqtai nazaridan juda muhim.

❌ **Hech qachon shunday qilma:**

```
// composables/counter.js - XATARLI
const count = ref(0) // module-scope ref
export const useCounter = () => count
```

Nega xatarli? Serverda **bitta process** minglab foydalanuvchiga xizmat qiladi. Module-scope `ref` esa **hamma so'rovlar uchun bitta** — ya'ni A foydalanuvchining ma'lumoti B foydalanuvchiga **leak** bo'ladi. Bu jiddiy xavfsizlik bug'i (cross-request state pollution).

✅ **To'g'ri yo'l — `useState`:**

```
// SSR-safe, har bir so'rov uchun alohida, clientga serialize bo'ladi
const counter = useState('counter', () => 0)
const user = useState('user', () => null)
```

```
<script setup>
const count = useState('count', () => 0)
</script>
<template>
```

```
<button @click="count++">{{ count }}</button>
</template>
```

Laravel analogiyasi. Module-scope `ref = static` propertyda request ma'lumotini saqlash (har request'da bir xil obyekt → leak). `useState` = request lifecycle'ga bog'langan, har so'rovda yangi (Laravel'da har request yangi container/instance bo'lgani kabi). Singleton vs request-scoped farqi.

useState vs Pinia

useState	Pinia (06-modul)
Oddiy shared SSR state	Murakkab app state
Faqat qiymat	State + getters + actions + plugins
Tezkor, kichik holatlar	Auth, cart, tenant kabi to'liq domenlar

Kichik narsaga `useState`, katta domenga Pinia. EduCore'da: theme toggle → `useState`; auth/tenant/cart → Pinia store.

12. Nitro qo'shimchalari (qisqacha, hero darajasi uchun)

routeRules — `nuxt.config.ts` da deklarativ rendering/cache/proxy qoidalari:

```
export default defineNuxtConfig({
  routeRules: {
    '/': { prerender: true }, // build'da static
    '/admin/**': { ssr: false }, // SPA (faqat client)
    '/blog/**': { isr: 3600 }, // ISR — har soatda
    regenerate
    '/api/legacy/**': { proxy: 'https://old.educore.uz/**' }, // proxy
  },
})
```

Cached event handler — server javobini keshlash:

```
// server/api/stats.get.ts
export default defineCachedEventHandler(async () => {
  return await computeHeavyStats()
}, { maxAge: 60 }) // 60 soniya kesh
```

useStorage — Nitro'ning built-in KV/storage qatlami (Redis, FS, memory):

```
const storage = useStorage('redis')
await storage.setItem('key', value)
const v = await storage.getItem('key')
```

Bularning hammasi alohida chuqur mavzu (rendering modes, Nitro deep) — keyingi to'plamda. Hozircha mavjudligini bil.

13. EduCore arxitekturası — hammasi birga

Multi-tenant SaaS uchun tipik Nuxt server oqimi:

```
// server/middleware/tenant.ts — subdomain'dan tenant aniqlash
export default defineEventHandler((event) => {
  const host = getHeader(event, 'host') || ''
  const subdomain = host.split('.')[0] // markaz.educore.uz →
  "markaz"
  event.context.tenant = subdomain
})
```

```
// server/api/students.get.ts — tenant bo'yicha scoping
export default defineEventHandler(async (event) => {
  const tenant = event.context.tenant
  const config = useRuntimeConfig()
  return await $fetch(`${config.apiBase}/students`, {
    headers: {
      'X-Tenant': tenant,
      Authorization: `Bearer ${config.apiSecret}`,
    },
  })
})
```

```
// app/middleware/auth.global.ts — UI darajasida himoya
export default defineNuxtRouteMiddleware((to) => {
  const auth = useAuthStore()
  const publicPages = ['/login', '/register']
  if (!auth.isLoggedIn && !publicPages.includes(to.path)) {
    return navigateTo('/login')
  }
})
```

To'liq oqim:

```
Brauzer → markaz.educore.uz
↓
```

```
server/middleware/tenant.ts (tenant'ni context'ga qo'yadi)
↓
server/middleware/auth.ts (token tekshiradi)
↓
useFetch('/api/students')
↓
server/api/students.get.ts (Laravel API'ni tenant header bilan proxy)
↓
Laravel backend (tenant_id scoping, DDD)
```

Frontend hech qachon `apiSecret` 'ni ko'rmaydi, tenant aniqlash serverda — bu xavfsiz va to'g'ri arxitektura.

Xulosa — mental model

```
DATA OLIISH:
  Sahifa yuklanishi (top-level) → useFetch / useAsyncData
  Foydalanuvchi harakati (POST) → $fetch

SERVER:
  server/api/          → endpointlar (defineEventHandler)
  server/middleware/   → har so'rovda (auth context, logging)
  app/middleware/      → navigatsiya guard (defineNuxtRouteMiddleware)

STATE:
  Oddiy SSR state → useState (module-scope ref ASLO!)
  Murakkab domen → Pinia
```

Eng ko'p qilinadigan 3 xato: 1. Top-level'da `$fetch` ishlatib, ikki marta fetch qilish (`useFetch` kerak). 2. Server'da module-scope `ref` — cross-request leak (`useState` kerak). 3. Route middleware bilan server middleware'ni adashtirish.

Masalalar (24 ta)

`npx nuxi init data-lab` bilan loyiha och. Server qismi uchun `server/api/` ichida soxta (mock) data qaytaruvchi endpointlar yozib, frontend bilan ulab mashq qil.

A daraja — data fetching

1. ★ `/users` sahifa yarat, `useFetch('https://jsonplaceholder.typicode.com/users')` bilan ro'yxat chiqar. `pending` va `error` holatlarini ham ko'rsat.

2. ★ Yuqoridagi so'rovga `pick: ['id', 'name', 'email']` qo'sh. DevTools → payload hajmi qanday o'zgardi, kuzat.
3. ★ `transform` bilan javobni faqat `{ id, name }` massiviga aylantir.
4. ★★ `/posts` sahifa: `?page` query bilan paginatsiya. `page` ref'ini `watch` opsiyasiga ber — sahifa o'zgarsa avtomatik refetch bo'lsin.
5. ★★ "Yangilash" tugmasi qo'y, `refresh()` ni chaqir. Tugma bosilganda `pending` ko'rsatilsinmi? Tekshir.
6. ★★ `lazy: true` va `lazy: false` farqini his qil: ikki sahifa yasab, navigatsiyada qaysi biri "kutadi", qaysi biri darrov ochiladi — kuzat.
7. ? Nima uchun setup top-level'da `const d = await $fetch('/api/x')` yomon? O'z so'zing bilan yoz. ✓
8. ★★ `useAsyncData('dash', ...)` bilan ikki endpointni `Promise.all` orqali bitta composable'da birlashtir.

B daraja — actions va xatolar

1. ★★ Forma yarat (title, body), "Saqlash" tugmasi `$fetch('/api/posts', { method: 'POST', body })` qilsin. Javobni konsolga chiqar. ✓
2. ★★ POST muvaffaqiyatli bo'lgach, ro'yxatni `refresh()` bilan yangila.
3. ★★ `[id].vue` sahifada `useFetch('/api/posts/' + id)`. ID mavjud bo'lmasa `createError({ statusCode: 404 })` tashla.
4. ★★ `app/error.vue` yarat, 404 va 500 ni chiroyli ko'rsat, "Bosh sahifaga" tugmasi `clearError` chaqirsin.
5. ? Nuxt 4'da `data.value.push(x)` nega UI'ni yangilamaydi? Qanday to'g'rilaysan? ✓
6. ★★★ Optimistic update: tugma bosilganda avval UI'ni yangila (`data.value = [...]`), keyin `$fetch` POST qil. Xato bo'lsa orqaga qaytar (rollback). (06-moduldagi Pinia versiyasi bilan solishtir.)

C daraja — server (Nitro)

1. ★ `server/api/ping.ts` yarat, `{ pong: true, time: Date.now() }` qaytarsin. Brauzerda `/api/ping` ni och. ✓
2. ★★ `server/api/users/[id].get.ts` — `getRouterParam` bilan ID'ni olib `{ id, name: 'User ' + id }` qaytar.
3. ★★ `server/api/echo.post.ts` — `readBody` bilan kelgan JSON'ni o'qib, qaytarib yubor (echo). `$fetch` bilan POST qilib sina.

4. ★★ Method-based resource: `server/api/todos/` ichida `index.get.ts`, `index.post.ts`, `[id].delete.ts` yoz. In-memory massivda saqla (server qayta ishga tushguncha yashaydi).
5. ★★★ BFF: `server/api/weather.get.ts` — `useRuntimeConfig` dan API kalit olib, tashqi ob-havo API'sini proxy qil. Kalit clientga chiqmasligiga ishonch hosil qil (DevTools → Network).

D daraja — middleware va state

1. ★★ `server/middleware/logger.ts` — har so'rovda `console.log(event.method, event.path)` chiqarsin. Bir nechta sahifa ochib, terminalda log'ni kuzat. ✓
2. ★★ `server/middleware/auth.ts` — cookie'dan token o'qib `event.context.user` ga qo'ysin. `server/api/me.get.ts` da contextdan o'qib qaytar, yo'q bo'lsa 401.
3. ★★ `app/middleware/guest.ts` — agar `useAuthStore().isLoggedIn` bo'lsa `/dashboard` ga yo'naltir (login sahifasiga ulang).
4. ? `useState('x', () => 0)` va `app/middleware/` dagi route middleware — bittasi qaysi muammoni hal qiladi, ikkinchisi qaysisini? Farqini yoz.
5. ★★★ **EduCore mini-BFF.** `server/middleware/tenant.ts` (host'dan subdomain → `event.context.tenant`) + `server/api/students.get.ts` (tenant'ni header qilib mock API qaytaradi) + `app/middleware/auth.global.ts` (login bo'lmasa `/login`). Uchallasini ulab, oqimni ishlatib ko'r. ✓

✓ Tanlangan yechimlar

7 — Nega top-level `$fetch` yomon

Top-level `$fetch` SSR'da bir marta serverda ishlaydi va HTML hosil qiladi. Lekin uning natijasi **payload sifatida saqlanmaydi** va **key bilan dedupe qilinmaydi**. Shu sababli client hydration paytida Vue yana o'sha kodni ishga tushiradi → **ikkinchi marta** fetch bo'ladi. Natija: ikki barobar so'rov, ortiqcha yuk, ba'zan hydration mismatch.

`useFetch / useAsyncData` esa serverdagi natijani `__NUXT__` payloadga joylaydi; client uni o'qiydi, qayta fetch qilmaydi. Shuning uchun: **top-level data** → **har doim** `useFetch / useAsyncData ; $fetch` → **faqat event handler (action) ichida**.



9 — POST forma



```
<script setup>
const title = ref('')
const body = ref('')
const result = ref(null)
const saving = ref(false)

async function save() {
  saving.value = true
  try {
    result.value = await
    $fetch('https://jsonplaceholder.typicode.com/posts', {
      method: 'POST',
      body: { title: title.value, body: body.value, userId: 1 },
    })
  } catch (e) {
    console.error('Xato:', e)
  } finally {
    saving.value = false
  }
}
</script>

<template>
  <input v-model="title" placeholder="Sarlavha" />
  <textarea v-model="body" placeholder="Matn" />
  <button :disabled="saving" @click="save">
    {{ saving ? 'Saqlanmoqda...' : 'Saqlash' }}
  </button>
  <pre v-if="result">{{ result }}</pre>
</template>
```

Diqqat: `$fetch` event handler (`save`) ichida — to'g'ri joy. `body` to'g'ridan-to'g'ri obyekt (o'fatch o'zi JSON qiladi).



13 — shallowRef gotcha



Nuxt 4'da `data` — `shallowRef`. `shallowRef` faqat `.value` butunlay almashtirilsa reaktivlikni ishga soladi; ichidagi massiv/obyektni mutare qilish (`push` , nested o'zgartirish) kuzatilmaydi.

```
const { data } = await useFetch('/api/todos')

// ❌ UI yangilanmaydi
data.value.push(newTodo)

// ✅ Yangi reference ber
data.value = [...data.value, newTodo]
```

Yoki deep reaktivlik kerak bo'lsa: `useFetch('/api/todos', { deep: true })`. Lekin katta data'da `deep` qimmat — odatda reassign yondashuvi yaxshiroq.



15 — birinchi server endpoint



```
// server/api/ping.ts
export default defineEventHandler(() => {
  return { pong: true, time: Date.now() }
})
```



`/api/ping` ga kirsang JSON ko'rasan. `return` qilingan obyekt avtomatik `Content-Type: application/json` bilan beriladi — qo'lda `JSON.stringify` shart emas. Bu Laravel'da controller'dan array qaytarsang avtomatik JSON bo'lganiga o'xshaydi.



20 — server logger middleware



```
// server/middleware/logger.ts
export default defineEventHandler((event) => {
  console.log(`[${new Date().toISOString()}] ${event.method} ${event.path}`)
  // hech narsa return qilinmaydi — middleware faqat "o'tib ketadi"
})
```



Muhim: server middleware **hech narsa qaytarmaydi** (qaytarsa, so'rovni shu yerda tugatib qo'yasan). Faqat yon ta'sir (log) yoki `event.context` ga yozish uchun. Har bir so'rovda — shu jumladan `/api/*` va sahifalar uchun ham — ishlaydi. Bu Laravel global middleware'ning aynan o'zi.



```
// server/middleware/tenant.ts
export default defineEventHandler((event) => {
  const host = getHeader(event, 'host') || 'demo.localhost'
  event.context.tenant = host.split('.')[0] // markaz.educore.uz → "markaz"
})
```

```
// server/api/students.get.ts
export default defineEventHandler((event) => {
  const tenant = event.context.tenant
  // haqiqiy loyihada bu yerda Laravel API'ga $fetch qilinadi (tenant header bilan)
  return {
    tenant,
    students: [
      { id: 1, name: 'Ali', tenant },
      { id: 2, name: 'Vali', tenant },
    ],
  }
})
```

```
// app/middleware/auth.global.ts
export default defineNuxtRouteMiddleware((to) => {
  const isLoggedIn = useCookie('auth_token').value
  const publicPages = ['/login', '/register']
  if (!isLoggedIn && !publicPages.includes(to.path)) {
    return navigateTo('/login')
  }
})
```

```
<!-- app/pages/students.vue -->
<script setup>
const { data } = await useFetch('/api/students')
</script>
<template>
  <h1>{{ data.tenant }} markazi o'quvchilari</h1>
  <ul><li v-for="s in data.students" :key="s.id">{{ s.name }}</li></ul>
</template>
```

Oqim: navigatsiya → auth.global.ts (login tekshiradi) → sahifa
useFetch('/api/students') → server'da tenant.ts middleware tenant'ni aniqlaydi →
students.get.ts tenant bo'yicha data qaytaradi. Bu real multi-tenant BFF'ning
soddalashtirilgan skeletini ko'rsatadi: **tenant aniqlash va maxfiy kalitlar serverda, frontend toza qoladi.**



Birinchi to'plam yakuni

Tabriklayman — Vue 3 yadrosini (01–06) va Nuxt'ning eng muhim qismini (07–08) tugatding. Endi sen: - Vue'da reaktiv, komponentli, router'li, state-managed SPA yoza olasan. - Nuxt'da SSR sahifalar, server API, middleware va to'g'ri data fetching arxitekturasini qura olasan. - EduCore kabi multi-tenant SaaS'ning frontend + BFF skeletini tushunasan.

Keyingi to'plam (so'rasang yozaman, shu uslubda): rendering modes (SSR/SSG/ISR/hybrid chuqur) · SEO/meta to'liq · Nitro deep (storage, caching, tasks) · modules ecosystem · **testing** (Vitest + Vue Test Utils) · **deployment** (VPS/Nginx, Docker, edge) · TypeScript chuqur · performance optimizatsiya.

 [README](#) ·  [Oldingi: 07 — Nuxt asoslari](#) · [Keyingi: 09 — Nuxt UI](#) 

09 — Nuxt UI: asoslar, o'rnatish va theming

← Oldingi: 08 — Data & Server · 🏠 README

Bu va keyingi (10-) modul **Nuxt UI** ga bag'ishlangan — Nuxt jamoasining rasmiy komponentlar kutubxonasi. Ikki qismga bo'ldim: - **09 (shu modul)** — Nuxt UI nima, o'rnatish, `UApp`, **design system** (rang/variant/size), **theming** (3 qatlamli tizim), `:ui` prop. Ya'ni "tizimni tushunish". - **10** — har bir komponentni kategoriya bo'yicha batafsil ko'rib chiqamiz (props, slot, misol).

Avval tizimni tushun, keyin komponentlarga o't. Aks holda har bir komponentda bir xil rang/variant tushunchalarini qayta-qayta o'qiysan.

1. Nuxt UI nima va nega v4 muhim

Nuxt UI — Vue/Nuxt uchun **tayyor, accessible, Tailwind asosidagi** komponentlar kutubxonasi. Button, Input, Modal, Table, butun Dashboard sidebar — hammasi tayyor.

Laravel analogiyasi. Nuxt UI ≈ **Filament**ning UI qatlami, lekin Vue/Nuxt uchun. Filament Laravel'da admin panel komponentlarini tayyor beradi; Nuxt UI ham xuddi shunday tayyor komponentlar beradi — lekin headless va to'liq sozlanuvchi. EduCore admin panelini noldan div/css yozmasdan yig'ish uchun aynan shu kerak.

v4 nima o'zgartirdi (2025 sentyabr): - **Nuxt UI va Nuxt UI Pro birlashdi** — endi hammasi bitta **bepul, MIT** `@nuxt/ui` paketda. Ilgari pullik bo'lgan Dashboard, Page, Auth, Pricing komponentlari endi bepul. (NuxtLabs Vercel'ga qo'shilgani sabab.) - **125+ komponent**, 12 ta bepul template, Figma kit. - Asosi: **Reka UI** (accessibility/primitivlar) + **Tailwind CSS v4 + Tailwind Variants** (class tizimi). - Faqat Nuxt emas — **toza Vue** (Vite) loyihalarda ham ishlaydi.

Hozirgi versiya: **v4.8+** (tez yangilanadi).

2. O'rnatish (Nuxt)

```
pnpm add @nuxt/ui tailwindcss
```



```
// nuxt.config.ts
export default defineNuxtConfig({
  modules: ['@nuxt/ui'],
  css: ['~/assets/css/main.css'],
})
```

```
/* app/assets/css/main.css */
@import "tailwindcss";
@import "@nuxt/ui";
```

Tamom. Nuxt UI o'zi avtomatik o'rnatadi: `@nuxt/icon` (ikonkalar), `@nuxt/fonts` (shriftlar), `@nuxtjs/color-mode` (dark mode). Ularni alohida `modules` ga qo'shma — konflikt bo'ladi.

Toza Vue (Nuxt'siz)

```
// vite.config.ts
import vue from '@vitejs/plugin-vue'
import ui from '@nuxt/ui/vite'

export default defineConfig({
  plugins: [vue(), ui()],
})
```

3. ⚠️ `UApp` — eng muhim wrapper (buni unutsang, yarmi ishlamaydi)

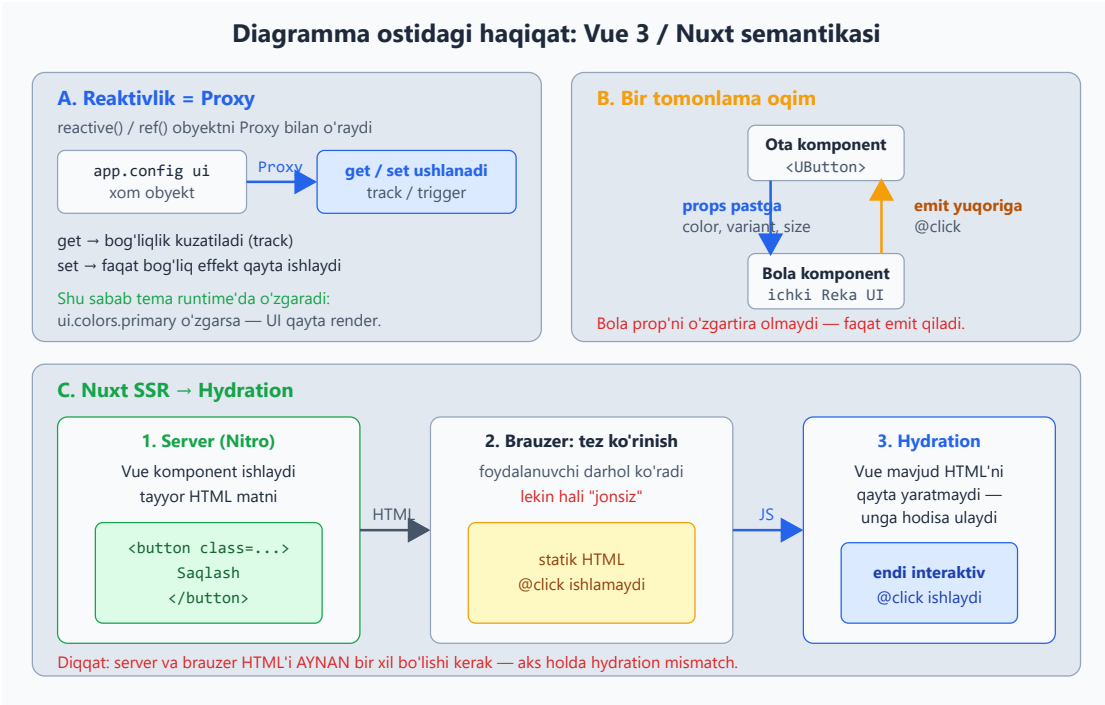
Butun ilovani `<UApp>` ichiga o'rab qo'yish **shart**. U global provayderlarni o'rnatadi: `Toast` (`useToast`), `Overlay` (`useOverlay`), `Tooltip`, dark mode konteksti.

```
<!-- app/app.vue -->
<template>
  <UApp>
    <NuxtLayout>
      <NuxtPage />
    </NuxtLayout>
  </UApp>
</template>
```

Why. `useToast()` yoki `useOverlay()` ishlamasa — 90% holatda sabab shu: `UApp` yo'q. U `Toaster` va `OverlayProvider`ni daraxtning tepasiga joylaydi (Vue'ning `provide/inject` mexanizmi — 03-modul). Laravel analogiyasi: bu app-level "service provider"ni `register` qilishga o'xshaydi — global servislar shu yerda ulanadi.

UApp qabul qiladigan foydali proplar: `:toaster` (toast joylashuvi), `:tooltip` (global tooltip sozlamalari), `locale` (i18n).

Bu `provide/inject`, reaktivlik va SSR Nuxt ostida aniq qanday ishlashini bir rasmda jamlab ko'rsataman — keyingi bo'limlardagi "runtime-reaktiv tema" va `@click` xulqi shu mexanizmlardan kelib chiqadi.



4. Design System — rang, variant, size

Bu — Nuxt UI'ning yuragi. Deyarli **har bir komponent** bir xil 3 ta propni qabul qiladi: `color`, `variant`, `size`. Buni bir marta tushunsang — 125 ta komponentni bilasan.

4.1. Semantik ranglar (`color`)

Nuxt UI to'g'ridan-to'g'ri `red` / `blue` ishlatmaydi. U **semantik** (ma'noga bog'langan) ranglarni ishlatadi:

Semantik rang	Vazifasi
<code>primary</code>	asosiy brend rangi (default: green)
<code>secondary</code>	ikkilamchi aksent

Semantik rang	Vazifasi
success	muvaffaqiyat (yashil)
info	ma'lumot (ko'k)
warning	ogohlantirish (sariq)
error	xato (qizil)
neutral	kulrang — fon, chegara, matn

```
<UIButton color="primary">Saqlash</UIButton>  
<UIButton color="error" variant="soft">O'chirish</UIButton>  
<UIAlert color="warning" title="Diqqat" />
```

Class sifatida ham ishlatasan: `bg-primary`, `text-primary`, `text-primary-500`, `ring-primary`. `primary` ning DEFAULT soyasi temaga moslashadi — light'da 500, dark'da 400.

Why semantik. `color="error"` yozsang, keyin brendni o'zgartirsang (qizil → to'q sariq) — faqat bitta config qatorini o'zgartirasan, butun ilova moslashadi. `bg-red-500` yozsang — har joyda qo'lda almashtirasan. Bu **design token** falsafasi.

4.2. Variant (`variant`)

Bir rang — bir necha ko'rinish. Button uchun:

Variant	Ko'rinish
solid	to'liq to'ldirilgan (default)
outline	faqat chegara
soft	yengil fon (rangning och varianti)
subtle	soft + nozik chegara
ghost	fonsiz, hover'da paydo bo'ladi
link	havola ko'rinishi

```
<UButton color="primary" variant="solid">Asosiy</UButton>
<UButton color="primary" variant="outline">Ikkilamchi</UButton>
<UButton color="neutral" variant="ghost">Bekor qilish</UButton>
```

Form-input komponentlarida variant boshqacha bo'ladi (outline, soft, subtle, ghost, none) — lekin g'oya bir xil.

4.3. Size (size)

xs · sm · md (default) · lg · xl — barcha komponentlarda bir xil shkala. Bitta size butun komponentning padding, font, ikonka o'lchamini muvofiq o'zgartiradi.

```
<UButton size="xs">Kichik</UButton>
<UInput size="lg" placeholder="Katta input" />
```

Quyidagi rasm uchala propni bitta o'qda jamlaydi — bu uch o'q mustaqil, ularni xohlagancha kombinatsiya qilasan.

Bitta komponent — 3 mustaqil o'q
`<UButton color="primary" variant="solid" size="md" />`

color — semantik rang (ma'no)

primary secondary success info warning error neutral

variant — ko'rinish (bir rang, ko'p ko'rinish)

solid outline soft subtle ghost link

size — o'lcham (padding + font + ikonka birga)

xs sm md (default) lg xl

5. Theming — 3 qatlamli tizim

Nuxt UI theming aynan 3 qatlamdan iborat. Har birining aniq vazifasi bor — adashtirma.

Qatlam 1: main.css	→ xom dizayn tokenlar (@theme) — "brend DNA"
Qatlam 2: app.config.ts	→ semantik mapping (qaysi rang primary)
Qatlam 3: app.config.ts	→ komponent override (default stillarni o'zgartirish)

Qatlam 1 — @theme (Tailwind v4 dizayn tokenlari)

Tailwind v4 "CSS-first" — sozlash JS config'da emas, CSS'da @theme ichida:

```
/* app/assets/css/main.css */
@import "tailwindcss";
@import "@nuxt/ui";

@theme {
  /* O'z brend rangingni 50-950 to'liq shkala bilan ber */
  --color-brand-50: #fff6ff;
  --color-brand-500: #2563eb;
  --color-brand-950: #172554;

  /* Shrift */
  --font-sans: 'Inter', sans-serif;
}
```

Muhim: custom rang qo'shsang, **50 dan 950 gacha** barcha soyalarni berishing kerak — komponentlar ularning hammasini ishlatadi. Hex'dan to'liq shkala uchun `uicolors.app` yoki Tailwind palitrasidan foydalan.

Qatlam 2 — semantik mapping (app.config.ts)

Xom tokenlarni semantik rolga ulaysan. Bu **runtime-reaktiv** — qayta build qilmasdan, hatto ish vaqtida temani almashtirib bo'ladi (HMR bilan):

```
// app/app.config.ts
export default defineAppConfig({
  ui: {
    colors: {
      primary: 'brand', // 1-qatlamdagi custom rang
      secondary: 'purple',
      success: 'green',
      info: 'blue',
      warning: 'amber',
      error: 'red',
      neutral: 'zinc', // kulranglar
    },
  },
})
```

Laravel analogiyasi. `app.config.ts` ≈ `config/` papkasidagi config fayllar — markazlashgan sozlamalar. Farqi: bu runtime'da reaktiv (Laravel config build/cache'lanadi).

Qatlam 3 — komponent override (app.config.ts)

Har bir komponentning **default** ko'rinishini global o'zgartirish. Masalan, "barcha button'lar default `neutral` + `subtle` bo'lsin":

```
// app/app.config.ts
export default defineAppConfig({
  ui: {
    button: {
      slots: {
        base: 'font-bold rounded-lg', // har bir button'ga
      },
      defaultVariants: {
        color: 'neutral',
        variant: 'subtle',
        size: 'lg',
      },
    },
  },
  card: {
    slots: {
      root: 'rounded-xl',
      body: 'p-6',
    },
  },
},
})
```

Compound variants — ma'lum rang+size kombinatsiyasiga maxsus stil:

```
button: {
  compoundVariants: [
    { color: 'primary', size: 'sm', class: 'ring-2 ring-primary/60' },
  ],
}
```

6. `:ui` prop va `class` — bitta instansiyani sozlash

Global config butun ilovaga ta'sir qiladi. Bitta joyda o'zgartirish kerak bo'lsa — `:ui` prop (slot'larni nishonga oladi) yoki `class` (root slot):

```
<UCard :ui="{ header: 'p-8 text-xl', body: 'p-8 space-y-4', footer:
'border-t' }">
  <template #header>Sarlavha</template>
  Asosiy kontent
  <template #footer>Pastki qism</template>
</UCard>

<!-- class — root/base slot uchun -->
```

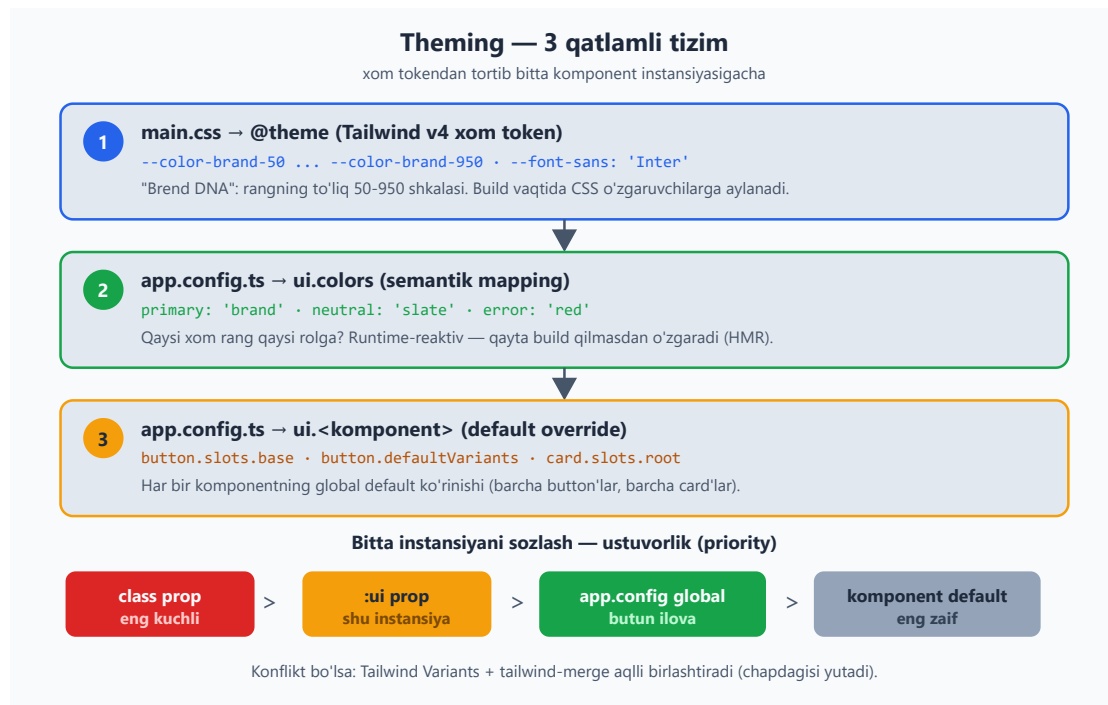
```
<UButton class="w-full" trailing-icon="i-lucide-arrow-right">Davom etish</UButton>
```

Ustuvorlik (priority):

```
class prop > :ui prop > app.config.ts global > komponent default
```

Why bu ishlaydi. Nuxt UI ostida **Tailwind Variants** + `tailwind-merge` bor. Ya'ni sening class'ing core class bilan **aqlli birlashadi** (konflikt bo'lsa seniki yutadi), CSS cascade bilan urishmaysan. `p-8` bersang — komponentning `p-4` i o'rniga qo'yiladi, ikkalasi qo'shilib ketmaydi.

Quyidagi rasm uchala theming qatlamini va bitta instansiyaga tushadigan to'liq ustuvorlik zanjirini bir joyda ko'rsatadi.



Laravel analogiyasi. `:ui` bilan slot override $\approx$ `vendor:publish` qilingan paket view'ini o'zgartirish — asl paketni buzmasdan ustiga o'z stilingni qo'yan.

Komponentni qayerdan o'rganish: slot'larni bilish

Har bir komponentning qanday slot'lari (`base`, `header`, `body`, `trailingIcon` ...) borligini hujjatdagi **Theme** bo'limidan ko'rasan — `:ui` da aynan o'sha kalitlarni ishlatasan. 10-modulda muhim komponentlarning asosiy slot'larini ko'rsataman.

7. Dark mode

`@nuxtjs/color-mode` avtomatik ulangan. Class-strategiya (`dark`: Tailwind). Komponentlar o'zi moslashadi. Tugma:

```
<UColorModeButton />    <!-- tayyor light/dark toggle -->
```

Yoki qo'lda:

```
<script setup>
const colorMode = useColorMode()
// colorMode.preference = 'dark' | 'light' | 'system'
</script>
```

`bg-default`, `text-default`, `text-muted`, `border-default` kabi semantik klasslar ham bor — ular light/dark'da o'zi to'g'ri rangga o'tadi (qo'lda `dark:bg-...` yozish shart emas).

8. Ikonkalar (Icon)

`@nuxt/icon` + Iconify — 200,000+ ikonka. Format: `i-{collection}-{name}`.

```
<UIcon name="i-lucide-rocket" class="size-5 text-primary" />
<UButton icon="i-lucide-plus">Qo' shish</UButton>
<UButton leading-icon="i-lucide-arrow-left" trailing-icon="i-lucide-arrow-right" />
```

Mashhur to'plamlar: `i-lucide-*` (Lucide — toza, tavsiya), `i-heroicons-*`, `i-simple-icons-*` (brendlar: github, google). Loading ikonkasini globalda o'zgartirish: `app.config.ts` → `ui.icons.loading`.

Xulosa — mental model

```
@nuxt/ui o'rnatish:
1. modules: ['@nuxt/ui'] + css faylga @import
2. app.vue ni <UApp> ga o'ra (Toast/Overlay uchun SHART)
```

Har komponent: color + variant + size

- color → semantik (primary/success/error/neutral...)
- variant → solid/outline/soft/subtle/ghost/link
- size → xs/sm/md/lg/xl

Theming (3 qatlam):

@theme (main.css) → xom token (rang 50–950, shrift)
ui.colors (app.config) → semantik mapping (runtime-reaktiv)
ui.<component> (app.config) → default override

Bitta instansiya:

:ui prop (slotlar) yoki class (root) – Tailwind Variants aqlli merge qiladi

Masalalar (20 ta)

`npx nuxt init ui-lab` → Nuxt UI o'rnat → `app.vue` ni `<UApp>` ga o'ra. Hamma masalani shu loyihada bajar.

A — setup va asoslar

1. ★ Yangi Nuxt loyiha och, `@nuxt/ui` o'rnat, `main.css` ni to'g'ri sozla.
`<UButton>Salom</UButton>` ekranga chiqsin. ✓
2. ★ `app.vue` ni `<UApp>` ga o'ra. Keyin `useToast().add({ title: 'Test' })` ni tugmaga ulab, toast chiqishini tekshir. (Avval `UApp` siz sinab ko'r — nima bo'ladi?) ✓
3. ★ Bitta qatorda 7 ta button yarat, har birida boshqa `color` (`primary`→`neutral`). Vizual farqni ko'r.
4. ★ Bitta `color="primary"` button'ning 6 ta variantini (`solid` → `link`) yonma-yon chiqar.
5. ★ `size` ni `xs` dan `xl` gacha o'zgartirib, button va input qanday kattalashishini kuzat.

B — theming

1. ★★ `@theme` ichida o'z `brand` rangingni (50–950) e'lon qil, `app.config.ts` da `primary: 'brand'` qil. Button rangi o'zgardimi? ✓
2. ★★ `app.config.ts` orqali **barcha** button'larni default `variant: 'soft'`, `size: 'lg'` qil. Endi `<UButton>` (propsiz) qanday ko'rinadi?
3. ★★ `app.config.ts` da `card.slots.root` ga `rounded-2xl` ber. Barcha kartalar o'zgardimi?
4. ★★ Bitta `UCard` ga `:ui="{ body: 'p-10' }"` ber. Global config'ni emas, faqat shu kartani o'zgartirdimi? Ustuvorlikni tushuntir. ✓

5. ? `class="p-8"` bilan `:ui="{ base: 'p-2' }"` urishsa, qaysi biri yutadi va nega? `tailwind-merge` roli nima?
6. ★★ `secondary` va `success` ranglarini boshqacha Tailwind ranglarga map qil (`app.config.ts`). Alert'larda sina.

C — dark mode va ikonka

1. ★ `<UColorModeButton />` qo'shib, light/dark almashtir. Komponentlar o'zi moslashdimi?
2. ★★ `bg-default`, `text-muted`, `border-default` klasslari bilan oddiy div yasab, dark mode'da rangi o'zgarishini kuzat (`dark:` yozmasdan).
3. ★★ `useColorMode()` bilan o'z toggle tugmangni yoz (`colorMode.preference` ni almashtir).
4. ★ 5 xil ikonka chiqar: `i-lucide-*` to'plamidan 3 ta, `i-simple-icons-github`, `i-heroicons-bell`. `class` bilan rang/o'lcham ber.
5. ★★ Button'ga `loading` prop ber — loading ikonkasi paydo bo'ladimi? Keyin `loading-auto` bilan form submit'da avtomatik loading'ni sina (08/Form bilan bog'la).

D — EduCore tematik

1. ★★ EduCore brendini o'rnat: `primary` = o'z rangin, `neutral` = `slate`, shrift = `Inter`. Bosh sahifada brendlangan button + card ko'rsat. ✓
2. ★★ "O'quvchi qo'shish" (primary, solid), "Tahrirlash" (neutral, outline), "O'chirish" (error, soft) — 3 ta amaliy button'ni to'g'ri rang/variant bilan yasab, ButtonGroup'ga joyla. (ButtonGroup — 10-modulda.)
3. ? Nega EduCore'da `bg-red-500` o'rniga `color="error"` ishlatish strategik jihatdan to'g'ri? Kelajakda brend o'zgarsa nima yutasan?
4. ★★★ `app.config.ts` da to'liq EduCore "design preset" yoz: ranglar + button default'lari + card default'lari + input default `size`. Buni bitta faylda jamlab, "tema" sifatida hujjatlashtir. ✓

✓ Tanlangan yechimlar

1 — o'rnatish

```
pnpm dlx nuxi@latest init ui-lab
cd ui-lab
pnpm add @nuxt/ui tailwindcss
```

```
// nuxt.config.ts
export default defineNuxtConfig({
  modules: ['@nuxt/ui'],
  css: ['~/assets/css/main.css'],
})
```

```
/* app/assets/css/main.css */
@import "tailwindcss";
@import "@nuxt/ui";
```

```
<!-- app/app.vue -->
<template>
  <UApp>
    <div class="p-8">
      <UButton>Salom</UButton>
    </div>
  </UApp>
</template>
```

pnpm dev → button chiroyli ko'rinadi. Agar style yo'q bo'lsa — main.css ni css: [...] ga qo'shishni unutgansan.

2 — UApp va toast

```
<script setup>
const toast = useToast()
function notify() {
  toast.add({ title: 'Test', description: 'Toast ishladi!', color: 'success' })
}
</script>
<template>
  <UApp>
    <UButton @click="notify">Toast chiqar</UButton>
  </UApp>
</template>
```

UApp siz: useToast() ishlaydi, lekin toast **ekranda ko'rinmaydi** (Toaster provayderi yo'q) — yoki konsolda inject ogohlantirishi chiqadi. UApp Toaster'ni daraxtga qo'shadi (provide/inject, 03-modul). Shuning uchun u **shart**.



6 — custom brand rang



```
/* main.css */
@import "tailwindcss";
@import "@nuxt/ui";

@theme {
  --color-brand-50: #eef2ff;
  --color-brand-100: #e0e7ff;
  --color-brand-200: #c7d2fe;
  --color-brand-300: #a5b4fc;
  --color-brand-400: #818cf8;
  --color-brand-500: #6366f1;
  --color-brand-600: #4f46e5;
  --color-brand-700: #4338ca;
  --color-brand-800: #3730a3;
  --color-brand-900: #312e81;
  --color-brand-950: #1e1b4b;
}
```

```
// app.config.ts
export default defineAppConfig({
  ui: { colors: { primary: 'brand' } },
})
```

Endi `<UButton>` indigo-brend rangida. Soyalaridan birortasini qoldirib ketsang (masalan 300) — o'sha soyani ishlatadigan komponent buziladi. Shuning uchun **to'liq 50–950**.



9 — :ui prop ustuvorligi



```
<UCard :ui="{ body: 'p-10' }">Faqat shu karta p-10</UCard>
<UCard>Bu karta default padding'da</UCard>
```

`:ui` faqat **shu instansiyaga** ta'sir qiladi — global config'ni o'zgartirmaydi. Ustuvorlik: `class` > `:ui` > `app.config.ts` global > komponent default. Ya'ni: umumiy qoidani `app.config.ts` da, istisnoni `:ui / class` da. Bu Laravel'da global config vs runtime override'ga o'xshaydi.



```
/* main.css */
@import "tailwindcss";
@import "@nuxt/ui";
@theme {
  --font-sans: 'Inter', sans-serif;
  --color-edu-50: #ecfeff; --color-edu-500: #0891b2; --color-edu-950:
#083344;
  /* ... to'liq shkala ... */
}
```

```
// app.config.ts
export default defineAppConfig({
  ui: {
    colors: { primary: 'edu', neutral: 'slate' },
  },
})
```

```
<template>
  <div class="p-8 space-y-4">
    <UCard>
      <template #header><h2 class="font-bold">EduCore</h2></template>
      <p class="text-muted">Multi-tenant ta'lim platformasi</p>
      <template #footer>
        <UButton color="primary">Boshlash</UButton>
      </template>
    </UCard>
  </div>
</template>
```

Brend bir joyda (`app.config.ts` + `@theme`) — butun ilova avtomatik moslashadi.



20 — EduCore design preset



```
// app/app.config.ts — EduCore "tema"
export default defineAppConfig({
  ui: {
    colors: {
      primary: 'edu',
      secondary: 'indigo',
      neutral: 'slate',
      error: 'red',
      success: 'emerald',
      warning: 'amber',
    },
    button: {
      slots: { base: 'font-medium rounded-lg' },
      defaultVariants: { size: 'md' },
    },
    card: {
      slots: { root: 'rounded-xl ring-default', body: 'p-6' },
    },
    input: {
      defaultVariants: { size: 'md' },
    },
    formField: {
      slots: { label: 'font-medium text-default' },
    },
  },
})
```

Bu — EduCore'ning yagona "manba" temasi. Yangi sahifa yozganda hech narsa sozlashing shart emas — barcha komponentlar shu presetdan keladi. Laravel'da `config/educore.php` ga o'xshaydi, lekin runtime-reaktiv.

[➡ Keyingi: 10 — Nuxt UI komponentlari \(to'liq ma'lumotnoma\)](#)

10 — Nuxt UI komponentlari (to'liq ma'lumotnoma)

[← Oldingi: 09 — Nuxt UI asoslar](#) · [🏠 README](#)

Bu — Nuxt UI'ning 125+ komponentini **kategoriya bo'yicha** ko'rib chiqadigan ma'lumotnoma. Eng ko'p ishlatadigan komponentlarni **chuqur** (props + slot + misol), nisbatan kam ishlatiladiganlarni **qisqa** (nima qiladi + asosiy props + mini-misol) ko'rsataman.

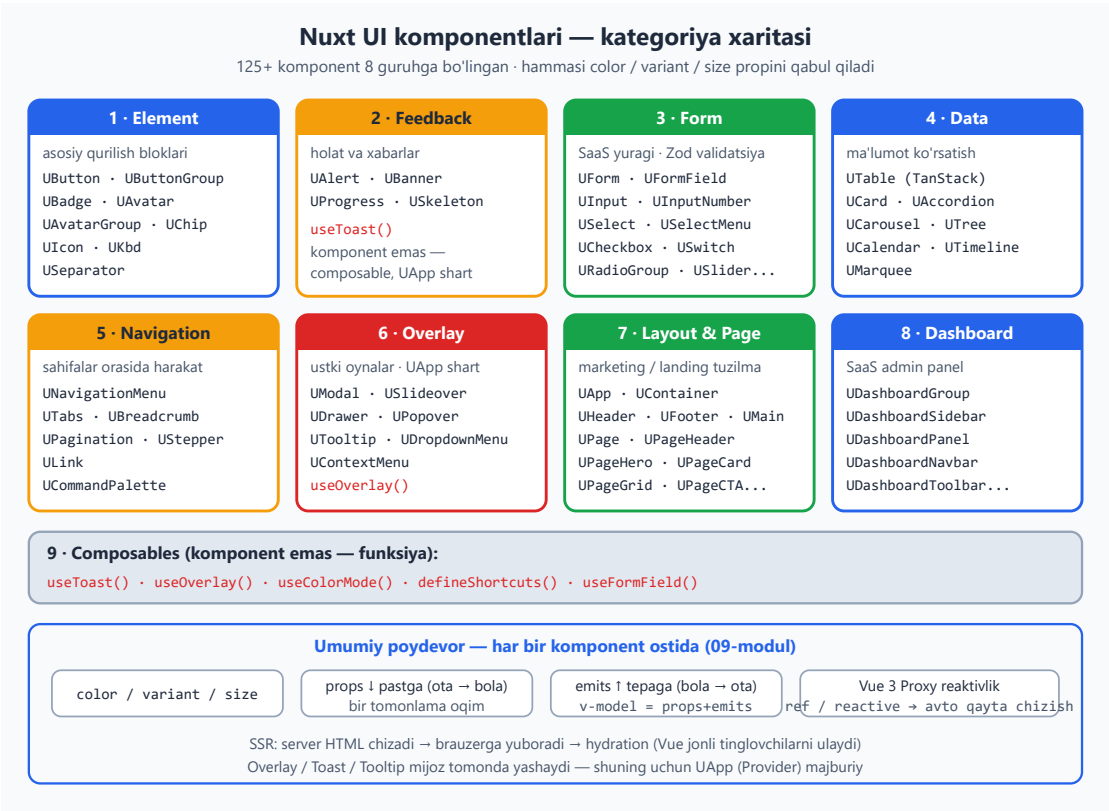
Buni o'qishdan oldin 09-modulni o'qigan bo'l. U yerda `color`, `variant`, `size`, `:ui`, slot tushunchalari bor — bu yerda ularni qayta tushuntirmayman. Deyarli har bir komponent o'sha 3 propni qabul qiladi.

💡 Har bir komponentning to'liq prop/slot ro'yxati

`ui.nuxt.com/docs/components/<nom>` da. Bu yerda **amaliy yadroni** beraman — kunlik ishda 90% shu kerak bo'ladi.

Kategoriyalar: 1. Element (Button, Badge, Avatar...) · 2. Feedback (Alert, Toast, Progress...) · 3. Form (Form + barcha inputlar) · 4. Data (Table, Card, Accordion...) · 5. Navigation (Menu, Tabs, Pagination...) · 6. Overlay (Modal, Slideover, Dropdown...) · 7. Layout & Page · 8. Dashboard · 9. Composables

Quyidagi xarita 8 kategoriyaning har birida nima borligini va hammasining tagidagi umumiy poydevorni (color/variant/size, props ↓ / emits ↑, Proxy reaktivlik, SSR/hydration) bir ko'rinishda jamlaydi:



1. Element — asosiy qurilish bloklari

Button — UIButton

Eng ko'p ishlatiladigan. Tugma yoki havola (to bersang <NuxtLink> bo'ladi).

Prop	Vazifa
label / default slot	matn
color, variant, size	09-modul
icon	faqat ikonka (label'siz → icon-only)
leading-icon / trailing-icon	chap/o'ng ikonka
avatar	ichida avatar ({ src })
loading	spinner ko'rsatadi

Prop	Vazifa
<code>loading-auto</code>	<code>type="submit"</code> da form yuborilayotganda avtomatik loading
<code>block</code>	to'liq kenglik (<code>w-full</code>)
<code>square</code>	kvadrat (icon-only uchun)
<code>disabled</code> , <code>to</code> , <code>type</code>	—

```
<UButton color="primary" icon="i-lucide-save" loading-auto
type="submit">Saqlash</UButton>
<UButton color="neutral" variant="ghost" square icon="i-lucide-x" />
<UButton to="/dashboard" trailing-icon="i-lucide-arrow-right">Panelga</UButton>
```

Slot'lar (`:ui` uchun): `base`, `label`, `leadingIcon`, `trailingIcon`.

ButtonGroup — UButtonGroup

Bir nechta button/inputni yopishtirib qo'yadi (segment).

```
<UButtonGroup size="md">
  <UButton color="neutral" variant="outline" label="Kun" />
  <UButton color="neutral" variant="outline" label="Hafta" />
  <UButton color="neutral" variant="outline" label="Oy" />
</UButtonGroup>
```

Badge — UBadge

Status/yorliq. `color`, `variant`, `size`, `label`, `icon`.

```
<UBadge color="success" variant="subtle">Faol</UBadge>
<UBadge color="error" variant="soft">To'lanmagan</UBadge>
```

Avatar / AvatarGroup — UAvatar, UAvatarGroup

```
<UAvatar src="https://github.com/nuxt.png" alt="Nuxt" size="lg" />
<UAvatar text="OQ" />
initisiallar -->
<UAvatar icon="i-lucide-user" />
```

```

<UAvatarGroup :max="3" size="md">
  <UAvatar src="..." /><UAvatar src="..." /><UAvatar text="AB" />
<UAvatar text="CD" />
</UAvatarGroup>                                <!-- ortig'i "+N"
bo'ladi -->

```

Chip — UChip

Boshqa elementga **nuqta/indikator** biriktiradi (masalan, avatarga "online" nuqtasi yoki xabar soni).

```

<UChip color="success" inset>
  <UAvatar src="..." />
</UChip>
<UChip :text="5" size="2x1" color="error">
  <UButton icon="i-lucide-bell" color="neutral" variant="ghost" />
</UChip>

```

Icon, Kbd, Separator

```

<UIcon name="i-lucide-rocket" class="size-5 text-primary" />
<UKbd value="meta" /> <UKbd value="K" />          <!-- klaviatura
tugmasi -->
<USeparator label="yoki" />                        <!-- ajratuvchi
chiziq -->

```

2. Feedback — holat va xabarlar

Alert — UAlert

Diqqat tortuvchi xabar (sahifa ichida turadi).

```

<UAlert
  color="warning" variant="subtle"
  icon="i-lucide-triangle-alert"
  title="Diqqat"
  description="To'lov muddati tugayapti."
  :actions="[{ label: 'To\'lash', color: 'warning' }]"
  :close="true"
/>

```

Props: title, description, icon, color, variant, :actions (button massivi), :close, orientation.

Toast — `useToast` (komponent emas, composable)

Vaqtinchalik bildirishnoma (o'ng pastda paydo bo'lib yo'qoladi). `<UApp>` **shart** (09-modul).

```
<script setup>
const toast = useToast()
toast.add({
  title: 'Saqlandi',
  description: 'O\'quvchi ma\'lumoti yangilandi.',
  color: 'success',
  icon: 'i-lucide-check',
  duration: 4000,
  actions: [{ label: 'Bekor qilish', onClick: () => {} }],
})
// toast.remove(id) · toast.update(id, {...}) · toast.clear()
</script>
```

Laravel analogiyasi. `useToast` $\approx$ session flash message (`->with('success', '...')`) — lekin client-side, dinamik va action'lar bilan.

Progress, Skeleton, Banner

```
<UProgress v-model="percent" :max="100" />          <!-- yoki animatsion <!-- yoki animatsion
(indeterminate) -->
<USkeleton class="h-12 w-full" />                  <!-- yuklanish
placeholder -->
```

`UBanner` — sahifa tepasidagi global e'lon chizig'i (`app.vue` da, `<UApp>` ichida):

```
<UBanner icon="i-lucide-megaphone" title="EduCore v2 chiqdi!"
to="/whats-new" />
```

3. Form — eng muhim qism (SaaS yuragi)

Umumiy arxitektura

Nuxt UI form tizimi 2 komponentdan iborat: - `UForm` — validatsiya + submit'ni boshqaradi. - `UFormField` — bitta maydon (label + xato xabari) o'rami.

Validatsiya **Standard Schema** kutubxonalari bilan: **Zod**, Valibot, Yup, Joi, Superstruct yoki o'z funksiyang. Hech qaysi biri ichida kelmaydi — kerakligini o'rnatasan (`pnpm add zod`).

```

<script setup lang="ts">
import * as z from 'zod'
import type { FormSubmitEvent } from '@nuxt/ui'

const schema = z.object({
  email: z.email('Email noto\'g\'ri'),
  password: z.string().min(8, 'Kamida 8 belgi'),
})
type Schema = z.output<typeof schema>

const state = reactive<Partial<Schema>>(>({ email: undefined, password:
undefined })
const toast = useToast()

async function onSubmit(event: FormSubmitEvent<Schema>) {
  // event.data - validatsiyadan o'tgan, type-safe data
  await $fetch('/api/login', { method: 'POST', body: event.data })
  toast.add({ title: 'Kirildi', color: 'success' })
}
</script>

<template>
  <UForm :schema="schema" :state="state" class="space-y-4"
  @submit="onSubmit">
    <UFormField label="Email" name="email" required>
      <UInput v-model="state.email" type="email" class="w-full" />
    </UFormField>

    <UFormField label="Parol" name="password" required>
      <UInput v-model="state.password" type="password" class="w-full"
/>
    </UFormField>

    <UButton type="submit" loading-auto>Kirish</UButton>
  </UForm>
</template>

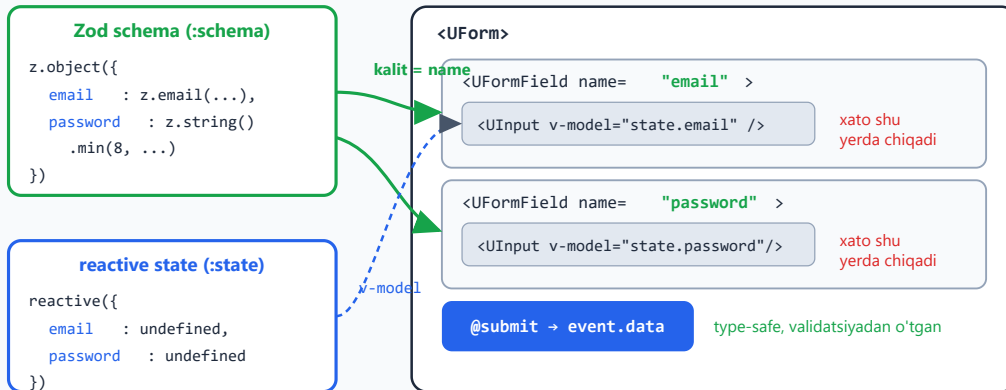
```

Eng muhim mexanizm: `UFormField` ning `name` proli `schema` dagi maydon nomi bilan bir xil bo'lishi kerak — shunda xato xabari avtomatik o'sha maydon tagida chiqadi. Qo'lda hech narsa ulamaysan.

Quyidagi diagramma bu bog'lanishni va Vue 3 oqim semantikasini (props ↓ pastga, emits ↑ tepaga, `v-model` = ikkalasi) ko'rsatadi:

UForm mexanizmi: name = schema kaliti

Standard Schema (Zod) bitta haqiqat manbai · client UX + server xavfsizlik bir fayldan



Vue 3 oqim semantikasi:

props ↓ pastga oqadi (ota → bola, faqat o'qish) · emits ↑ tepaga signal beradi (bola → ota)
v-model = :modelValue (prop, pastga) + @update:modelValue (emit, tepaga) – sun'iy ikki tomon
Agar name schema kaliti bilan mos kelmasa — xato o'sha maydonga ulanmaydi, umumiy bo'lib qoladi.

Laravel analogiyasi. Bu `FormRequest` validatsiyasiga juda o'xshaydi: `rules()` = Zod schema, validation messages avtomatik maydonlarga biriktiriladi. **Bonus:** Zod schema'ni `server/api/` da **qayta ishlatish** mumkin — bitta schema ham clientda, ham serverda validatsiya qiladi (DRY). Laravel'da rule'lar faqat serverda; bu yerda ikkalasi.

UForm props/xususiyatlari: - `:schema`, `:state`, `@submit`, `@error` - `validate-on` — qachon tekshirish: `['blur', 'input', 'change', 'submit']` - `:transform` — submit'da Zod `.transform()` natijasini qaytarish (default true) - `nested` — ichki (child) form - Default slot `{ errors, loading }` ni beradi. - **Template ref** orqali metodlar: `formRef.value.validate()`, `.clear()`, `.submit()`, `.setErrors([...])`, `.getErrors()`.

UFormField props: `label`, `name`, `description`, `help`, `hint`, `error` (qo'lda xato), `required`, `size`, `eager-validation`.

Input komponentlari

Hammasi `v-model`, `color` (focus ring), `variant`, `size`, `icon`, `disabled` ni qabul qiladi.

UInput — matn/email/parol/raqam:

```
<UInput v-model="q" icon="i-lucide-search" placeholder="Qidirish..." />
<UInput v-model="pwd" type="password" :ui="{ trailing: 'pe-1' }">
  <template #trailing>
    <UButton color="neutral" variant="link" icon="i-lucide-eye"
      @click="toggle" />
  </template>
</UInput>
```

```
</template>
</UInput>
```

UInputNumber — raqam (stepper bilan): `v-model`, `:min`, `:max`, `:step`, `:format-options`.

UTextarea — ko'p qatorli: `:rows`, `autoresize`, `:maxrows`.

USelect — oddiy dropdown (native-ga yaqin):

```
<USelect v-model="role" :items="['admin', 'teacher', 'student']"
placeholder="Rol" />
<!-- obyekt items: -->
<USelect v-model="val" :items="[{ label: 'Admin', value: 'a' }]" />
```

USelectMenu — kuchaytirilgan select: **qidiruv**, multiple, custom item rendering, "yangi yaratish":

```
<USelectMenu v-model="selected" :items="users" label-key="name"
searchable multiple />
```

UInputMenu — combobox (autocomplete + erkin matn / teglar kiritish).

UCheckbox / **UCheckboxGroup**, **URadioGroup**, **USwitch**:

```
<UCheckbox v-model="agree" label="Shartlarga roziman" />
<USwitch v-model="active" label="Faol" />
<URadioGroup v-model="plan" :items="[{label:'Free',value:'free'},
{label:'Pro',value:'pro'}]" />
```

USlider — diapazon: `v-model`, `:min`, `:max`, `:step`.

UPinInput — OTP/PIN kiritish: `:length`, `type`, `otp`.

UFileUpload — fayl yuklash (drag-drop): `v-model`, `accept`, `multiple`.

UColorPicker — rang tanlash.

Barchasi `UFormField` ichida ishlaydi va `name` orqali avtomatik xato ko'rsatadi.

4. Data — ma'lumot ko'rsatish

Table — **UTable** (eng kuchli data komponenti)

TanStack Table (`useVueTable`) + **TanStack Virtual** ustiga qurilgan. To'liq type-safe: sort, filter, paginatsiya, qator tanlash, kengaytirish, virtualizatsiya.

```
<script setup lang="ts">
import { h, resolveComponent } from 'vue'
import type { TableColumn } from '@nuxt/ui'

const UBadge = resolveComponent('UBadge')

type Student = { id: number; name: string; status: 'active' | 'inactive'; balance: number }

const data = ref<Student[]>([
  { id: 1, name: 'Ali Valiyev', status: 'active', balance: 250000 },
  { id: 2, name: 'Lola Karimova', status: 'inactive', balance: 0 },
])

const columns: TableColumn<Student>[] = [
  { accessorKey: 'id', header: 'ID' },
  { accessorKey: 'name', header: 'F.I.O' },
  {
    accessorKey: 'status',
    header: 'Holat',
    // cell – render funksiya (h yoki resolveComponent bilan)
    cell: ({ row }) => h(UBadge, {
      color: row.original.status === 'active' ? 'success' : 'neutral',
      variant: 'subtle',
    }, () => row.original.status),
  },
  {
    accessorKey: 'balance',
    header: () => h('div', { class: 'text-right' }, 'Balans'),
    cell: ({ row }) => h('div', { class: 'text-right' },
      new Intl.NumberFormat('uz').format(row.original.balance) + "
      so'm"),
  },
]
</script>

<template>
  <UTable :data="data" :columns="columns" :loading="pending" sticky />
</template>
```

Asosiy nuqtalar: - `:data` — obyektlar massivi. `:columns` bermasang — kalitlardan avtomatik yasaladi. - `column.cell: ({ row }) => ...` — `row.original` orqali qator datasiga kirasan; `h()` bilan istalgan komponent render qilasan (Badge, Button...). - `column.meta.class.{ th, td }` — ustun stilini berish. - **Reaktiv holatlar:** `v-model:row-selection`, `v-model:sorting`, `v-model:column-filters`, `v-model:expanded`, `v-model:pagination`. - **Slot'lar:** `#<column>-cell`, `#<column>-header`, `#expanded`, `#empty`, `#loading`. - Template ref → `tableApi` (TanStack instansiyasi — full nazorat).

Laravel analogiyasi. UTable + TanStack ≈ Laravel'dagi data-table paketlari (yoki Filament Table) — lekin frontend-side. Server-side paginatsiya/sort kerak bo'lsa: holatlarni (sorting , pagination) watch qilib, useFetch query'siga uzatasan (08-modul bilan birlashtir).

Card — UCard

```
<UCard>
  <template #header><h3 class="font-semibold">Statistika</h3>
</template>
  <p>Asosiy kontent shu yerda.</p>
  <template #footer>
    <UButton color="primary">Batafsil</UButton>
  </template>
</UCard>
```

Slot/ :ui kalitlari: root , header , body , footer .

Accordion — UAccordion

```
<UAccordion :items="[
  { label: 'Savol 1', icon: 'i-lucide-help-circle', content: 'Javob 1' },
  { label: 'Savol 2', content: 'Javob 2' },
]" type="single" collapsible />
```

type: single | multiple . Murakkab kontent uchun #content slot.

Boshqalar (qisqa)

- UCarousel — slayder (Embla asosida): :items + #default="{ item }" . loop , :autoplay , arrows , dots .
- UMarquee — uzluksiz aylanuvchi lenta (logotiplar uchun).
- UTree — daraxt tuzilma: :items (nested), v-model .
- UCalendar — sana tanlash kalendar (range, multiple).
- UTimeline — vaqt chizig'i (qadamlar tarixi).

5. Navigation — navigatsiya

NavigationMenu — UNavigationMenu

Asosiy menyu (gorizontal yoki vertikal — sidebar uchun ideal).

```
<script setup lang="ts">
import type { NavigationMenuItem } from '@nuxt/ui'
const items: NavigationMenuItem[] = [
  { label: 'Bosh sahifa', icon: 'i-lucide-house', to: '/' },
  { label: 'O\'quvchilar', icon: 'i-lucide-users', to: '/students',
    badge: '24' },
  {
    label: 'Sozlamalar', icon: 'i-lucide-settings',
    children: [
      { label: 'Profil', to: '/settings/profile' },
      { label: 'Billing', to: '/settings/billing' },
    ],
  },
]
</script>
<template>
  <UNavigationMenu :items="items" orientation="vertical" />
</template>
```

Item maydonlari: `label`, `icon`, `to`, `active`, `badge`, `children`, `disabled`, `onSelect`.

Tabs — UTabs

```
<UTabs :items="[
  { label: 'Umumiy', icon: 'i-lucide-info', slot: 'general' },
  { label: 'Xavfsizlik', slot: 'security' },
]" variant="link">
  <template #general>Umumiy kontent</template>
  <template #security>Xavfsizlik kontenti</template>
</UTabs>
```

`variant`: `pill` | `link`. `orientation`: `:content="false"` (faqat tab tugmalari).

Breadcrumb, Pagination, Stepper, Link

```
<UBreadcrumb :items="[ {label:'Bosh',to:'/'},
{label:'O\'quvchilar',to:'/students'}, {label:'Ali'}]" />

<UPagination v-model:page="page" :total="240" :items-per-page="20"
show-edges />

<UStepper v-model="step" :items="[
  { title: 'Ma'lumot', icon: 'i-lucide-user' },
  { title: 'To'lov', icon: 'i-lucide-credit-card' },
  { title: 'Tasdiq', icon: 'i-lucide-check' },
]" />
```

```
<ULink to="/about" active-class="text-primary">Biz haqimizda</ULink>
```

CommandPalette — UCommandPalette

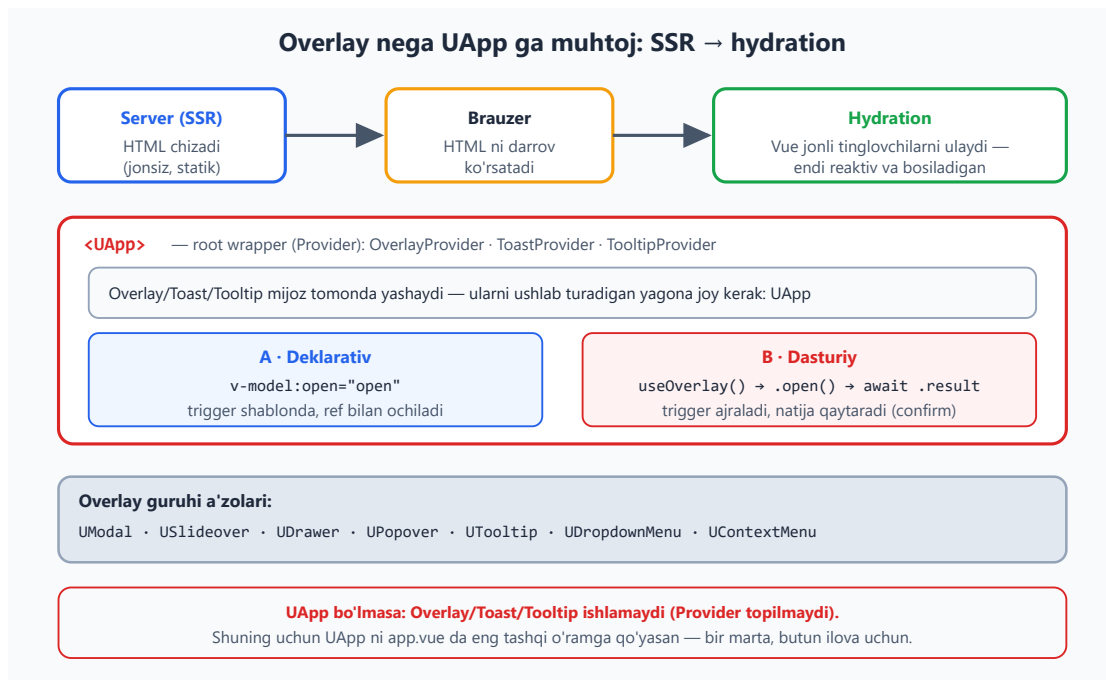
Cmd+K qidiruv oynasi (Fuse.js fuzzy qidiruv). Odatda Modal ichida:

```
<UCommandPalette :groups="[
  { id: 'pages', label: 'Sahifalar', items: [
    { label: 'O\'quvchilar', icon: 'i-lucide-users', to: '/students' },
  ] },
]" placeholder="Buyruq yoki sahifa qidirish..." />
```

6. Overlay — ustki oynalar

Bu guruh **<UApp>** ga muhtoj (OverlayProvider). Ikki ishlatish usuli bor: **deklarativ** (v-model:open) va **dasturiy** (useOverlay).

Quyidagi diagramma SSR → hydration oqimini va nega Overlay/Toast/Tooltip uchun UApp Provider majburiy ekanini, hamda ikki ochish usulini ko'rsatadi:



Modal — UModal

Deklarativ:

```

<script setup>
const open = ref(false)
</script>
<template>
  <UModal v-model:open="open" title="O'quvchini o'chirish"
    description="Bu amalni qaytarib bo'lmaydi.">
    <UButton color="error" label="O'chirish" />    <!-- trigger (default
slot) -->

    <template #body>
      <p>Haqiqatan ham o'chirmoqchimisiz?</p>
    </template>
    <template #footer="{ close }">
      <UButton color="neutral" variant="outline" label="Bekor"
@click="close" />
      <UButton color="error" label="O'chirish" @click="remove" />
    </template>
  </UModal>
</template>

```

Slot'lar: default (trigger), #content (butun ichki), yoki #header / #body / #footer alohida. Props: title, description, dismissible (tashqariga bosib yopish), :overlay, :transition, fullscreen, :close.

Slideover — USlideover

Ekran chetidan suriladi (filtrlar, detallar paneli uchun). side: right (default) | left | top | bottom. Slot/proplar Modal'ga o'xshash.

```

<USlideover v-model:open="open" title="Filtrlar" side="right">
  <UButton label="Filtr" icon="i-lucide-filter" />
  <template #body><!-- filtr formasi --></template>
</USlideover>

```

Drawer — UDrawer

Pastdan chiqadigan "bottom sheet" (mobil-uslub). modal, dismissible, #content.

useOverlay — dasturiy ochish (kuchli pattern)

Trigger'ni komponentdan ajratish va **natija qaytarish** uchun. Confirm dialog'lar uchun ideal.

```

<!-- 1. Alohida modal komponent - close event chiqaradi -->
<!-- components/ConfirmModal.vue -->
<script setup lang="ts">

```

```
defineProps<{ title: string }>()
const emit = defineEmits<{ close: [boolean] }>()
</script>
<template>
  <UModal :title="title">
    <template #footer>
      <UButton variant="outline" label="Yo'q" @click="emit('close', false)" />
      <UButton color="error" label="Ha" @click="emit('close', true)" />
    </template>
  </UModal>
</template>
```

```
<!-- 2. Istalgan joyda dasturiy chaqirish -->
<script setup lang="ts">
import { LazyConfirmModal } from '#components'
const overlay = useOverlay()
const confirmModal = overlay.create(LazyConfirmModal)

async function deleteStudent(id: number) {
  const instance = confirmModal.open({ title: 'O\'chirishni tasdiqlang' })
  const confirmed = await instance.result // close event qiymatini kutadi
  if (confirmed) {
    await $fetch(`/api/students/${id}`, { method: 'DELETE' })
  }
}
</script>
```

Why bu zo'r. Trigger button modal'dan ajraladi — bitta `ConfirmModal` ni butun ilovada qayta ishlatasan, `await` bilan "ha/yo'q" natijani olasan. `useOverlay` `createSharedComposable` asosida — holat butun ilovada bitta.

DropdownMenu, ContextMenu — `UDropdownMenu`, `UContextMenu`

```
<UDropdownMenu :items="[
  [{ label: 'Profil', icon: 'i-lucide-user' }, { label: 'Sozlama', icon: 'i-lucide-settings' }],
  [{ label: 'Chiqish', icon: 'i-lucide-log-out', color: 'error', onSelect: logout }],
]">
  <UButton icon="i-lucide-ellipsis-vertical" color="neutral" variant="ghost" />
</UDropdownMenu>
```

Ichki massivlar — guruhlar (orasiga ajratuvchi qo'yadi). `UContextMenu` — o'ng-klik menyusi (xuddi shu items).

Popover, Tooltip — UPopover, UTooltip

```
<UPopover mode="click">
  <UButton label="Ko'proq" />
  <template #content><div class="p-4">Erkin kontent</div></template>
</UPopover>

<UTooltip text="O'chirish" :kbds="['meta', 'D']">
  <UButton icon="i-lucide-trash" color="error" variant="ghost" />
</UTooltip>
```

UTooltip ham <UApp> (TooltipProvider) ga muhtoj.

7. Layout & Page (ilgari Pro — endi bepul)

Marketing/landing va hujjat sahifalari uchun tayyor tuzilmalar.

- **UApp** — root wrapper (majburiy, 09-modul).
- **UContainer** — markazlashgan, max-width konteyner.
- **UHeader** / **UFooter** — sayt header/footer (slot'lar: #left, #right, #center).
- **UMain** — asosiy kontent zonasi (header balandligini hisobga oladi).
- **UPage** — #left (sidebar) + default + #right (TOC) ustunli layout.
- **UPageHeader** — title, description, :links, headline.
- **UPageBody**, **UPageGrid**, **UPageColumns** — kontent tartibi.
- **UPageCard**, **UPageFeature**, **UPageSection**, **UPageHero**, **UPageCTA** — landing bloklari.
- **UPageList**, **UPageLinks**, **UPageLogos**, **UPageMarquee**, **UPageAccordion** — yordamchi bloklar.

```
<template>
  <UContainer>
    <UPageHero
      title="EduCore"
      description="O'quv markazlar uchun zamonaviy boshqaruv tizimi"
      :links="{ label: 'Boshlash', to: '/register', color: 'primary'
    }]"
    />
    <UPageGrid>
      <UPageCard title="Multi-tenant" description="Har markaz alohida"
        icon="i-lucide-building" />
      <UPageCard title="To'lovlar" description="Avtomatik hisob-kitob"
        icon="i-lucide-wallet" />
    </UPageGrid>
  </UContainer>
</template>
```

```
</UContainer>
</template>
```

8. Dashboard (ilgari Pro — endi bepul) — SaaS admin uchun

EduCore admin paneli aynan shu komponentlar bilan tez yig'iladi. Bular birga ishlaydi.

- **UDashboardGroup** — butun panel wrapper (sidebar holatini, kenglikni boshqaradi, localStorage'da saqlaydi).
- **UDashboardSidebar** — yon panel: `collapsible`, `resizable`, `mode` (mobil: `drawer` / `slideover` / `modal`). Slot'lar: `#header`, `default`, `#footer`. { `collapsed` } slot prop.
- **UDashboardPanel** — kontent paneli (`id`, `resizable`). Slot'lar: `#header`, `#body`, `#footer`.
- **UDashboardNavbar** — panel ustki paneli: `title`, `icon`, `:toggle`. Slot'lar: `#leading`, `#trailing`, `#right`.
- **UDashboardToolbar** — navbar ostidagi qo'shimcha panel (filtrlar, tablar).
- **UDashboardSearch** / **UDashboardSearchButton** — Cmd+K qidiruv.
- **UDashboardResizeHandle** — o'lcham o'zgartirish dastagi.

```
<!-- app/layouts/dashboard.vue -->
<script setup lang="ts">
import type { NavigationMenuItem } from '@nuxt/ui'
const items: NavigationMenuItem[] = [
  { label: 'Bosh', icon: 'i-lucide-house', to: '/' },
  { label: 'O\'quvchilar', icon: 'i-lucide-users', to: '/students' },
  { label: 'To'lovlar', icon: 'i-lucide-wallet', to: '/payments' },
]
</script>
<template>
  <UDashboardGroup>
    <UDashboardSidebar collapsible resizable>
      <template #header="{ collapsed }">
        <span v-if="!collapsed" class="font-bold text-
lg">EduCore</span>
        <UIcon v-else name="i-lucide-graduation-cap" class="size-6
text-primary" />
      </template>

      <UNavigationMenu :items="items" orientation="vertical" />

      <template #footer>
```

```

        <UDropdownMenu :items="[[{ label: 'Chiqish', icon: 'i-lucide-
log-out' }]]">
            <UButton block color="neutral" variant="ghost" :avatar="{
src: '/me.png' }" label="Profil" />
        </UDropdownMenu>
    </template>
</UDashboardSidebar>

<UDashboardPanel>
    <template #header>
        <UDashboardNavbar title="Boshqaruv paneli">
            <template #trailing><UBadge color="primary"
variant="subtle">Demo</UBadge></template>
            <template #right>
                <UColorModeButton />
                <UButton icon="i-lucide-bell" color="neutral"
variant="ghost" />
            </template>
        </UDashboardNavbar>
    </template>

    <template #body>
        <slot /> <!-- sahifa kontenti -->
    </template>
</UDashboardPanel>
</UDashboardGroup>
</template>

```

Laravel analogiyasi. Bu komponentlar to'plami ≈ **Filament panel** — sidebar, navbar, panel hammasi tayyor, faqat `items` va kontent berasan. Lekin bu yerda to'liq Vue, to'liq sozlanuvchi. EduCore admin'i uchun haftalarcha vaqtni tejaydi.

9. Composables — xulosa

Composable	Vazifa
<code>useToast()</code>	toast qo'shish/o'chirish (<code>.add</code> , <code>.remove</code> , <code>.update</code> , <code>.clear</code>)
<code>useOverlay()</code>	Modal/Slideover'ni dasturiy ochish (<code>.create</code> , <code>.open</code> , <code>awaitable</code>)
<code>useColorMode()</code>	dark/light boshqarish (<code>.preference</code> , <code>.value</code>)
<code>defineShortcuts({ ... })</code>	klaviatura yorliqlari (<code>meta_k: () => ...</code>)

Composable	Vazifa
<code>useFormField()</code>	maxsus input yozganda Form bilan ulanish (advanced)

```
<script setup>
defineShortcuts({
  'meta_k': () => { searchOpen.value = true },    // Cmd/Ctrl + K
  'g-d': () => navigateTo('/dashboard'),          // ketma-ket: g,
  keyin d
})
</script>
```

EduCore capstone — to'liq sahifa

Yuqoridagi `dashboard` layout + bu sahifa = ishlaydigan admin ekrani:

```
<!-- app/pages/students/index.vue -->
<script setup lang="ts">
import { h, resolveComponent } from 'vue'
import type { TableColumn } from '@nuxt/ui'
definePageMeta({ layout: 'dashboard' })

const UBadge = resolveComponent('UBadge')
const slideoverOpen = ref(false)

const { data, pending, refresh } = await useFetch('/api/students')

type Student = { id: number; name: string; status: string }
const columns: TableColumn<Student>[] = [
  { accessorKey: 'name', header: 'F.I.O' },
  { accessorKey: 'status', header: 'Holat',
    cell: ({ row }) => h(UBadge, { color: row.original.status ===
'active' ? 'success' : 'neutral', variant: 'subtle' }, () =>
row.original.status) },
]
</script>

<template>
  <UDashboardToolbar>
    <UInput icon="i-lucide-search" placeholder="Qidirish..." />
    <UButton icon="i-lucide-plus" label="O'quvchi qo'shish"
@click="slideoverOpen = true" class="ms-auto" />
  </UDashboardToolbar>

  <div class="p-4">
    <UTable :data="data ?? []" :columns="columns" :loading="pending" />
  </div>
```

```
<USlideover v-model:open="slideoverOpen" title="Yangi o'quvchi"
side="right">
  <template #body>
    <!-- 3-bo'limdagi UForm shu yerga -->
  </template>
</USlideover>
</template>
```

Bu — 07/08-moduldagi Nuxt asoslari + 09-modul theming + shu modul komponentlari birga. Mana shunday EduCore admin'i yig'iladi.

Masalalar (22 ta)

09-moduldagi `ui-lab` loyihasida davom et. Mock data uchun `server/api/` (08-modul) ishlat.

A — element va feedback

1. ★ `UButton` ning barcha variant + 3 ta rangini jadval ko'rinishida chiqar (matritsa).
2. ★ `UAvatarGroup` (`:max="3"`) bilan 6 ta avatar — ortig'i "+3" bo'lsin.
3. ★ Bell ikonkali button'ga `UChip` bilan o'qilmagan xabar soni (5) qo'sh.
4. ★★ `UAlert` ning 4 holatini (success/info/warning/error) `:actions` va `:close` bilan yasab chiqar. ✓
5. ★★ `useToast` bilan 4 xil rangdagi toast chiqaruvchi 4 tugma. Bittasiga `actions` ("Bekor qilish") qo'sh. Avval `UApp` borligini tekshir. ✓

B — form (eng muhim)

1. ★★ Login formasi: Zod schema (email + parol min 8). `UForm` + 2 ta `UFormField` + submit. Xato xabarlari to'g'ri maydonda chiqsinmi? ✓
2. ★★ Ro'yxatdan o'tish formasi: ism, email, parol, parolni takrorlash (`.refine` bilan moslik tekshir), shartlarga rozilik (`UCheckbox`). ✓
3. ★★ `USelectMenu` (searchable) bilan "o'qituvchi tanlash" maydoni qo'sh (mock ro'yxatdan). `UFormField` ichida.
4. ★★ `UInputNumber` bilan "narx" maydoni (`:min="0"` , `format-options` so'm uchun).
5. ? Nima uchun `UFormField` ning `name` proli schema kaliti bilan bir xil bo'lishi shart? Bir xil bo'lmasa nima bo'ladi?

6. ★★★ Bitta Zod schema'ni ham `UForm` da (client), ham `server/api/students.post.ts` da (server) ishlat. DRY validatsiya. (08-modul bilan birlashtir.) ✓

C — data

1. ★★ `UTable` yasab, mock o'quvchilar ro'yxatini ko'rsat. `status` ustunini `UBadge` bilan render qil (`cell + h`). ✓
2. ★★ `UTable` ga `v-model:row-selection` qo'shib, tanlangan qatorlar sonini tepada ko'rsat.
3. ★★ `UTable` + `UPagination`: `page` ni `useFetch` query'siga uzatib, server-side paginatsiya qil.
4. ★★ `UAccordion` bilan FAQ bo'limi (5 savol) yasab, `#content` slot bilan boyit.
5. ★ `UCard` bilan 3 ta "statistika kartasi" (o'quvchilar soni, daromad, faol guruhlar) — header/body/footer bilan.

D — navigation va overlay

1. ★★ `UNavigationMenu` (vertical) bilan EduCore sidebar menyusi — `children` (ichki menyu) va `badge` bilan. ✓
2. ★★ `UTabs` bilan profil sahifasi: "Umumiy", "Xavfsizlik", "Biljing" tablari.
3. ★★ `UModal` (deklarativ) bilan "o'chirishni tasdiqlash" dialogi — footer'da Bekor/O'chirish.
4. ★★★ `useOverlay` bilan **qayta ishlatiluvchi** `ConfirmModal` yoz (close event qaytaradi). `await` bilan "ha/yo'q" olib, jadvaldan qator o'chir. ✓
5. ★★ `UDropdownMenu` bilan jadval qatoridagi "..." amallar menyusi (Tahrirlash / O'chirish, guruhlangan).

E — to'liq yig'ish

1. ★★★ **EduCore admin ekrani.** `dashboard` layout (`UDashboardGroup` + `Sidebar` + `Navbar`) + o'quvchilar sahifasi (`UTable` + `UDashboardToolbar` + qo'shish uchun `USlideover` ichida `UForm`). Capstone misolini asos qilib, ishlaydigan holatga keltir. ✓

✓ Tanlangan yechimlar



4 — Alert 4 holati



```
<template>
  <div class="space-y-3">
    <UAlert v-for="a in alerts" :key="a.color"
      :color="a.color" :icon="a.icon" :title="a.title"
      variant="subtle" :close="true"
      :actions="[ { label: 'Ko\'rish', color: a.color, variant: 'outline' } ]"
    />
  </div>
</template>
<script setup>
const alerts = [
  { color: 'success', icon: 'i-lucide-check-circle', title: 'Muvaffaqiyat' },
  { color: 'info', icon: 'i-lucide-info', title: 'Ma\'lumot' },
  { color: 'warning', icon: 'i-lucide-alert-triangle', title:
'Ogohlantirish' },
  { color: 'error', icon: 'i-lucide-x-circle', title: 'Xato' },
]
</script>
```

6 — Login formasi

```
<script setup lang="ts">
import * as z from 'zod'
import type { FormSubmitEvent } from '@nuxt/ui'

const schema = z.object({
  email: z.email('Email noto\'g\'ri'),
  password: z.string().min(8, 'Kamida 8 belgi'),
})
type Schema = z.output<typeof schema>
const state = reactive<Partial<Schema>>({ email: undefined, password:
undefined })
const toast = useToast()

async function onSubmit(e: FormSubmitEvent<Schema>) {
  toast.add({ title: 'Kirildi', description: e.data.email, color: 'success'
})
}
</script>
<template>
  <UForm :schema="schema" :state="state" class="space-y-4 max-w-sm"
@submit="onSubmit">
    <UFormField label="Email" name="email" required>
      <UInput v-model="state.email" type="email" class="w-full" icon="i-
lucide-mail" />
    </UFormField>
    <UFormField label="Parol" name="password" required>
      <UInput v-model="state.password" type="password" class="w-full" />
    </UFormField>
    <UButton type="submit" block loading-auto>Kirish</UButton>
  </UForm>
</template>
```

`name="email"` schema kaliti bilan bir xil — shuning uchun "Email noto'g'ri" aynan email maydoni tagida chiqadi.

7 — Parol moslik tekshiruvi (.refine)

```
const schema = z.object({
  name: z.string().min(2, 'Ism kamida 2 belgi'),
  email: z.email(),
  password: z.string().min(8),
  confirm: z.string(),
  agree: z.boolean().refine(v => v === true, 'Shartlarni qabul qiling'),
}).refine(data => data.password === data.confirm, {
  message: 'Parollar mos kelmadi',
  path: ['confirm'], // xato 'confirm' maydoniga biriktiriladi
})
```

`path: ['confirm']` — bu cross-field xatoni qaysi maydon tagida ko'rsatishni aytadi. Aks holda xato umumiy bo'lib qoladi.



11 — bitta schema, ikki tomon (DRY)



```
// shared/schemas/student.ts (Nuxt 4 shared/ - client+server)
import * as z from 'zod'
export const studentSchema = z.object({
  name: z.string().min(2),
  email: z.email(),
})
export type StudentInput = z.output<typeof studentSchema>
```

```
// server/api/students.post.ts
import { studentSchema } from '~/shared/schemas/student'
export default defineEventHandler(async (event) => {
  const body = await readBody(event)
  const result = studentSchema.safeParse(body) // serverda QAYTA
  validatsiya
  if (!result.success) {
    throw createError({ statusCode: 422, data: result.error.flatten() })
  }
  // result.data - ishonchli
  return { ok: true, student: result.data }
})
```

```
<!-- formada -->
<script setup>
import { studentSchema } from '~/shared/schemas/student'
const state = reactive({})
</script>
<template>
  <UForm :schema="studentSchema" :state="state" @submit="onSubmit">...
</UForm>
</template>
```

Bitta haqiqat manbai. Client UX uchun tez tekshiradi, server xavfsizlik uchun qayta tekshiradi (clientga ishonib bo'lmaydi). Laravel'da rule'lar faqat serverda edi — bu yerda ikkalasi bitta fayldan.

12 — UTable + Badge

```
<script setup lang="ts">
import { h, resolveComponent } from 'vue'
import type { TableColumn } from '@nuxt/ui'
const UBadge = resolveComponent('UBadge')

type Student = { id: number; name: string; group: string; status: 'active' | 'inactive' }
const data = ref<Student[]>([
  { id: 1, name: 'Ali Valiyev', group: 'Frontend-1', status: 'active' },
  { id: 2, name: 'Lola Karimova', group: 'Backend-2', status: 'inactive' },
])
const columns: TableColumn<Student>[] = [
  { accessorKey: 'id', header: 'ID' },
  { accessorKey: 'name', header: 'F.I.O' },
  { accessorKey: 'group', header: 'Guruh' },
  { accessorKey: 'status', header: 'Holat',
    cell: ({ row }) => h(UBadge,
      { color: row.original.status === 'active' ? 'success' : 'neutral',
        variant: 'subtle' },
      () => row.original.status === 'active' ? 'Faol' : 'Nofaol' ) },
  ]
</script>
<template>
  <UTable :data="data" :columns="columns" />
</template>
```

cell ichida `h(Component, props, slotFn)` — Vue render funksiyasi. `row.original` — shu qatarning xom obyekti.

17 — sidebar menyu

```
<script setup lang="ts">
import type { NavigationMenuItem } from '@nuxt/ui'
const items: NavigationMenuItem[] = [
  { label: 'Bosh sahifa', icon: 'i-lucide-house', to: '/' },
  { label: 'O\'quvchilar', icon: 'i-lucide-users', to: '/students', badge: '24' },
  { label: 'To'lovlar', icon: 'i-lucide-wallet', to: '/payments' },
  { label: 'Sozlamalar', icon: 'i-lucide-settings', children: [
    { label: 'Profil', to: '/settings/profile' },
    { label: 'Markaz', to: '/settings/center' },
    { label: 'Biljing', to: '/settings/billing' },
  ]},
]
</script>
<template>
  <UNavigationMenu :items="items" orientation="vertical" class="w-64" />
</template>
```

`badge` — yorliq (o'qilmagan/son), `children` — ochiluvchi ichki menyu. `to` bo'lgani uchun active holat avtomatik (Nuxt routing).



```
<!-- components/ConfirmModal.vue -->
<script setup lang="ts">
defineProps<{ title: string; description?: string }>()
const emit = defineEmits<{ close: [boolean] }>()
</script>
<template>
  <UModal :title="title" :description="description">
    <template #footer>
      <UIButton color="neutral" variant="outline" label="Yo'q"
@click="emit('close', false)" />
      <UIButton color="error" label="Ha, o'chirish" @click="emit('close',
true)" />
    </template>
  </UModal>
</template>
```

```
<script setup lang="ts">
import { LazyConfirmModal } from '#components'
const overlay = useOverlay()
const confirm = overlay.create(LazyConfirmModal)
const toast = useToast()

async function onDelete(student: { id: number; name: string }) {
  const instance = confirm.open({
    title: 'O\'chirishni tasdiqlang',
    description: `${student.name} o'chiriladi. Qaytarib bo'lmaydi.`
  })
  const ok = await instance.result
  if (ok) {
    await $fetch(`/api/students/${student.id}`, { method: 'DELETE' })
    toast.add({ title: 'O\'chirildi', color: 'success' })
  }
}
</script>
```

ConfirmModal butun ilovada bitta — har joyda await bilan ishlatasan. Trigger button modal ichida emas — bu moslashuvchanlik beradi.



22 — EduCore admin (yo'nalish)



`app/layouts/dashboard.vue` — yuqoridagi "Dashboard capstone" layout'ini ishlat (UDashboardGroup + Sidebar + Panel + Navbar).

`app/pages/students/index.vue` — "EduCore capstone" sahifasini ishlat: `definePageMeta({ layout: 'dashboard' })` + `UDashboardToolbar` (qidiruv + "Qo'shish") + `UTable` (mock `/api/students`) + `USlideover` ichida `UForm` (6/7-masaladagi forma).

Bog'lanish: - `server/api/students.get.ts` mock ro'yxat qaytaradi (08-modul). - Qo'shish: forma submit → `$fetch('/api/students', { method: 'POST', body })` → `refresh()`. - O'chirish: 20-masaladagi `useOverlay` confirm pattern.

Natija: to'liq ishlaydigan, brendlangan (09-modul), CRUD admin ekrani — Vue + Nuxt + Nuxt UI birga.



Yakun

Endi sen Nuxt UI bilan: - Tizimni tushunasan: `color / variant / size` + 3 qatlamli theming + `:ui` override (09). - 8 kategoriyadagi komponentlarni ishlata olasan: element, feedback, **form (Zod bilan)**, **table (TanStack)**, navigation, **overlay (useOverlay)**, layout/page, **dashboard**. - EduCore kabi SaaS admin panelini noldan emas, tayyor bloklardan yig'asan.

Komponentlarning **to'liq** prop/slot ro'yxati kerak bo'lganda —

`ui.nuxt.com/docs/components` doim ochiq tursin. Bu modul senga "nima borligini va qachon ishlatishni" berdi; hujjat esa "har bir mayda propni" beradi.



[README ga qaytish](#)